

Современные архитектуры нейронных сетей

Прокопов Е.М.

Содержание

I	Введение в глубокое обучение	6
1	Базовые архитектуры нейронных сетей	7
1.1	Полносвязные нейронные сети	7
1.2	Функции активации	8
1.3	Свёрточные нейронные сети	11
1.4	Рекуррентные нейронные сети	12
2	Обучение нейронных сетей	13
2.1	Функции потерь	13
2.2	Обратное распространение ошибки	17
2.3	Градиентный спуск	19
2.4	Регуляризация	23
2.5	Нормализация	24
II	Трансформеры	26
3	Токенизация	26
3.1	Посимвольная токенизация	26
3.2	Токенизация по словам	26
3.3	Byte-pair encoding (BPE)	27
3.4	Embedding-слой	28
4	Трансформер	29
4.1	Преобразование одной последовательности в другую	29
4.2	Внимание (Attention)	30
4.2.1	Мультипликативное внимание (Multiplicative attention)	30
4.2.2	Самовнимание (Scaled-dot Self-Attention)	30
4.2.3	Многоглавое внимание (Multihead Attention)	34
4.2.4	Маскированное многоглавое внимание (Masked Multihead Attention)	36
4.2.5	Перекрытое внимание (Cross Attention)	36
4.3	FFN-слой	37
4.4	Нормализация	37
4.5	Слой трансформера	38
4.5.1	Остаточные связи (Residual connections)	38
4.6	Позиционное кодирование	39
4.6.1	Абсолютное позиционное кодирование	40
4.6.2	Синусоидальное кодирование	40
4.7	Трансформер кодировщик и трансформер декодировщик	41
4.8	Обучение трансформера	43
4.8.1	Perplexity	44

4.9	Инференс трансформера	45
4.9.1	Температура трансформера	46
4.10	Масштабируемость трансформеров	46
5	Vision Transformer	46
5.1	Проблемы свёрточных нейронных сетей	46
5.2	Токенизация изображений	47
5.3	Обучаемое абсолютное позиционное кодирование	48
5.4	CLS токен	48
III	Обучение на GPU	50
6	Анатомия GPU	51
6.1	Streaming Multiprocessor	51
6.2	Потоки и варпы	52
6.3	CUDA-ядра	52
6.4	Тензорные ядра	53
6.5	Варп-планировщики	53
7	Иерархия памяти	54
7.1	Обмен данными между хостом и девайсом	54
7.2	Глобальная память (GMEM)	55
7.3	Разделяемая память (SMEM)	56
7.4	Память регистров (RMEM)	56
7.5	L1 и L2 кэш	56
7.6	Согласованный доступ к памяти	56
8	Метрики производительности GPU	57
8.1	Задержка и пропускная способность	58
8.2	FLOP/s	58
8.3	Пропускная способность памяти (memory bandwidth)	58
8.4	Арифметическая интенсивность	59
8.5	Roofline-модель	59
8.6	Occupancy	60
9	Типовые операции нейронных сетей на GPU	61
9.1	Поэлементные операции	61
9.2	Редукции	62
9.3	GEMV	62
9.4	GEMM	62
9.5	Нормализации	63

10 Программная модель CUDA	64
10.1 Технологический стек CUDA	64
10.2 CUDA-кernels	64
10.3 Потоки, блоки, сетка	64
10.4 Индексация	65
10.5 Синхронизации	65
10.6 Race Conditions	66
10.7 Редукции	66
10.8 Стримы и асинхронность	67
10.9 CUDA-события	67
11 Оптимизация вычислений нейронных сетей на GPU	69
11.1 Тайлинг в GEMM	69
11.2 Повышение арифметической интенсивности	71
11.3 Вычисления в пониженной точности	72
11.4 Переполнение и округление значений с плавающей точкой	73
11.5 Тензорные ядра в GEMM	74
11.6 Слияние кернелов (kernel fusion)	75
12 Flash Attention	76
12.1 Проблемы наивного внимания	76
12.2 Идея Flash Attention	77
12.3 Численно стабильный Softmax (Safe Softmax)	77
12.4 Online Softmax	78
12.5 QKV-тайлинг	79
12.6 Сравнение наивного внимания и Flash Attention	81
12.7 Маскированное внимание в Flash Attention	81
12.8 Обратное распространение в Flash Attention	82
13 Программная модель Triton	85
13.1 Зачем нужен Triton?	85
13.2 Кернел в Triton	86
13.3 Program Instance	86
14 Профилирование	87
14.1 Бенчмарк	87
14.2 Профилирование кернелов	88
14.3 Профилирование обучения	89
IV Введение в распределенное обучение	90
15 Связь нескольких GPU	90
15.1 Устройство GPU-ноды	90
15.2 Внутринодовая связь GPU	91
15.3 Межнодовая связь	91

15.4	Топология кластера	92
15.5	Стоимость коммуникаций	92
16	Коммуникации	93
16.1	Point-to-point коммуникации	93
16.2	Коллективные коммуникации	94
16.3	Объединение коллективных коммуникаций	96
17	NCCL	97
17.1	Модель исполнения	97
17.2	Коммуникационные каналы	98
17.3	Протоколы передачи данных	99
17.4	Алгоритмы коммуникаций	99
18	Параллелизм данных (DP)	101
18.1	Распределенный параллелизм данных (DDP)	101
18.2	ZeRO	102
18.3	FSDP	103
19	Тензорный параллелизм (TP)	104
19.1	Параллелизм по столбцам и строкам	104
19.2	Параллелизм последовательностей (SP)	107
20	Контекстный параллелизм (CP)	108
20.1	Кольцевое внимание	108
20.2	Зиг-заг кольцевое внимание	109
21	Конвейерный параллелизм (PP)	110
21.1	Зачем нужен PP	110
21.2	Стадии конвейерного параллелизма	111
21.3	AFAB	112
21.4	1F1B	113
22	Иерархия параллелизмов	113

Часть I

Введение в глубокое обучение

Несколько десятилетий назад, на заре зарождения компьютерных технологий решение задач компьютерами было строго алгоритмическим: после запуска программы следом за выполнением первой, заранее прописанной инструкции, следовало выполнение следующей, также заранее прописанной инструкции. И так до окончания работы программы.

Логика вызова этих инструкций определялась с помощью логических конструкций на основе простейших логических операций `if`, `else`, операторов `and`, `or` и так далее.

Такой подход способен автоматизировать множество задач. Например, таким образом возможно реализовать логику работы манипулятора на конвейерном производстве. Однако, таким образом очень сложно определять в данных какие либо паттерны, закономерности.

Оцифрованных данных в свою очередь, по мере развития технологий, становилось все больше и больше. Если есть аналитики, способные по определенным данным прогнозировать, например, стоимость недвижимости, почему нельзя научить это делать компьютер, используя статистические методы (как, например, линейная регрессия)?

Такие алгоритмы, которые вместо использования жестких, заранее прописанных правил, способны обучаться на имеющихся данных и извлекать из них закономерности (зачастую даже не видные человеку), являются алгоритмами машинного обучения.

Машинное обучение делится на несколько разделов, из них можно выделить два основных: обучение с учителем и обучение без учителя.

В ходе обучения с учителем модель учится на парах данных “вход” (обозначим его x) и “правильный ответ” (обозначим его y). Такие данные называются размеченными, поскольку для определения правильного ответа при составлении набора данных требуется участие эксперта-разметчика (однако, есть и исключения, о которых мы поговорим позднее).

При обучении без учителя модель работает с не размеченными данными и самостоятельно пытается обнаружить в них закономерности. Глубокое обучение, наоборот, по большей части, является подмножеством обучения с учителем. Оно основано на глубоких, то есть многослойных, нейронных сетях. Глубокие нейронные сети позволяют определять сложные высокоуровневые признаки через распознавание на каждом слое более простых низкоуровневых признаков и их комбинаций.

1 Базовые архитектуры нейронных сетей

1.1 Полносвязные нейронные сети

Наиболее распространенным слоем среди нейронных сетей является полносвязный линейный слой. В этом слое каждый нейрон связан с каждым нейроном предыдущего слоя. Связи между нейронами, веса, являются обучаемыми параметрами нейронной сети. Каждый нейрон вычисляет взвешенную сумму входов:

$$y_j = \sum_i x_i w_{ij}$$

Запишем выход полносвязного слоя в виде вектора y :

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_d \end{pmatrix}^T = \begin{pmatrix} \sum_{j=1} x_j w_{1j} \\ \sum_{j=1} x_j w_{2j} \\ \dots \\ \sum_{j=1} x_j w_{dj} \end{pmatrix}^T = xW$$

Рассматривая выражение $y = xW$ можно заметить, что оно описывает гиперплоскость, проходящую через начало координат. Поэтому, при $x = 0$ всегда $y \equiv 0$ при любых параметрах. Соответственно, при входе $x = 0$ результат глубокой сети, состоящей из полносвязных слоёв, всегда будет нулевым. Добавив в выражение вектор $b \neq 0$ обучаемых параметров

$$y = xW + b,$$

мы получим гиперплоскость, способную проходить где угодно в пространстве признаков.

Попробуем построить сеть из двух слоёв f_1 и f_2 , описываемых параметрами $\{W_1, b_1\}$ и $\{W_2, b_2\}$ соответственно:

$$y_1 = xW_1 + b_1, \quad y_2 = y_1W_2 + b_2$$

Подставим y_1 в выражение для y_2 :

$$y_2 = (xW_1 + b_1)W_2 + b_2 = xW_1W_2 + b_1W_2 + b_2 = x\tilde{W} + \tilde{b}$$

Мы получили из двух слоёв один. Получается, это значит, что глубокие нейронные сети невозможны? "Схлопывание" слоёв происходит из-за линейности функции $y = xW + b$. Для линейной функции выполняется выражение

$$f(ax + by) \equiv af(x) + bf(y),$$

а композиция линейных функций — линейная функция. Поэтому, чтобы сделать глубокие сети возможными, добавим в композицию слоёв нелинейную функцию ϕ :

$$y_1 = \phi(f_1(x)), \quad y_2 = \phi(f_2(y_1))$$

После подстановки, выражение не удастся упростить:

$$y_2 = \phi(\phi(xW_1 + b_1)W_2 + b_2)$$

Нелинейную функцию ϕ называют функцией активации. Нелинейность функции активации также позволяет нейронной сети проще аппроксимировать сложные нелинейные преобразования. От функции активации также должна существовать производная, это свойство потребуется нам в дальнейшем.

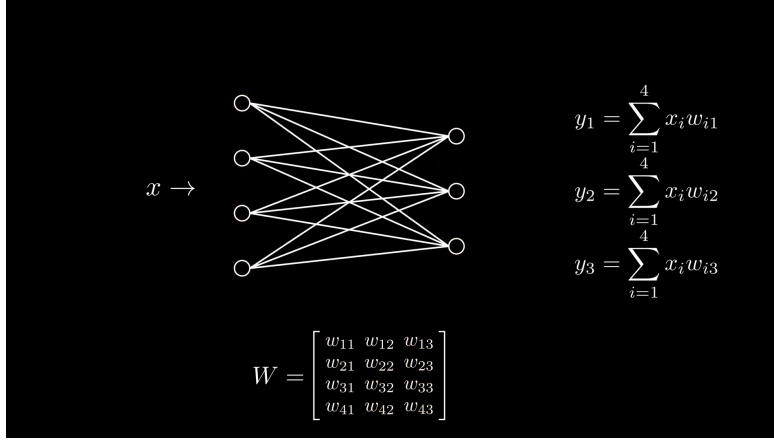


Рис. 1: Схема полносвязного слоя.

Полносвязная нейронная сеть (fully connected network) также называется многослойным перцептроном (multilayer perceptron, MLP) или сетью прямого распространения (feedforward network, FFN).

1.2 Функции активации

Исторически, одними из первых широко использовавшихся функций были сигмоида:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

и гиперболический тангенс:

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

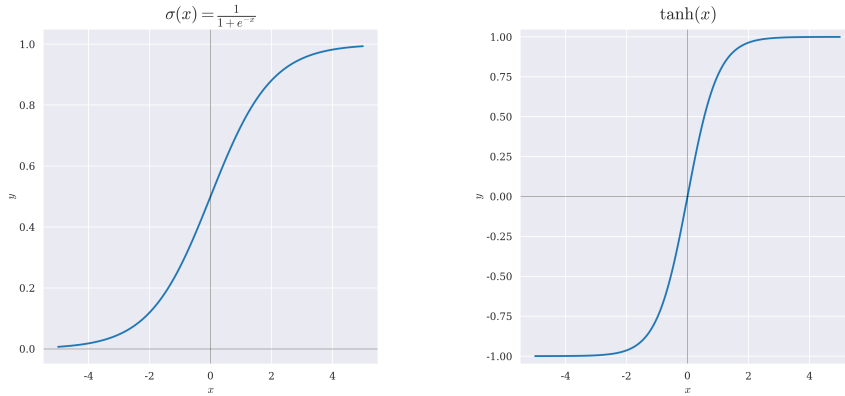


Рис. 2: Графики функций активации сигмоиды (слева) и гиперболического тангенса (справа).

Однако обе функции имеют большой недостаток: при больших по модулю входных значениях они "насыщаются", то есть их значение почти не изменяется при изменении аргумента. Из-за этого их производная будет близка к нулю. О том, почему это плохо, мы поговорим далее.

Сигмоида отражает входное значение в диапазон от 0 до 1, благодаря чему его удобно интерпретировать как вероятность.

На смену сигмоиде и гиперболическому тангенсу в качестве функции активации пришла функция ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0, \\ 0, & x < 0 \end{cases}$$

ReLU нелинейна на $x \in \mathbb{R}$, даже несмотря на линейность лучах $x \geq 0$ и $x < 0$. Ее главное преимущество заключается в простоте вычислений, а также в простоте производной. Но в производной кроется и её главный недостаток: при $x < 0$ производная $\text{ReLU}'(x) \equiv 0$.

Эта проблема решается в модификации ReLU — Leaky ReLU:

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \quad \alpha \neq 0 \end{cases}$$

Обычно, коэффициент α берут небольшим, например $\alpha = 0.1$. Так мы сохраняем нелинейность функции и решаем проблему нулевого градиента.

В PReLU идея развивается дальше и коэффициент α становится обучаемым параметром.

Такие функции как ELU устроены сложнее:

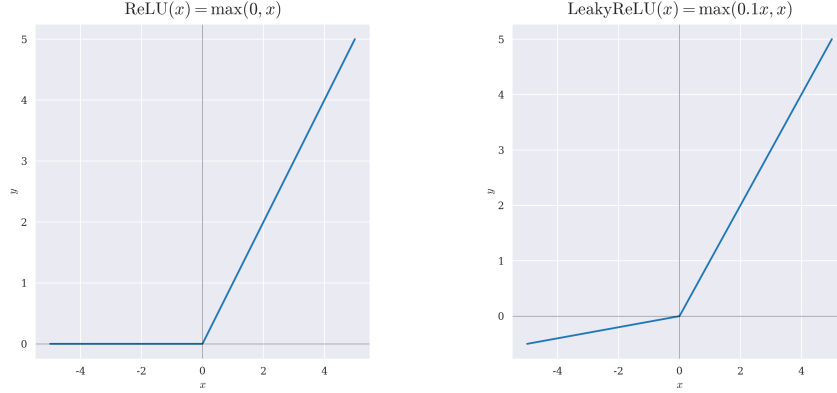


Рис. 3: Графики функций активации ReLU (слева) и LeakyReLU (справа).

$$\text{ELU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha * (\exp(x) - 1), & x < 0 \end{cases}$$

Их идея заключается в том, чтобы сделать отрицательную часть не линейной, а плавной, чтобы получить более гладкую функцию и, за счёт этого, улучшить динамику обучения (правда, пожертвовав простотой вычислений).

Широкое распространение также получила функция SiLU, поскольку является непрерывно гладкой на $\mathbb{R}$:

$$\text{SiLU}(x) = x\sigma(x)$$

И ее обобщение — функция активации SiLU_β :

$$\text{SiLU}_\beta(x) = x\sigma(\beta x)$$

Устремление коэффициента β к бесконечности позволяет превратить SiLU_β в ReLU.

Отдельно стоит рассмотреть функцию Softmax. Формально, она также является функцией активации:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{\dim(x)} \exp(x_j)}$$

Её главная особенность заключается в том, что сумма всех компонент вектора, пропущенного через Softmax, равна 1, что позволяет трактовать этот вектор как многомерное распределение вероятностей. Softmax обычно используют при решении задачи многоклассовой классификации.

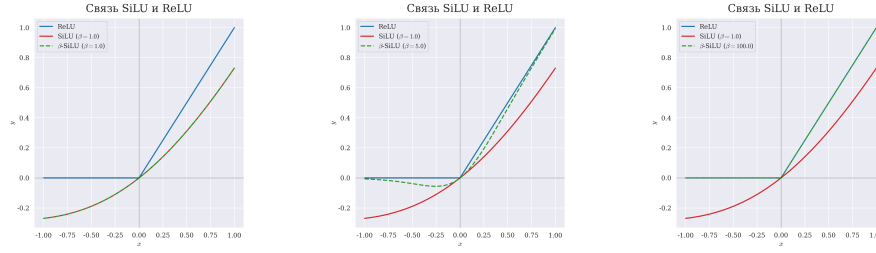


Рис. 4: График функции активации SiLU_β при разных значениях β .

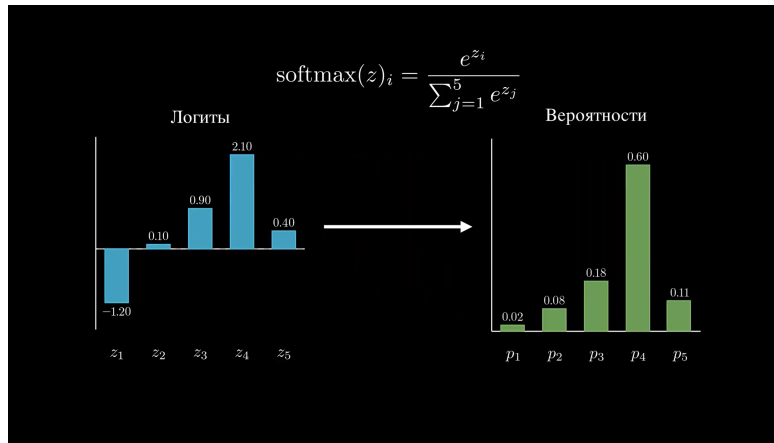


Рис. 5: Преобразование распределения логитов в распределение вероятностей с помощью Softmax.

1.3 Свёрточные нейронные сети

Полносвязные нейронные сети не учитывают пространственных связей в данных. Например в таких, как изображения или текст. Во время обработки, изображение выпрямляется в вектор, после чего каждая компонента обрабатывается как отдельный признак. При этом, если перемешать пиксели изображения, а также соответствующим образом перемешать веса нейронной сети, результат обработки не изменится.

Для обработки таких данных, в которых важна информация о пространственном взаиморасположении признаков, используются свёрточные нейронные сети (convolutional neural network, CNN). Такие сети состоят из свёрточных слоёв. Эти слои применяют к данным фильтры - однослойные полносвязные сети, преобразующие участок данных в число.

В одномерном, простейшем, случае при обработке последовательности данных фильтром размера 3, на первом шаге обрабатываются первые 3 элемента, после чего фильтр смещается на некоторый шаг свёртки (stride)

дальше и обрабатывает еще 3 элемента последовательности. Когда последовательность завершается, из результатов обработки формируется карта признаков — новая последовательность, которую также можно обработать свёрточным слоем.

При обработке двумерного одноканального изображения обрабатывается участок размером 3×3 . В этом случае сеть-фильтр имеет 9 параметров (10 в случае наличия смещения). Когда фильтр доходит до края изображения, он опускается вниз на шаг свёртки и смещается к левому краю. По завершению обработки двумерных данных карта активации получается двумерной.

При обработке многоканального изображения, каждый канал сначала обрабатывается по отдельности, с использованием своего фильтра, а затем полученные карты активаций складываются. Для получения ещё одной карты признаков исходные каналы обрабатываются независимо с использованием новых фильтров.

Важной особенностью свёрточных слоёв является то, что они применяют одинаковый набор параметров к каждому участку изображения. Таким образом, если на изображении присутствует полезный признак (например, наличие вертикальных или горизонтальных линий), то неважно, в какой части он находится, он все равно будет обработан одинаково. Это свойство называется инвариантностью сдвига.

Для свёрточных сетей, также как и для полносвязных, важно использование функции активации.

Отдельно стоит упомянуть выбор размера фильтра. Оказывается, что при использовании соответствующего размера паддинга (заполнения нулями краёв изображения), последовательная обработка изображения двумя свёрточными слоями с фильтрами 3×3 может быть сведена к обработке изображения свёрточным слоем с фильтром 5×5 и наоборот. Аналогично с тремя слоями с фильтрами 3×3 и слоем с фильтром 7×7 . Однако, в первом случае возможно использование дополнительно двух функций активации и всего 20 параметров, а во втором — только одной функции активации при 26 параметрах. Таким образом очевидно, что выгоднее использовать два слоя с фильтрами 3×3 , чем один слой с фильтрами 5×5 , поскольку большее число функций активации способствует большей выразительности нейронной сети. При этом два таких слоя выгоднее по количеству параметров сети.

1.4 Рекуррентные нейронные сети

Рекуррентные нейронные сети (RNN) предназначены для работы с последовательными данными, где присутствует зависимость между текущим входом и предыдущими. Такими данными могут быть, например, текст или временные ряды.

$$\begin{cases} h_t = \phi(x_t W + h_{t-1} U + b_1), \\ y_t = \varphi(h_t V + b_2) \end{cases}$$

Главная особенность RNN - наличие обратных связей: сеть способна передать информацию из предыдущих элементов последовательности. Это можно представить в следующем виде:

W , U , V - матрицы параметров, ϕ , φ - активации, b_1 , b_2 - смещения.

h_t - в некотором смысле “память” о предыдущих элементах последовательности. Однако “память” RNN очень короткая и не способна запоминать долгосрочные зависимости. С этой задачей лучше справляется другая рекуррентная архитектура - LSTM.

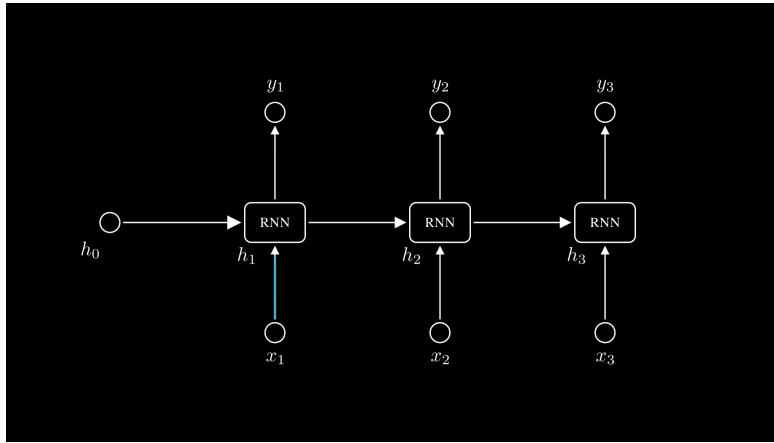


Рис. 6: Схема RNN слоя.

2 Обучение нейронных сетей

Нейронные сети обучаются, изменяя свои параметры таким образом, чтобы минимизировать функцию потерь L . Задача обучения становится оптимизационной задачей, и для ее решения будем использовать инструменты из математической оптимизации, а конкретно градиентные оптимизаторы.

Процесс обучения можно разделить на следующие этапы: прямое распространение (forward pass), вычисление **функции потерь** (loss-функции) и обратное распространение (backpropagation) для обновления весов. Данный алгоритм обучения называется алгоритмом **обратного распространения ошибки**.

2.1 Функции потерь

Функция потерь — это оценка стоимости ошибки модели. Пусть y — истинные значения, а $\hat{y}$ - предсказания модели. Простейший способ оценить ошибку — это взять модуль разности по всем объектам и просуммировать их:

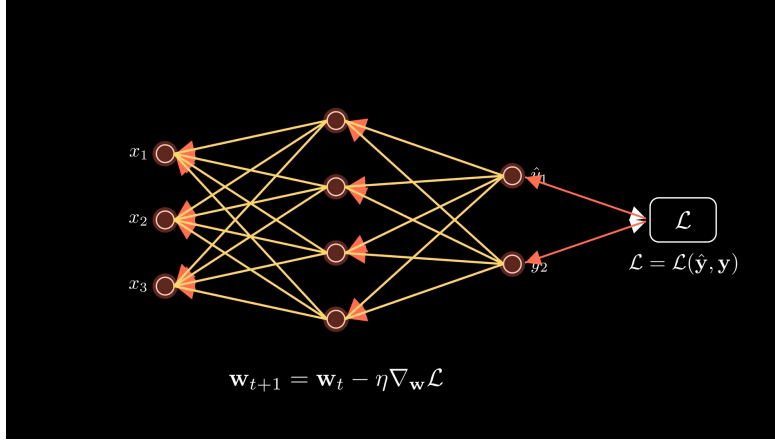


Рис. 7: Обратное распространение ошибки.

$$\text{Absolute Error}(y, \hat{y}) = \sum_{i=1}^N |y_i - \hat{y}_i|$$

Поскольку перед нами стоит задача оптимизации, мы можем применять монотонные возрастающие преобразования и точки минимумов и максимумов не изменятся. Поэтому, разделим абсолютную ошибку на количество объектов, по которым считали ошибку, и получим среднюю абсолютную ошибку (Mean Absolute Error, MAE):

$$\text{MAE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Также используется средне квадратичная ошибка в качестве функции потерь (Mean Squared Error, MSE). Эта функция сильнее штрафует большие ошибки и слабее — малые.

$$\text{MSE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Какую же функцию потерь выбрать? Пусть данные устроены следующим образом:

$$y = f(x) + \varepsilon,$$

где ε - случайная величина, шум.

Предположим, что шум центрирован и распределен нормально:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Если мы зафиксируем значение x , то значение y тоже будет распределено нормально:

$$y|x \sim \mathcal{N}(f(x), \sigma^2)$$

Запишем плотность такого распределения:

$$p(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{y - f_w(x)}{\sigma} \right)^2 \right]$$

Теперь нам нужно понять, какая функция будет лучше соответствовать модели шума ε . Для этого воспользуемся методом максимального правдоподобия.

Необходимо максимизировать величину, называемую правдоподобием:

$$l = \prod_{i=1}^N p(y_i|x_i)$$

Правдоподобие показывает, насколько хорошо выбранные параметры объясняют наблюдаемые данные. В случае, если ответы y - дискретная величина, мы можем говорить, что правдоподобие равно произведению вероятностей выбора правильного ответа моделью.

Произведение оптимизировать неудобно, поэтому вместо него рассматривают логарифм правдоподобия, чтобы превратить произведение в сумму. Также, обычно рассматривается не задача максимизации $\log(l)$, а задача минимизации $-\log(l)$:

$$\text{NLL} = -\sum_{i=1}^N \log p(y_i|x_i) = -\log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2} \left(\frac{y - f_w(x)}{\sigma} \right)^2 \right] \right)$$

Упростим:

$$\text{NLL} = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - f(x))^2}{2\sigma^2}$$

Здесь $\frac{1}{2} \log(2\pi\sigma^2)$ - это константа, поэтому от нее можно избавиться.

Остаток домножим на $\frac{2\sigma^2}{N}$ (потому что можем это сделать) и получим эквивалентную задачу оптимизации

$$w^* = \operatorname{argmin}_w (\text{NLL}) = \operatorname{argmin}_w \left(\frac{1}{N} \sum_{i=1}^N (y - \hat{y})^2 \right)$$

Таким образом, если шум в данных распределен нормально, то наиболее оптимальная функция потерь — это MSE. Если же шум в данных имеет распределение Лапласа, то наиболее оптимальной функцией потерь будет MAE.

В задаче регрессии шум чаще всего распределен нормально. Шум, распределенный Лапласово, встречается, также может встретиться в задаче регрессии, но чаще его можно обнаружить, например, в задаче генерации изображений.

А вот для задачи классификации не подойдут ни MAE, ни MSE, поскольку шум в такой задаче имеет категориальное распределение. Чтобы определить оптимальную функцию потерь, воспользуемся теорией информации.

Пусть имеется C классов. Тогда для объекта x модель f выдает вектор вероятностей

$$q(x) = (q_1(x), q_2(x), \dots, q_C(x))$$

Если некоторое событие X произошло с вероятностью p , то количество информации, которое несёт это событие (мера самоинформации), равно

$$I[X] = -\log p$$

отсюда видно, что маловероятное событие несет больше информации, чем вероятное. Математическое ожидание информации в одном наблюдении называется энтропией (Шенноновской энтропией):

$$H(p) = \mathbb{E}[I[X]] = -\sum_{i=1}^C p_i \log p_i$$

Теперь предположим, что истинное распределение классов — p , а модель предсказывает распределение q . Тогда среднее количество информации, которое потребуется для описания истинных ответов с использованием распределения q , задаётся кросс-энтропией:

$$H(p, q) = -\sum_{i=1}^C p_i \log q_i$$

В задаче обучения с учителем p — это вектор, у которого $p_c = 1$ для правильного класса, а для остальных $p_i = 0$. Тогда можем упростить:

$$H(p, q) = -\log q_c,$$

где q_c — вероятность, которую модель присвоила правильному классу.

Если мы решим применить метод максимального правдоподобия для решения задачи классификации, то для одного объекта получим:

$$\text{NLL} = -\log q_c = H(p, q)$$

Значит мы можем записать функцию потерь кросс-энтропии:

$$L_{CE} = \frac{1}{N} \sum_{n=1}^N H(p^{(n)}, q^{(n)}) = -\frac{1}{N} \sum_{n=1}^N \log q_{c^{(n)}}^{(n)}$$

Но что эта функция значит? Какая интуиция? Вернемся к полной формуле кросс-энтропии и немного перепишем её:

$$H(p, q) = - \sum_i p_i \log q_i = - \underbrace{\sum_i p_i \log p_i}_{H(p)} + \underbrace{\sum_i p_i \log \frac{p_i}{q_i}}_{D_{KL}\langle p||q \rangle},$$

Таким образом:

$$H(p, q) = H(p) + D_{KL}\langle p||q \rangle,$$

где:

- $D_{KL}\langle p||q \rangle$, — расхождение Кульбака-Лейблера, "расстояние" от распределения p к распределению q (на самом деле это не расстояние, поскольку расхождение Кульбака-Лейблера не обладает свойством симметричности).
- $H(p)$ — Шеннонова энтропия данных, мера "шума". Это — теоретический минимум для корректной функции потерь.

Таким образом, минимизация кросс-энтропии означает минимизацию расхождения Кульбака-Лейблера, а значит приближает моделируемое распределение q к истинному распределению p .

2.2 Обратное распространение ошибки

Пусть f - функция, которая представляет нашу нейронную сеть, а $f_1, f_2, \dots, f_n$ — ее слои. Этап прямого распространения тогда записывается просто:

$$y = f(x) = f_1 \circ f_2 \circ \dots \circ f_n(x)$$

После прямого распространения и вычисления функции потерь, необходимо вычислить градиент $\nabla L(y, \hat{y})$. После чего используется алгоритм градиентного спуска для обновления параметров:

$$w_t = w_{t-1} - \lambda \nabla L_{w_{t-1}}(y, \hat{y})$$

Этап обратного распространения заключается в вычислении градиента ∇L_w . Градиент — это вектор частных производных:

$$\nabla L_w = \left(\frac{\partial L}{\partial w_1} \quad \frac{\partial L}{\partial w_2} \quad \dots \quad \frac{\partial L}{\partial w_N} \right)$$

Переменные $w_1, w_2, \dots$ — это параметры нейронной сети, а именно веса конкретных слоёв. Допустим, $w_1 = (W_1)_{ij}$ - это параметр слоя f_1 , а $w_2 = (W_2)_{ij}$ - параметр слоя f_2 .

Найдем $\frac{\partial L}{\partial w_1}$. Для этого продифференцируем сложную функцию с помощью цепного правила:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w_1}$$

$\frac{\partial L}{\partial y}$ — это просто производная функции потерь, вычисляется достаточно легко. Найдем производную выхода слоя по параметру $\frac{\partial y}{\partial w_1}$.

Пусть

$$z_1 = f_2 \circ \dots \circ f_n(x)$$

Тогда:

$$y = f_1 \circ f_2 \circ \dots \circ f_n(x) = f_1(z_1) = z_1 W_1 + b_1$$

Поскольку y — это вектор, то только одна его компонента зависит от элемента $w_1 = (W_1)_{ij}$, и это компонента y_j . Вычислим её:

$$y_j = \sum_k z_{1k} (W_1)_{kj} + b_{1j}$$

Теперь найти $\frac{\partial y}{\partial w_1}$ достаточно просто:

$$\frac{\partial y_k}{\partial w_1} = 0 \quad \forall k \neq j,$$

$$\frac{\partial y_j}{\partial w_1} = z_{1i}$$

Значит, векторная производная:

$$\frac{\partial y}{\partial w_1} = z_{1i} e_j,$$

где e_j — это единичный вектор, все компоненты которого, кроме компоненты на j -ой позиции, равны 0.

Теперь аналогичным образом попробуем найти $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_2}$$

Разберем это выражение по порядку. Производная $\frac{\partial z_1}{\partial w_2}$ нам знакома — это производная выхода второго слоя по параметру, мы только что вычисляли аналогичную производную для первого слоя, дальнейшие математические выкладки остаются внимательному читателю в качестве самостоятельной работы.

А производная $\frac{\partial L}{\partial z_1}$ нам не знакома. Снова воспользуемся цепным правилом:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z_1}$$

Производную $\frac{\partial L}{\partial y}$ мы уже считали. А вот $\frac{\partial y}{\partial z_1}$ — это производная выхода слоя f_1 по его входу (а вход f_1 слоя — это выход f_2 слоя). Таким образом становится понятно, что:

1. На каждом слое, исключая f_n , нам необходимо считать 2 производные $\frac{\partial L}{\partial w}$ и $\frac{\partial L}{\partial z}$ — производную по параметру, которая нам нужна для вычисления градиента, и производную по входу, чтобы с помощью нее вычислить производные следующего слоя.
2. Невозможно вычислить производные слоя f_k , не вычислив производные слоев $f_1, f_2, \dots, f_{k-1}$. Это ограничение задает строгое направление обратного распространения ошибки.
3. Для вычисления $\frac{\partial L}{\partial w_k}$, производной на k -ом слое, нам необходимо найти произведение $\frac{\partial L}{\partial y} \frac{\partial y}{\partial z_1} \frac{\partial z_1}{\partial z_2} \frac{\partial z_2}{\partial z_3} \dots \frac{\partial z_{k-2}}{\partial z_{k-1}}$.

Если каждая производная $\frac{\partial y}{\partial z}$ этого произведения будет меньше 1 по модулю, например $\frac{\partial y}{\partial z} = 0.5$, то для седьмого слоя мы получим множитель $(0.5)^6 \frac{\partial L}{\partial y} \approx 0.016 \frac{\partial L}{\partial y}$.

Эта проблема называется проблемой угасающих градиентов (или исчезающих градиентов). В некоторых случаях возможна противоположная ситуация — градиенты "взрываются". Из-за этого мы (пока) не можем обучать по настоящему глубокие сети.

Для вычисления на компьютере алгоритма обратного распространения ошибки строится граф вычислений. Узлы этого графа — простейшие операции, такие как сложение, умножение, деление и тд, а ребра отображают их последовательность. Мы можем создавать собственные операции, объединяя однотипные элементы в один узел, тем самым упрощая граф. Во время прямого распространения выполнение операций осуществляется последовательно, а при вычислении градиента — в обратном порядке.

2.3 Градиентный спуск

Вычислив градиент ∇L_w , мы можем обновить параметры нейронной сети с помощью градиентного спуска.

Стандартный алгоритм градиентного спуска вычисляется по всей выборке:

$$w_{t+1} = w_t - \lambda \nabla L_{w_t},$$

$$\nabla L_{w_t} = \sum_{i=1}^N \nabla l_{w_t}(y_i, \hat{y}_i)$$

где N - количество объектов выборки.

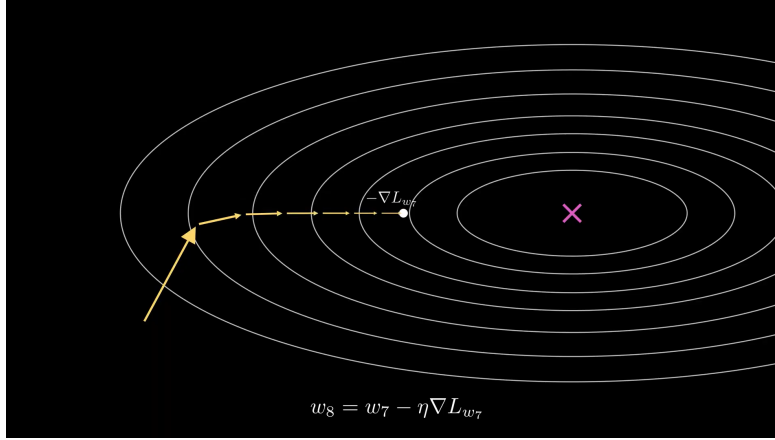


Рис. 8: Траектория градиентного спуска.

Очевидный недостаток такого подхода — потребность в огромном количестве памяти и вычислительных ресурсов. Не самый очевидный недостаток — высокая вероятность попадания в локальные минимумы или седловые точки функции ошибки.

Стохастический градиентный спуск

Решение — реализуем градиентный спуск не по всему набору данных, а только по его части — по мини-батчу. Объекты в мини-батче выбираются случайным образом, делая процесс стохастическим. Такой оптимизатор называется стохастическим градиентным спуском (SGD).

$$w_{t+1} = w_t - \lambda \nabla L_{w_t},$$

$$\nabla L_{w_t} = \sum_{i=1}^B \nabla l_{w_t}(y_i, \hat{y}_i)$$

где B - количество объектов минибатча.

Реальные данные чаще всего шумные, а значит и градиент при вычислении SGD также будет шумным. Чем больше размер батча, тем точнее градиент. Но также этот шум позволяет избегать седловых точек и некоторых локальных минимумов.

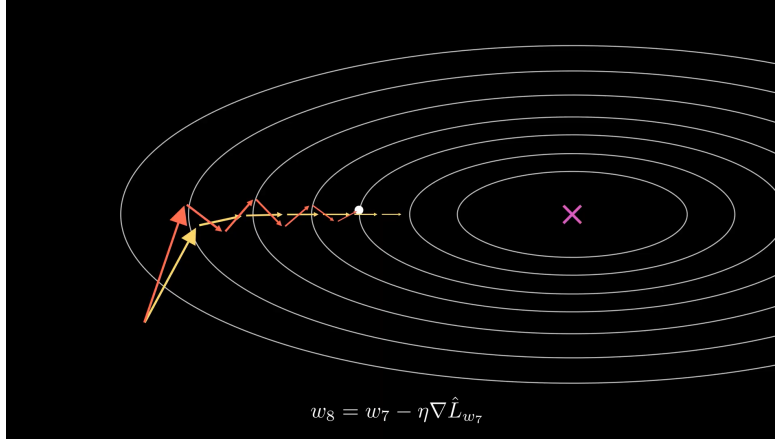


Рис. 9: Траектория стохастического градиентного спуска.

Momentum

Градиентный спуск можно представить как шарик, скатывающийся по поверхности функции. Шарик, будучи объектом физическим, обладает скоростью. Реализация "скорости" в SGD (через добавление изменения весов за предыдущий шаг) называется momentum:

$$w_{t+1} = w_t - \lambda \nabla L_{w_t} + \alpha(w_t - w_{t-1})$$

Это выражение можно записать иначе:

$$\begin{cases} v_{t+1} = \alpha v_t - \lambda \nabla L_{w_t}, \\ w_{t+1} = w_t + v_{t+1} \end{cases}$$

Nesterov momentum Для momentum существует модификация — Nesterov momentum. Его идея заключается в том, чтобы считать градиент не в текущей точке, а в точке, в которую попали бы, следуя импульсу:

$$\begin{cases} v_{t+1} = \alpha v_t - \lambda \nabla L_{w_t + \alpha v_t}, \\ w_{t+1} = w_t + v_{t+1} \end{cases}$$

Данный момент как бы "заглядывает в будущее", корректируя ошибку оптимизации на текущем шаге.

Скорость обучения

Вычисление градиента — дорогая задача, поэтому хотелось бы найти оптимальные параметры нейронной сети за как можно меньшее число шагов градиентного спуска. Конечно, momentum и nesterov momentum ускоряют обучение сети, однако на скорость обучения больше всего влияет гиперпараметр λ . Чем больше λ , тем быстрее обучается модель. Однако в конце обучения хотелось бы, чтобы шаг был как можно меньше, чтобы добиться большей точности.

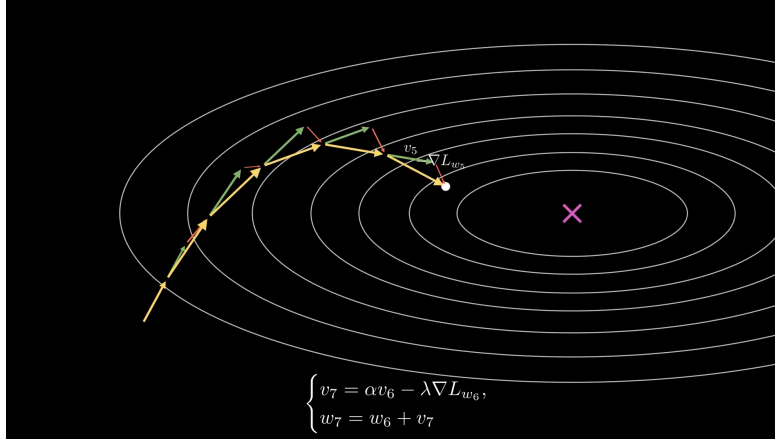


Рис. 10: Траектория momentum.

Пусть λ — это некоторая функция $\lambda(t)$. Тогда возможно просто подобрать подходящую функцию, чтобы значение λ соответствовало нашим желаниям. Такая техника называется **learning rate schedule**.

Стоит отметить, что параметры модели инициализируются случайным образом и градиенты на первых шагах могут быть слишком шумными. Тогда, большой шаг обучения может отбросить параметры в “плохую” часть функции потерь. Поэтому, часто скорость обучения запускают с малых, околонулевых, значений, постепенно, разогревая до ”пика”, после чего она начинает затухать. Такая техника называется **warm-up**.

Adagrad

Learning rate schedule не идеален, поскольку основан на предположении, что заранее известно, на каких участках шаг обучения должен быть большим, а на каких - малым. Поэтому существуют методы адаптивной настройки скорости обучения. Первый такой метод — **Adagrad**:

$$\begin{cases} r_{t+1} = r_t + (\nabla L_{w_t})^2, \\ w_{t+1} = w_t - \frac{\lambda}{\sqrt{r_{t+1} + \epsilon}} \nabla L_{w_t} \end{cases}$$

Этот метод старается компенсировать слишком большие и слишком малые компоненты градиента.

RMSProp

Модификация Adagrad — RMSProp (root mean square propagation). Идея этого оптимизатора заключается в том, чтобы не складывать градиенты, а брать скользящее среднее:

$$\begin{cases} r_{t+1} = \beta r_t + (1 - \beta) \left(\nabla L_{w_t} \right)^2, \\ w_{t+1} = w_t - \frac{\lambda}{\sqrt{r_{t+1} + \epsilon}} \nabla L_{w_t} \end{cases}$$

Adam

Дело осталось за малым — объединить RMSProp и momentum и получить Adam (Adaptive momentum):

$$\begin{cases} v_{t+1} = \beta_1 v_t + (1 - \beta_1) \nabla L_{w_t}, \\ r_{t+1} = \beta_2 r_t + (1 - \beta_2) \left(\nabla L_{w_t} \right)^2 \end{cases}$$

Поскольку v_0 и r_0 инициализируются нулями, то в их оценке есть смещение. Для их коррекции в Adam присутствует следующий шаг:

$$\begin{cases} \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_1}, \\ \hat{r}_{t+1} = \frac{r_{t+1}}{1 - \beta_2} \end{cases}$$

Следом идёт само обновление весов:

$$w_{t+1} = w_t + \lambda \frac{\hat{v}_{t+1}}{\sqrt{\hat{r}_{t+1} + \epsilon}}$$

Adam является серебряной пулей при оптимизации параметров нейронной сети.

2.4 Регуляризация

Нейронные сети обучаются на некотором тренировочном наборе данных, однако работать им приходится с теми данными, которые они, скорее всего, не видели (тестовые данные). При вычислении функции потерь на разных наборах данных возможны следующие сценарии: высокая ошибка как на тренировочном, так и на тестовом наборах данных, либо высокая ошибка на тестовых данных, но на тренировочных — почти нулевая. Говорят, что в первом случае модель недообучилась, а во втором — переобучилась.

Пусть данные генерируются с помощью следующего генератора:

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Поскольку реальные данные шумные, то всегда будет присутствовать шум, который невозможно предсказать на тестовых данных. Высокая тестовая ошибка может быть результатом недостаточной “подгонки” модели, если она слишком простая (или мало обучалась) и не способна улавливать связи в данных. Также, тестовая ошибка может быть обеспечена слишком точным следованием за шумом на тренировочных данных, из-за чего итоговая функция не способна дать правильное предсказание на тестовых данных.

Среднеквадратичная ошибка может быть разложена на три составляющие:

$$\text{expected loss} = (\text{bias})^2 + \text{variance} + \text{noise}$$

Bias (смещение) — это систематическая ошибка модели, а **variance** (разброс) — оценка чувствительности модели к изменчивости данных. Недообучение выражается в высоком смещении и низком разбросе. Переобучение — в высоком разбросе и низком смещении. Хорошо обученная модель — компромисс между смещением и разбросом (**bias-variance tradeoff**).

Чтобы уменьшить bias, нужно увеличить количество параметров модели.

Уменьшить variance возможно через уменьшение сложности модели. В этом помогают методы регуляризации. При этом будет увеличиваться bias. Регуляризация позволяет достичь компромисса для более сложной модели на тех же данных. При обучении нейронных сетей чаще всего используются l_1 и l_2 регуляризации (в общем случае - **weight decay**), идея которых заключается в добавлении нормы весов как дополнительного компонента функции потерь:

$$L_{reg1} = L + c\|w\|_1,$$

$$L_{reg2} = L + c\|w\|_2^2,$$

Weight decay “сдерживает” веса нейронной сети у нуля, уменьшая тем самым сложность модели. Сложность модели уменьшает и другой метод регуляризации, называемый **dropout**. Основная идея dropout — зануление некоторых выходов слоя случайным образом.

Чтобы понизить variance, не уменьшая сложность модели, необходимо расширить набор данных. Если добавить новые объекты не представляется возможным, можно воспользоваться **аугментацией данных**.

2.5 Нормализация

Данные имеют разный масштаб. Некоторые признаки могут быть близки к нулю, некоторые — колебаться вблизи некоторого числа. Большие по модулю значения модель будет считать важнее, даже если это не так. Для избежания этого данные нормализируются:

$$\tilde{x}^i = \frac{x^i - \mu^i}{\sigma^i + \epsilon}, \quad \mu^i = \frac{1}{N} \sum_{n=1}^N x_n^i, \quad \sigma^i = \sqrt{\frac{1}{N} \sum_{n=1}^N (x_n^i - \mu^i)^2}$$

Однако, распределение входов каждого слоя модели (кроме первого) также будет изменяться на каждом шаге обучения, поскольку изменяются параметры модели. Это означает, что каждому слою придется заново

переучиваться на новое распределение. Для решения этой проблемы применяется нормализация по батчу:

$$\hat{z} = \frac{z - \mu_B}{\sigma_B + \epsilon}$$

Но нормализуя данные таким образом, мы уменьшаем предсказательную способность сети. Поэтому результат ремасштабируется с помощью обучаемых параметров γ и β :

$$\tilde{z} = \gamma \hat{z} + \beta$$

Позже, интерпретацию с изменением распределений входов слоев стали считать недостаточной. Было выяснено, что эффект успешной нормализации заключается в том, что поверхность функции потерь становится более гладкой, а градиенты - более предсказуемыми.

Часть II

Трансформеры

3 Токенизация

Нейронные сети не способны обрабатывать текстовые данные в “сыром” виде, поскольку спроектированы для работы с числовыми тензорами. Текст представлен последовательностью символов, таких как буквы, цифры, пробелы, знаки пунктуации и специальные символы. Какие-то из этих элементов встречаются чаще, какие-то — реже. Сами тексты, очевидно, могут быть разной длины.

Процесс разбиения текста на последовательность элементов называется **токенизацией**, а сами эти элементы — токенами. Все уникальные токены сводятся в словарь, где им присваивается собственный идентификатор. Это обеспечивает важное свойство токенизации — обратимость. Благодаря этому свойству, исходный текст в неизменном виде может быть декодирован из последовательности токенов.

3.1 Посимвольная токенизация

Самой первой идеей, которая приходит в голову, как разделить текст на последовательность, является посимвольное разбиение. Исходный текст итак можно представить в виде последовательности идентификаторов. Например, мы можем взять таблицу кодировки utf как словарь.

Поскольку уникальных символов немного (за исключением некоторых языков), размер итогового словаря будет небольшой, что благоприятно отразится на размере модели. Однако, если такой подход и будет успешен при обработке иероглифических языков, таких как китайский, японский или, например, древнешумерский, то при работе с другими языками могут возникнуть проблемы. При разбиении текста на символы, количество токенов может быть в десяток раз больше, чем количество слов. Также, стоит учитывать, что в отличие от китайского иероглифа, буква латинского или кириллического алфавита почти не несёт семантической (то есть смысловой) ценности.

3.2 Токенизация по словам

Посимвольная токенизация приводит к очень большой длине входной последовательности. Как решить эту проблему? Попробуем разбивать текст не по символам, а по словам. Хотя это решение и значительно сократит размер последовательности, а каждый токен будет иметь большую семантическую ценность, из-за большого количества различных форм слов, а также множества редких слов, размер словаря может раздуться до миллионов значений.

Конечно, редкие слова можно заменять одним специальным токеном, предназначенным для неизвестных элементов. Однако, это мало того, что всё еще не решает проблему огромного словаря, так и при обработке текста может потеряться смысл этих редких слов.

3.3 Byte-pair encoding (BPE)

Очевидно, что для эффективной обработки текста необходим компромиссный вариант между посимвольной токенизацией и токенизацией по словам, сочетающий преимущества обоих вариантов и минимизирующий их недостатки. Одним из таких компромиссных вариантов стал **ВРЕ-токенизатор**.

Мы хотим, чтобы часто повторяющиеся части слов были одним токеном. Возьмём эту идею за основу. Сначала разобьём каждое слово на символы и добавим сепаратор конца слова. Словарь в начале состоит из символов стартового алфавита, сепаратора конца слова и специальных токенов: <PAD>, <BOS>, <EOS>, <UNK>.

Далее, на большом корпусе текста необходимо найти самую частую пару соседних токенов. Например, для текста

Byte-pair encoding tokenization

если разбить слова на символы и добавить сепаратор '</w>', то получим пары соседних токенов:

'By', 'yt', 'te', 'e-', 'p', 'pa', 'ai', 'ir', 'r</w>';
'en', 'nc', 'co', 'od', 'di', 'in', 'ng', 'g</w>';
'to', 'ok', 'ke', 'en', 'ni', 'iz', 'za', 'at', 'ti', 'io', 'on', 'n<w>'.

Пара 'en' встретится 2 раза, а остальные — по 1 разу. Поэтому добавим токен 'en' в словарь и запомним merge-правило, после чего повторим эту последовательность шагов до того, как размер словаря не достигнет желаемого. При запоминании merge-правил крайне важно учитывать их последовательность.

Предположим, что мы обучили ВРЕ-токенизатор, но как с его помощью токенизировать текст? Добавленные элементы начального словаря определённо будут подсловами токенов в словаре. Да и неизвестно, какого размера будет каждый токен, чтобы найти его. Для этого мы и запоминали merge-правила. Чтобы токенизировать текст, применим merge-правила по порядку, в котором их сохраняли.

Алгоритм обучения и токенизации может показаться очень долгим. Однако, токенизацию текста можно применять параллельно по участкам текста, например по предложениям. Обучение ВРЕ также можно частично распараллелить: на разных участках корпуса параллельно подсчитывать частоты пар, а затем складывать их. Само обучение остается итеративным, поскольку после каждого merge-правила частоты пар меняются.

ВРЕ-токенизация предлагает компромисс между посимвольной токенизацией и токенизацией по словам, разбивая текст на последовательность подслов. В свою очередь оказывается, что такие подслова часто являются

морфемами: приставками, суффиксами, корнями, окончаниями слов. Также, ВРЕ-токенизатор способен запоминать имена и названия как отдельные токены.

Byte-level ВРЕ токенизация

Пусть мы хотим обучить ВРЕ-токенизатор для русского языка. Для этого взяли корпус сочинений Льва Николаевича Толстого (предварительно удалив отрывки на французском языке). Однако, когда модель с обученным токенизатором будет использована в чат-боте, то она может столкнуться с неожиданным входом: множество эмодзи и особые символы. А что делать, если мы хотим обучить мультязычный токенизатор?

Byte-level ВРЕ токенизатор решает проблему тем, что сначала переводит текст в байты и работает уже не с набором символов, а с набором байтов. Пробел кодируется одним байтом. Такой токенизатор подойдет как для кириллицы, так и для латиницы, эмодзи и даже бинарных файлов.

3.4 Embedding-слой

После токенизации, заменим токены в тексте на их индексы. Технически, уже эту последовательность можно обрабатывать нейронной сетью. Однако, идентификаторы токенов обладают “псевдопорядком”, из-за этого, например, токен ‘tok’ с номером 29 будет ближе к токenu ‘byte</w>’ с номером 28, чем к токenu ‘en</w>’ с номером 239. Хотя вряд ли можно говорить о какой то их особой смысловой близости, или, тем более, о том, что один токен почему-то больше другого.

При работе с табличными данными для обработки качественных признаков часто используется one-hot-encoding. Но в текущей ситуации он вряд ли применим, поскольку потребует создания гигантской квадратной матрицы с количеством строк и столбцов равными длине словаря. К тому же, эта матрица будет крайне разреженной, то есть состоять в основном из нулей, с редкими единицами.

Вместо разреженной матрицы попробуем обучить плотную матрицу — **embedding-слой**. В этой матрице вектор соответствует тому токenu, чей индекс равен индексу этого вектора. Эти вектора (а вернее весь embedding-слой) инициализируются случайным образом и обучаются вместе с остальными параметрами модели.

Приятным бонусом embedding-слоя является то, что токены, часто встречающиеся в похожих контекстах, например ‘король’ и ‘монарх’, нередко имеют близкие вектора — то есть угол между ними будет мал. Из-за этого возможна арифметика векторов. Например, если из токена ‘король’ вычесть токен ‘мужчина’ и добавить токен ‘женщина’, то получится токен, очень близкий к токenu ‘королева’. Другими словами, embedding-слой отображает токены в **семантическое пространство**.

4 Трансфомер

4.1 Преобразование одной последовательности в другую

Рассмотрим задачу преобразования одной последовательности в другую (seq2seq). Это может быть, например, машинный перевод или чат-бот, который должен выдавать ответ на вопрос. Какие модели обработки последовательностей нам пока известны?

Это RNN, LSTM и GRU, а также одномерные свёрточные нейронные сети. Стоит также вспомнить архитектуру кодировщик-декодировщик (encoder-decoder). В ней один блок, кодировщик, кодирует исходную последовательность в контекстное представление, затем другой блок, декодировщик, опираясь на него генерирует выходную последовательность токен за токеном, пока не будет выведен токен $\langle \text{EOS} \rangle$.

Такая архитектура позволяет естественно работать с входными и выходными последовательностями разной длины.

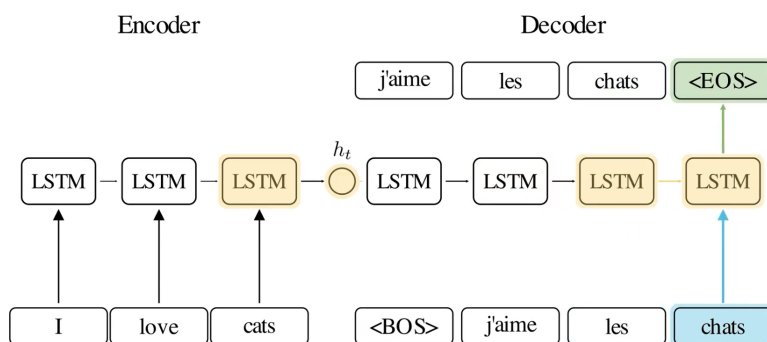


Рис. 11: Архитектура LSTM кодировщик-декодировщик.

Проблема такого подхода в том, что при обработке токенов тяжело учитывать дальние зависимости. Для решения этой проблемы в рекуррентные архитектуры стали добавлять механизмы внимания (attention), позволяющие модели обращаться к различным частям входной последовательности напрямую.

Эти методы стали в итоге state-of-the-art в задачах обработки последовательных данных. Однако они имеют существенные недостатки: подобные архитектуры сложно масштабировать, а их вычисления трудно распараллелить. Из-за своей рекуррентной природы такие сети вынуждены обрабатывать токены последовательно, поскольку за счёт этой архитектурной особенности закладывается индуцированная предвзятость (inductive bias) о порядке токенов в обрабатываемом тексте.

А что если мы откажемся от рекуррентности вообще и сделаем ставку только на механизм внимания? Вдруг окажется так, что внимание — это всё, что нам нужно? В 2017 году была предложена архитектура, которая полностью отказывается от рекуррентности и свёрток и полагается только на механизмы внимания. Эта архитектура называется трансформер.

4.2 Внимание (Attention)

Рассмотрим основу архитектуры трансформеров — внимание.

4.2.1 Мультипликативное внимание (Multiplicative attention)

Что вообще такое внимание? Этим термином будем обозначать силу связи между элементами (векторами) двух последовательностей — $G = \{g_i\}_{i=1}^{s_1} \in \mathbb{R}^{s_1 \times D}$ и $H = \{h_j\}_{j=1}^{s_2} \in \mathbb{R}^{s_2 \times D}$. Сделаем упрощение, что вектора этих последовательностей находятся в общем семантическом пространстве. Тогда для векторов, g_i и h_j , соответствующих хорошо согласованным или близким по смыслу элементам, скалярное произведение $g_i h_j^\top$ будет большим.

На основе скалярного произведения строится механизм мультипликативного внимания:

$$e_{ij} = g_i h_j^\top$$

Чтобы придать большую выразительную силу этому механизму, мы можем использовать билинейную форму:

$$e_{ij} = g_i W h_j^\top,$$

где:

$$W \in \mathbb{R}^{D \times D},$$

или же в матричном виде:

$$E = G W H^\top$$

Так мы можем учитывать, какие компоненты в семантическом пространстве имеют больший вес на матрицу оценок внимания E , а какие — меньший.

Естественно, мы не будем вручную проставлять веса для различных компонент, а сделаем матрицу W обучаемой матрицей весов.

4.2.2 Самовнимание (Scaled-dot Self-Attention)

Вывод самовнимания Связь между векторами g_i и h_j не всегда имеет смысл моделировать одной матрицей W . Выполним небольшой трюк: наложим на W условие возможного разложения на две матрицы такой же размерности, назовём их W_Q и W_K . Тогда:

$$W = W_Q W_K^\top$$

где:

$$W_Q \in \mathbb{R}^{D \times D}, \quad W_K \in \mathbb{R}^{D \times D},$$

Перепишем выражение мультипликативного внимания следующим образом:

$$E = GWH^\top = G(W_Q W_K^\top)H^\top = (GW_Q)(W_K^\top H^\top),$$

поскольку для матриц верно следующее свойство

$$B^\top A^\top = (AB)^\top,$$

то

$$E = (GW_Q)(HW_K)^\top$$

До сих пор мы рассматривали внимание между двумя различными последовательностями векторов G и H . Поскольку наша цель — решить задачу обработки одной последовательности $X = \{x_i\}_{i=1}^s$, то будем вычислять не внимание между двумя разными последовательностями, а самовнимание (self-attention) внутри последовательности X :

$$E = (XW_Q)(XW_K)^\top$$

Пусть $XW_Q = Q$ и $XW_K = K$. Тогда получим матрицу оценок самовнимания последовательности X

$$E = QK^\top$$

Матрица весов внимания Каждая строчка матрицы E отвечает за один вектор q_i , который смотрит на все векторы $\{k_j\}_{j=1}^s$, и дает оценку, насколько сильно вектор q_i должен обратить внимание на вектор k_j

$$e_{ij} = q_i k_j^\top$$

для фиксированного q_i мы получаем строку оценок

$$e_i = (e_{i1}, e_{i2}, \dots, e_{is})$$

применим к этой строке функцию Softmax, чтобы получить нормированное распределение весов внимания:

$$a_i = \text{Softmax}(e_i)$$

или же в матричной форме:

$$A = \text{Softmax}(QK^\top),$$

где Softmax применяется независимо по строкам.

В такой форме мы избегаем появления отрицательных чисел, все значения нормализованы к интервалу $(0, 1)$, то есть находятся в одном масштабе, и $\sum_j a_{ij} = 1$.

Матрицу весов внимания можно трактовать следующим образом: у каждого вектора q_i есть 'бюджет' внимания, равный 1. Он может построить распределение этого 'бюджета' между всеми векторами $\{k_j\}_{j=1}^s$.

Нормализация матрицы внимания Для двумерного случая, функция $\text{Softmax}(x)$ почти то же самое, что и сигмоида — $\sigma(x)$. Как мы помним, основная проблема $\sigma(x)$ заключается в быстром насыщении. Softmax страдает от той же проблемы.

При вычислении матрицы внимания, нам необходимо найти скалярное произведение

$$q_i k_j^\top = \sum_{k=1}^D q_i^{(k)} k_j^{(k)},$$

отсюда видно, что при большем значении D стоит ожидать больших по модулю значений $q_i k_j^\top$. Это приведет к перенасыщению Softmax при работе с векторами большой размерности. В свою очередь, перенасыщение Softmax крайне негативно скажется на способности модели к масштабированию, поскольку стабильность обучения будет падать при увеличении размерности векторов.

Оценим среднее и дисперсию суммы

$$\sum_{k=1}^D q_i^{(k)} k_j^{(k)}$$

Поскольку

$$\mathbb{E}[q_i^{(k)}] = 0, \quad \text{Var}[q_i^{(k)}] = 1$$

и

$$\mathbb{E}[k_j^{(k)}] = 0, \quad \text{Var}[k_j^{(k)}] = 1,$$

то, сделав предположение о независимости $q_i^{(k)}$ и $k_j^{(k)}$, получаем

$$\mathbb{E}[q_i^{(k)} k_j^{(k)}] = 0, \quad \text{Var}[q_i^{(k)} k_j^{(k)}] = 1,$$

Если дополнительно предположить, что $q_i^{(k)} k_j^{(k)}$ независимы по k , то

$$\mathbb{E}\left[\sum_{k=1}^D q_i^{(k)} k_j^{(k)}\right] = 0, \quad \text{Var}\left[\sum_{k=1}^D q_i^{(k)} k_j^{(k)}\right] = D,$$

Чтобы снизить дисперсию результата скалярного произведения до 1, разделим его на $\sqrt{D}$. Таким образом получаем нормализацию

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{D}}\right),$$

которая предотвращает чрезмерное насыщение Softmax.

Перевзвешивание токенов на основе самовнимания Однако, одних весов внимания A недостаточно. Матрица A показывает лишь, каким образом каждый вектор распределяет свое внимание между остальными векторами последовательности. Однако, сама по себе, она не выполняет никаких преобразований и не говорит, какие вектора будут преобразовываться.

Самый простой вариант — агрегировать исходные вектора последовательности X :

$$O = AX$$

В этом случае, каждый вектор o_i будет являться взвешенной суммой (а поскольку $\sum_j a_{ij} = 1$, то можно говорить о средневзвешенном) исходных векторов x_j :

$$o_i = \sum_{j=1}^s a_{ij} x_j$$

Но такой подход ограничен. Как ранее в мультипликативном внимании мы ввели обучаемую матрицу W (которую потом разделили на две обучаемые матрицы W_Q и W_K), так и сейчас введем преобразование с помощью обучаемой матрицы W_V

$$V = XW_V,$$

чтобы таким образом повысить выразительность итогового преобразования

$$O = AV$$

Можно сказать о том, что матрица W_V является фильтром, определяющим с каким весом и какую информацию извлекать из каждой компоненты вектора x_j :

$$v_j = x_j W_V = \begin{pmatrix} x_{j1} \\ x_{j2} \\ \dots \\ x_{jD} \end{pmatrix}^\top \begin{pmatrix} w_{v11} & w_{v12} & \dots & w_{v1D} \\ \dots & & & \\ w_{vD1} & w_{vD2} & \dots & w_{vDD} \end{pmatrix} = \begin{pmatrix} \sum_{k=1}^D x_{jk} w_{vk1} \\ \sum_{k=1}^D x_{jk} w_{vk2} \\ \dots \\ \sum_{k=1}^D x_{jk} w_{vkD} \end{pmatrix}^\top$$

Переставим местами x_{jk} и w_{vki} и увидим, что столбцы матрицы W_V представляют собой веса взвешенной суммы по компонентам вектора x_j :

$$v_j = \begin{pmatrix} \sum_{k=1}^D x_{jk} w_{v_{k1}} \\ \sum_{k=1}^D x_{jk} w_{v_{k2}} \\ \dots \\ \sum_{k=1}^D x_{jk} w_{v_{kD}} \end{pmatrix}^T = \begin{pmatrix} \sum_{k=1}^D w_{v_{k1}} x_{jk} \\ \sum_{k=1}^D w_{v_{k2}} x_{jk} \\ \dots \\ \sum_{k=1}^D w_{v_{kD}} x_{jk} \end{pmatrix}^T$$

Таким образом, общая форма для scaled-dot self-attention выглядит следующим образом:

$$O = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V$$

Матрицы Q , K и V называются соответственно матрицами **запросов** (**queries**), **ключей** (**keys**) и **значений** (**values**).

Что такое запросы, ключи и значения Элемент матрицы оценок внимания $e_{ij} = q_i k_j^T$ представляет собой силу связи вектора запроса q_i и вектора ключа k_j . А строка $e_i = q_i K^T$ представляет собой набор оценок того, насколько сильно вектор q_i должен обратить внимание на каждый из векторов ключей последовательности.

Отсюда вектор q_i можно трактовать как **запрос**, какую именно информацию текущий элемент последовательности хочет найти в остальных элементах. А вектор k_j можно трактовать как **ключ**, по которому другие элементы последовательности могут понять, насколько данный элемент подходит их запросу.

Если запрос q_i и ключ k_j близки, то скалярное произведение получается высоким, соответственно внимание от элемента x_i к элементу x_j повышается.

Как уже было отмечено, столбцы матрицы W_V представляют собой веса взвешенной суммы по компонентам вектора x_j . Соответственно вектор $v_j = x_j W_V$ представляет собой **значения**, которые, согласно матрице W_V , должны передаваться дальше.

4.2.3 Многоглавое внимание (Multihead Attention)

Механизм самовнимания, построенный на одной тройке матриц $\{W_Q, W_K, W_V\}$ позволяет построить только одно распределение внимания. Однако, в реальных данных зависимости между элементами могут быть совершенно разными, и их может быть трудно описать с помощью одной тройки пространств ключей, запросов и значений.

Поэтому, вместо одного механизма самовнимания, будем использовать несколько параллельных и независимых механизмов — голов внимания. Так, каждая голова будет описываться своими собственными матрицами $\{W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}\}$, а значит может искать зависимости в своих собственных пространствах.

Пусть число голов внимания равно H . Для каждой головы h

$$W_Q^{(h)} \in \mathbb{R}^{D \times d_h} \quad W_K^{(h)} \in \mathbb{R}^{D \times d_h} \quad W_V^{(h)} \in \mathbb{R}^{D \times d_h}$$

где $d_h = \frac{D}{H}$, чтобы после объединения голов сохранить исходную размерность.

Запишем результат механизма самовнимания для головы h :

$$O^{(h)} = \text{Softmax}\left(\frac{Q^{(h)} K^{(h)\top}}{\sqrt{d_h}}\right) V^{(h)}$$

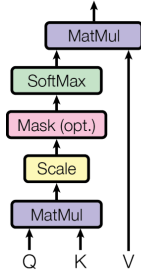
Объединим головы:

$$O_{\text{concat}} = \text{Concat}[O^{(1)}, O^{(2)}, \dots, O^{(H)}]$$

Выходы разных голов находятся в разных пространствах. Пусть, после объединения голов мы и получили матрицу O_{concat} той же размерности, что и исходная матрица X , информация, полученная из разных голов, все еще разделена покомпонентно. Чтобы перемешать информацию из разных голов, добавим еще одно обучаемое линейное преобразование $W_O \in \mathbb{R}^{D \times D}$:

$$O = O_{\text{concat}} W_O$$

Scaled Dot-Product Attention



Multi-Head Attention

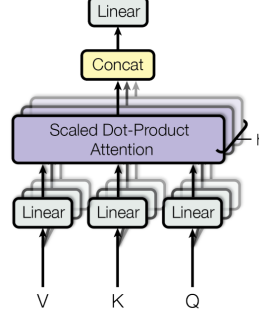


Рис. 12: Сравнение механизма самовнимания и многоглавого внимания. Взято из оригинальной статьи “Attention is all you need”.

На первый взгляд может показаться, что механизм многоглавого внимания требует значительно больше вычислений и параметров, чем механизм самовнимания с одной головой. На деле, увеличение сложности лишь связано с появлением нового слоя W_O .

4.2.4 Маскированное многоглавое внимание (Masked Multihead Attention)

До сих пор мы рассматривали лишь случай, когда каждый элемент последовательности всегда может обратить внимание на любой другой элемент. Однако, это не всегда допустимо, в некоторых задачах необходимо заранее ограничить множество токенов, на которые разрешено смотреть. Для этого в механизм внимания вводится маска.

Мы могли бы обнулять некоторые элементы a_{ij} матрицы весов внимания. Но тогда мы ломаем правило, что сумма элементов каждой строки a_i равна 1, а также сместим распределение весов внимания. Поэтому, чтобы не приходилось снова нормализовать значения весов, удобнее накладывать маску до вычисления Softmax.

Вспомним формулу Softmax:

$$\text{Softmax}_j(x) = \frac{\exp(x^{(j)})}{\sum_{k=1}^D \exp(x^{(k)})}$$

$\text{Softmax}_j(x) = 0$ тогда, когда $\exp(x^{(j)}) = 0$. А $\exp(x^{(j)}) = 0$ тогда, когда аргумент экспоненты равен $-\infty$.

Таким образом, чтобы наложить маску на матрицу весов A , необходимо к матрице оценок внимания E прибавить матрицу M , которая состоит из $-\infty$ на тех местах, которые необходимо маскировать, и из нулей во всех остальных случаях.

4.2.5 Перекрестное внимание (Cross Attention)

Ранее, при переходе от мультипликативного внимания к самовниманию, мы сделали упрощение, ограничив вход только одной последовательностью X .

$$E = (GW_Q)(W_K^T H^T) = \tilde{Q}\tilde{K}^T$$

Но, все последующие выкладки никак не противоречат тому, что последовательности $\tilde{Q}$ и $\tilde{K}$ могут происходить из разных источников и быть разной длины — $s_{\tilde{Q}}$ и $s_{\tilde{K}}$. Для корректности вычислений необходимо только, чтобы

$$s_{\tilde{V}} = s_{\tilde{K}},$$

то есть длина последовательности $\tilde{V}$ была равна длине последовательности $\tilde{K}$.

Пусть матрица $\tilde{Q}$ происходит из последовательности входов X

$$\tilde{Q} = XW_Q,$$

а матрицы $\tilde{K}$ и $\tilde{V}$ происходят из другой последовательности — Z :

$$\tilde{K} = ZW_K, \quad \tilde{V} = ZW_V,$$

Механизм внимания, построенный на основе $\{\tilde{Q}, \tilde{K}, \tilde{V}\}$ называется **перекрестным вниманием (cross-attention)**.

$$O = \text{Softmax}\left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_h}}\right)\tilde{V}$$

Перекрестное внимание позволяет вычислять внимание между разными последовательностями.

4.3 FFN-слой

Механизм внимания отвечает за обмен информацией между различными токенами входной последовательности. Но после этого еще необходимо обработать полученное представление каждого токена более сложным и нелинейным образом.

Для этого используется двуслойная нейронная сеть, называемая FFN-слоем. FFN в трансформере применяется независимо к каждому токenu входной последовательности X .

$$\text{FFN}(x_i) = \phi(x_i W_1 + b_1) W_2 + b_2,$$

где

$$W_1 \in \mathbb{R}^{D \times D_{FFN}}, \quad W_2 \in \mathbb{R}^{D_{FFN} \times D}$$

Обычно, размерность внутреннего слоя D_{FFN} выбирается больше, чем D . FFN сначала расширяет пространство признаков, а следом сжимает.

При обучении трансформера в FFN-слое используют dropout. Его обычно применяют между двумя полносвязными слоями.

$$\text{FFN}(x_i) = \text{Dropout}\left[\phi(x_i W_1 + b_1)\right] W_2 + b_2,$$

Иногда dropout применяют и после второго линейного преобразования. FFN-слой содержит большое количество параметров и выполняется независимо для каждого токена. Из-за этого возможно переобучение, которое и компенсируется регуляризацией.

4.4 Нормализация

LayerNorm и BatchNorm основаны на одной идее нормализации активаций. Разница заключается в том, как мы считаем $\mathbb{E}[X]$ и $\text{Var}[X]$. BatchNorm вычисляет эти статистики по батчу, LayerNorm же вычисляет статистики по одному объекту.

LayerNorm работает одинаково при обучении и инференсе, в отличие от BatchNorm, который требует агрегирования статистик по батчу во время обучения и использует накопленные статистики для инференса. Также,

LayerNorm, в отличие от BatchNorm, лучше применять к последовательностям разной длины и к малым батчам, так как статистики BatchNorm в этих случаях становятся более шумными.

Кроме того, LayerNorm лучше подходит для малых батчей, так как статистики BatchNorm при этом становятся слишком шумными.

4.5 Слой трансформера

Соберем все модули вместе: механизм многоглавого внимания (МНА), LayerNorm (LN) и FFN-слой. Попробуем применить их последовательно:

$$\text{TransformerLayer}(x) = \text{LN} \circ \text{FFN} \circ \text{LN} \circ \text{МНА}(x)$$

Наивный подход, когда мы применяем преобразования одно за другим, оказывается не слишком удачным для обучения. Даже один такой слой уже содержит в себе четыре последовательных матричных умножения из механизма многоглавого внимания, функцию Softmax, две нормализации, два линейных преобразования из FFN-слоя и функцию активации.

4.5.1 Остаточные связи (Residual connections)

Уже в внутри одного трансформерного слоя мы рискуем столкнуться с проблемой затухающих градиентов. А в реальной модели потребуется несколько таких слоёв. Чтобы этого избежать, к выходу каждого крупного подслоя прибавляется его вход — остаточная связь (residual connection):

$$z = \text{LN}\left(x + \text{МНА}(x)\right),$$

$$y = \text{LN}\left(z + \text{FFN}(z)\right),$$

Попробуем продифференцировать такие преобразования по входу (будем рассматривать частную производную компоненты выхода y по компоненте входа x), для второго подслоя тогда получим

$$\frac{\partial y}{\partial z} = \frac{\partial \text{LN}\left(z + \text{FFN}(z)\right)}{\partial \left(z + \text{FFN}(z)\right)} \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right),$$

а для первого

$$\frac{\partial z}{\partial x} = \frac{\partial \text{LN}\left(x + \text{МНА}(x)\right)}{\partial \left(x + \text{МНА}(x)\right)} \left(1 + \frac{\partial \text{МНА}(x)}{\partial x}\right)$$

Пусть $\tilde{z} = z + \text{FFN}(z)$ и $\tilde{x} = x + \text{МНА}(x)$. Найдём $\frac{\partial y}{\partial x}$:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial x} = \frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right) \cdot \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})} \left(1 + \frac{\partial \text{MHA}(x)}{\partial x}\right),$$

сгруппируем производные LayerNorm по входу, для этого рассмотрим частную производную:

$$\frac{\partial y}{\partial x} = \frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \cdot \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})} \cdot \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right) \cdot \left(1 + \frac{\partial \text{MHA}(x)}{\partial x}\right),$$

раскроем скобки:

$$\frac{\partial y}{\partial x} = \underbrace{\frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})}}_{\text{Градиенты почти не затухают}} \cdot \underbrace{\left(1 + \frac{\partial \text{FFN}(z)}{\partial z} + \frac{\partial \text{MHA}(x)}{\partial x}\right)}_{\text{Градиенты затухают слабее}} + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} \cdot \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты сильно затухают}}$$

Таким образом видно, что добавление остаточных связей позволяет градиентам доходить до более глубоких слоев, почти избегая затухания.

4.6 Позиционное кодирование

Рассмотрим слой трансформера. Сначала идет механизм внимания. Механизм внимания смотрит на все элементы последовательности разом, преобразование вектора никак не зависит от его порядка, а зависит от значений других векторов последовательности. Далее идет слой LayerNorm, который обрабатывает каждый токен независимо от других. Следом — FFN-слой, который также обрабатывает каждый токен независимо.

В рекуррентных сетях мы закладывали индуктивное предубеждение о порядке токенов за счёт порядка обработки элементов последовательности. В трансформере же мы обрабатываем все элементы разом. Мы нигде не добавляем информацию о порядке слов в предложении.

Между тем, порядок слов в предложении крайне важен, поскольку передает логический акцент и определяет контекст. Например, в немецком языке крайне важно располагать частицу *nicht* в нужном месте, чтобы не исказить смысл:

- Er kommt morgen **nicht** — Он не придет завтра,
- Er kommt **nicht** morgen — Он придет, но не завтра.

В этих предложениях используются абсолютно одинаковые слова, но смысл изменяется на противоположный, когда мы поменяли расположение **nicht**.

Поскольку пока мы никак не добавляем в модель информацию о позиции токенов, добавим эту информацию к самой последовательности.

4.6.1 Абсолютное позиционное кодирование

К каждому токёну прибавим некоторый вектор, который будет кодировать его абсолютную позицию в последовательности:

$$\hat{x}_{\text{pos}} = x_{\text{pos}} + \text{PE}(\text{pos}),$$

где $\text{PE} : \mathbb{N} \rightarrow \mathbb{R}^D$ — функция, отображающая позицию токёна (число) в вектор позиционного кодирования, размерность которого совпадает с размерностью вектора обрабатываемой последовательности.

Для такой функции обязательно должно выполняться то, что разные позиции должны кодироваться разными векторами. Также желательно, чтобы позиции имели близкие вектора и кодирование было таким, чтобы модель была способна восстановить относительные сдвиги между позициями.

4.6.2 Синусоидальное кодирование

Последнее особенно важно, поскольку было бы удобно, если бы кодировка позиции $\text{pos} + k$ могла быть выражена через кодировку позиции pos и величину сдвига k . Для этого естественно использовать синус и косинус, поскольку для этих функций выполняется:

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta,$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta.$$

Если теперь в некоторой компоненте вектора позиционного кодирования записать $\sin(\omega \text{pos})$, а в соседней — $\cos(\omega \text{pos})$, то для позиции $\text{pos} + k$ получим:

$$\sin(\omega(\text{pos} + k)) = \sin(\omega \text{pos}) \cos(\omega k) + \cos(\omega \text{pos}) \sin(\omega k),$$

$$\cos(\omega(\text{pos} + k)) = \cos(\omega \text{pos}) \cos(\omega k) - \sin(\omega \text{pos}) \sin(\omega k).$$

таким образом, кодировка позиции $\text{pos} + k$ в паре компонент с одной и той же частотой ω выражается через кодировку позиции pos .

Выберем набор частот, поскольку если использовать одну частоту, то разные позиции начнут периодически совпадать. Покомпонентно используются сразу многие частоты, это изменять одни компоненты быстро, а другие медленно, кодируя тем самым как локальное положение токёна, так и глобальное:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/D}}\right),$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/D}}\right),$$

где i — номер пары компонент. Малые i соответствуют низким частотам, большие i — высоким. Итоговый вектор выглядит следующим образом:

$$PE_{pos} = \begin{pmatrix} \sin\left(\frac{pos}{10000^{0/D}}\right) \\ \cos\left(\frac{pos}{10000^{0/D}}\right) \\ \sin\left(\frac{pos}{10000^{2/D}}\right) \\ \cos\left(\frac{pos}{10000^{2/D}}\right) \\ \vdots \\ \cos\left(\frac{pos}{10000^{(D-2)/D}}\right) \\ \sin\left(\frac{pos}{10000^{(D-2)/D}}\right) \end{pmatrix}^T$$

Число 10000 задает диапазон используемых частот. У каждой пары компонент свой период:

$$T_i = \frac{2\pi}{w_i} = 2\pi \cdot 10000^{2i/D},$$

так что использование различных частот для каждой компоненты также позволяет выполнить требование неравенства векторов позиционного кодирования для разных позиций.

4.7 Трансформер кодировщик и трансформер декодировщик

Рассмотрим архитектуру трансформера-кодировщика. Его задача — некоторым образом кодировать входную последовательность X_1 в последовательность Z . Архитектура представлена рядом из N_{enc} последовательных слоёв.

Каждый такой слой можно записать следующим образом:

$$\tilde{x} = \text{LN}\left(x + \text{MHA}(x)\right),$$

$$y = \text{LN}\left(\hat{x} + \text{FFN}(\hat{x})\right),$$

Теперь рассмотрим архитектуру трансформера-декодировщика. В отличие от кодировщика, декодировщик преобразует последовательность X_2 не в какое то латентное представление Z , а в другую последовательность Y , которая является продолжением последовательности X_2 . Трансформер-декодировщик предсказывает новую последовательность токенов за токеном, но об обучении трансформера немного позже.

Важно, что обычный механизм внимания не подходит для этой задачи, поскольку тогда каждый токен будет видеть все следующие, что позволяет модели переобучаться, как бы заглядывая в будущее. Поэтому будем маскировать внимание, прибавив к матрице оценок внимания маску.

В этой маске на всех позициях выше главной диагонали будет значение $-\infty$, а на главной диагонали и ниже — нули. Это позволить обнулить веса внимания для всех следующих токенов последовательности:

$$\text{MaskedMHA}(x) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

где:

$$m_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i \end{cases}$$

Запишем теперь слой трансформера-декодировщика:

$$\tilde{x} = \text{LN}\left(x + \text{MaskedMHA}(x)\right),$$

$$y = \text{LN}\left(\tilde{x} + \text{FFN}(\tilde{x})\right),$$

Также, как и кодировщик, трансформер-декодировщик можно представить рядом N_{dec} последовательных слоёв.

Теперь рассмотрим архитектуру трансформера кодировщика-декодировщика. Это более общий случай, когда необходимо преобразовать последовательность X_1 в Y . Такая задача ставится, например, при выполнении машинного перевода.

В этом случае, каждый модуль берет на себя свою задачу: кодировщик обрабатывает входную последовательность, декодировщик генерирует выходную последовательность, на основе информации из кодировщика.

Кодировщик видит входную последовательность целиком, поэтому ему достаточно обработать ее один раз. Декодировщик же будет работать авторегрессионно, последовательно генерируя токен за токеном. В этом случае одного только маскированного многоглавого внимания не хватит, поскольку декодировщику, помимо входа X_2 необходимо также учитывать и выход кодировщика Z .

Для этого в слой декодировщика добавляется перекрестное внимание:

$$\tilde{x} = \text{LN}\left(x + \text{MaskedMHA}(x)\right),$$

$$\hat{x} = \text{LN}\left(\tilde{x} + \text{CrossAttention}(\tilde{x}, Z)\right),$$

$$y = \text{LN}\left(\hat{x} + \text{FFN}(\hat{x})\right),$$

В перекрестном внимании последовательность Q происходит из входной последовательности X_2 , а последовательности K и V — из Z .

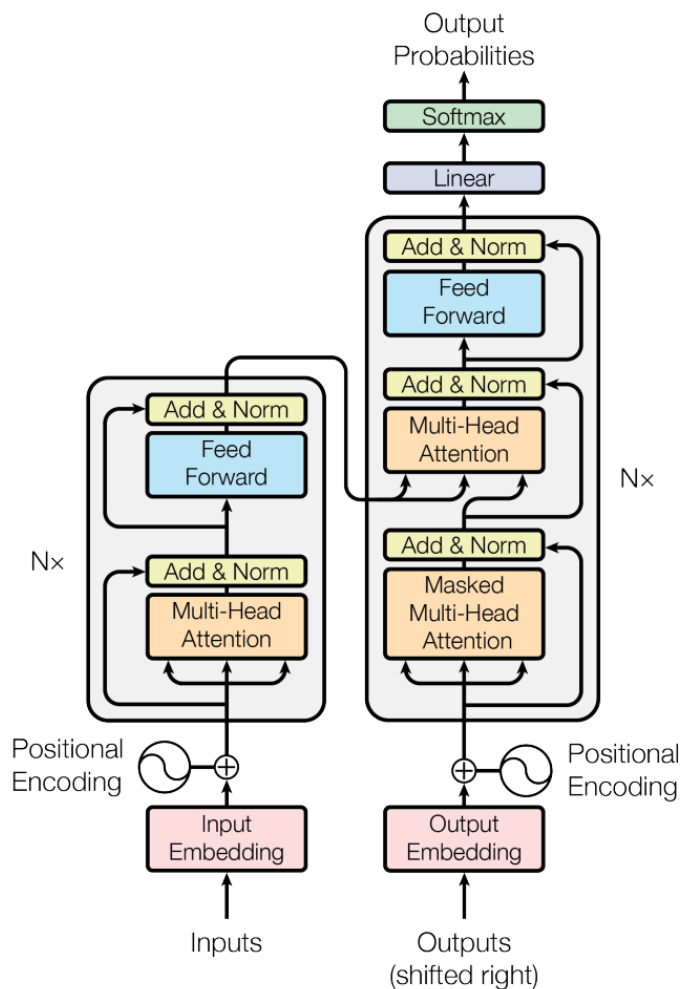


Рис. 13: Архитектура трансформера кодировщика-декодировщика, взято из оригинальной статьи “Attention is all you need”.

4.8 Обучение трансформера

Разберем обучение трансформера на примере декодировщика. Во время обучения необходимо предсказать последовательность токенов, сдвинутую на 1 шаг влево. То есть если на вход идет последовательность

<BOS>, <Архитек> <тура\w>, < >, <Трансфор>, <мер/\w>

то целевая последовательность на выходе выглядит так:

<Архитек> <тура>, < >, <Трансфор>, <мер/w>, <EOS>

Модель должна для каждой позиции входной последовательности предсказать один правильный токен из словаря. Говоря иначе, задача обучения сводится к многоклассовой классификации. Мы должны по каждому из выходных векторов предсказать следующий токен.

Для этого к выходам последнего слоя трансформера применим классификационную голову, состоящую из линейного слоя, преобразующего скрытое состояние токена в вектор логитов размерности словаря. Чтобы получить распределение вероятностей выходного токена q используем Softmax.

Далее, чтобы приблизить это распределение к целевому p , будем минимизировать кросс-энтропию $H(p, q)$ между ними.

Кросс-энтропия выступает мерой неопределенности модели в выборе следующего токена. Вспомним, что мы можем разложить кросс-энтропию на две составляющие части:

$$H(p, q) = H(p) + D_{KL}\langle p||q \rangle,$$

Пусть $\mathcal{C}$ — множество всех возможных контекстов, C — случайная величина, принимающая значения в $\mathcal{C}$, c — конкретный контекст из $\mathcal{C}$, а Y — случайная величина, соответствующая следующему токenu. Для конкретной пары (c, y) , где y — конкретный токен из Y , распределение $p(y|C = c)$ будет вырожденным. Тогда $H(p) = 0$.

Однако, если рассматривать (C, Y) как случайную пару, семплированную из истинного распределения данных, то распределение $p(y|C)$ будет отлично от вырожденного, а значит $H(p) \neq 0$.

$H(p)$ в этом случае мы можем трактовать как энтропию текста, на котором мы обучаем модель. Энтропия языка характеризует неопределенность и непредсказуемость появления следующего токена. Если бы энтропия была равна нулю, то каждый текст был бы предопределен с первого токена. В некотором смысле это шум в данных, который мы не можем устранить. И очевидно, что при обучении значение функции потерь не опустится ниже $H(p)$

$D_{KL}\langle p||q \rangle$ — это та часть, на которую мы в силах повлиять. В естественном языке существует множество правил, которые так или иначе накладывают ограничения на генерацию следующего токена. Это, например, правила грамматики, устойчивые словосочетания, синтаксические зависимости и так далее. В начале обучения модель ничего о них не знает и должна выучить их неявным образом, минимизируя $D_{KL}\langle p||q \rangle$ составляющую кросс-энтропии.

4.8.1 Perplexity

Вычисление кросс-энтропии можно представить как взвешенную сумму логарифмов вероятностей. Такое выражение тяжело интерпретировать. Переделаем его. Возьмем экспоненту от кросс-энтропии:

$$\exp(H(p, q)) = \exp\left(-\sum_i p_i \log q_i\right) = \prod_i \exp(-p_i \log q_i) = \prod_i \exp(\log q_i)^{-p_i}$$

Упростим:

$$\exp(H(p, q)) = \prod_i q_i^{-p_i}$$

Чем выше вероятность, которую модель назначает правильным токенам, тем меньше это значение. И наоборот, если модель распределяет вероятность между большим числом возможных токенов и не уверена в выборе, то значение $\exp(H(p, q))$ растёт.

Эту величину называют перплексией (perplexity):

$$\text{Perplexity}(p, q) = \exp(H(p, q)) = \prod_i q_i^{-p_i}$$

Эту величину можно трактовать как "среднее" число вариантов, между которыми колеблется модель при выборе следующего токена. Например, $\text{Perplexity} = 10$ можно интерпретировать так, будто бы на каждом шаге модель выбирает между 10 равнозначными вариантами.

Перплексию можно использовать в качестве метрики обучения языковой модели и трансформера в частности.

4.9 Инференс трансформера

Во время обучения мы подавали на вход декодировщику текстовую последовательность и учили модель предсказывать новую последовательность, сдвинутую на один шаг влево. Однако во время инференса полной текстовой последовательности ещё нет, модель должна построить её самостоятельно, токен за токеном, начиная с токена $\langle \text{BOS} \rangle$ и генерируя новые токены до тех пор, пока либо не будет достигнут токен $\langle \text{EOS} \rangle$, либо максимальная длина последовательности.

Во время обучения предсказания для всех позиций можно было считать параллельно. Во время инференса мы вынуждены все токены предсказывать последовательно, так как генерация следующего токена зависит от того, какие токены мы сгенерировали на предыдущих шагах.

На одном шаге генерации мы получаем вектор вероятностей. После получения такого вектора, необходимо выбрать токен. Самый простой вариант — использовать жадный алгоритм, всегда выбирая токен с большей вероятностью. Этот подход полностью детерминированный, что может приводить к однообразным ответам и циклам в генерации.

Другой вариант — семплировать токен из полученного распределения. Однако, в этом случае есть шанс генерации маловероятного токена, который никак не подходит под входную последовательность. Компромиссный вариант — семплировать токен из k лучших вариантов с сохранением общего распределения.

4.9.1 Температура трансформера

Для получения вероятностей слов из вектора размера `vocab_size` необходимо применить функцию `Softmax`.

При использовании `Softmax` наибольшему логиту соответствует наибольшая вероятность. Если необходимо регулировать разброс значений вероятностей, можно использовать функцию `Softmax` с температурой:

$$\text{Softmax}_T(x) = \frac{e^{x_k/T}}{\sum_{i=0}^{n-1} e^{x_i/T}}$$

Чем больше температура — тем меньший приоритет отдается большим компонентам. Это добавляет больше случайности при генерации слов в последовательности.

4.10 Масштабируемость трансформеров

Одно из главных преимуществ трансформера перед рекуррентными архитектурами заключается в том, что его вычисления хорошо параллелизируются. Это позволяет значительно ускорить обучение, используя большое количество графических процессоров (GPU).

Большая часть операций внутри трансформера — это матричные умножения (GEMM, General Matrix Multiplication). А GPU хорошо оптимизированы для вычисления матричных умножений.

Итого, параллелизм в трансформере возникает сразу на нескольких уровнях:

1. Параллелизм по батчам
2. Параллелизм по головам
3. Параллельное вычисление GEMM в механизме внимания внутри одной головы

Если раньше масштабирование количества вычислений упиралось в архитектуру модели, то теперь это чисто инженерная задача, заключающаяся в организации вычисления модели на одной видеокарте и на нескольких.

5 Vision Transformer

5.1 Проблемы свёрточных нейронных сетей

Свёрточные нейронные сети до относительно недавнего времени были стандартом в задачах обработки изображений, каждый раз являясь основой для SOTA-архитектур. Однако, как и у многих подходов, у CNN есть свои проблемы.

Во-первых, это **отсутствие восприятия глобального контекста**. На каждом слое CNN обрабатывают изображения через локальные поля. Это означает, что они рассматривают только небольшую область за раз. Такой подход ограничивает возможность сети воспринимать глобальный контекст изображения. Например, CNN не могут гарантировать того, что информация из левого верхнего угла будет связана с информацией из правого нижнего. Либо, CNN будут связывать дальние области косвенно и постепенно, что плохо скажется на обучении сети.

Во-вторых, CNN используют **одинаковые фильтры**, применяя их ко всему изображению. Такой подход может быть неэффективен, если полезная информация распределена неравномерно.

Обсудим проблему отсутствия восприятия глобального контекста свёрточными сетями. Она кажется довольно знакомой: из-за локального восприятия контекста и последовательной обработки отдельных участков сети тяжело воспринимать всю информацию целиком. Да ведь это же почти та же самая проблема, которая была решена с помощью механизма внимания! А как мы уже выяснили, наиболее удачная архитектура, основанная на внимании — это трансформер.

Если задуматься, то трансформеры решают и вторую проблему: механизм самовнимания может отдавать приоритет различным участкам при неравномерно распределённой на изображении информации, а нацеленность архитектуры трансформера на работу с последовательностями может позволить работать с изображениями различного размера.

Однако, как мы можем адаптировать архитектуру трансформера, модели изначально предназначенной для решения задачи машинного перевода, для решения задач обработки изображений? Ведь, в отличие от предложений, состоящих из сравнительно небольшого количества элементов - токенов, изображения состоят из огромного количества пикселей. Даже небольшая картинка размером 256×256 состоит из 65536 пикселей. А если изображение будет больше? Тогда не хватит никакого контекстного окна, чтобы его обработать.

Посмотрим на проблему с другой стороны: а почему мы должны считать всего один пиксель — токеном? В трансформерах после embedding-слоя токен превращается в плотный вектор (например, размерностью 512). Будет довольно странно сравнивать с ним пиксель, который для трёхканальных изображений — трёхмерный вектор. Что если попробовать подавать на вход трансформеру несколько пикселей изображения сразу, как бы нарезая его на несколько патчей? Примерно таким же вопросом в 2020м году задалась авторы модели Vision Transformer (ViT).

5.2 Токенизация изображений

Авторы ViT решали задачу классификации изображений и решили никак не менять оригинальную архитектуру кодировщика трансформера, показав, что такой подход работает лучше всего. Различия с оригинальной статьёй заключаются лишь в способе обработки данных, поступающих на вход

МОДЕЛИ.

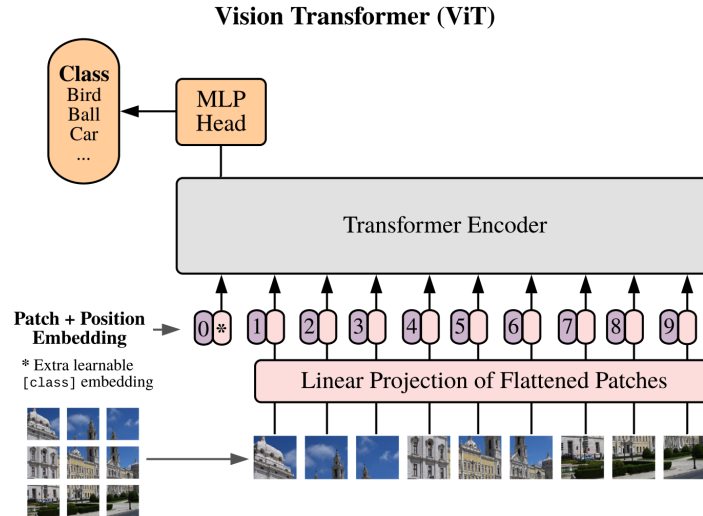


Рис. 14: Архитектура ViT. Взято из оригинальной статьи “An Image is Worth 16x16 Words”.

Для того чтобы подать изображение $x \in \mathbb{R}^{H \times W \times C}$ на кодировщик трансформера, авторы ViT предлагают разбивать его на патчи $16 \times 16 \times C$, потом проецировать в вектора размерностью D . Сделать это можно по-разному: можно выполнить с помощью линейного слоя, а можно с помощью свертки с размером ядра и шагом равными 16.

5.3 Обучаемое абсолютное позиционное кодирование

В отличие от текстовых последовательностей в задачах NLP, изображения имеют двумерную структуру. В оригинальной статье, посвященной модели трансформера, для решения проблемы позиционирования элементов последовательности использовалось одномерное позиционное кодирование. При обработке изображений логичным шагом будет применить двумерное позиционное кодирование для патчей. Авторы ViT сравнили одномерное и двумерное обучаемое позиционное кодирование и показали, что существенного прироста точности для изображений фиксированного размера от использования двумерного метода нет.

5.4 CLS токен

Если обратить внимание на схему ViT, можно заметить токен под номером 0 — токен CLS. Концепция этого токена была впервые предложена в модели BERT, используемой для задач обработки естественного языка. CLS-токен

добавляется в начало входной последовательности и служит для агрегации информации из всей последовательности, чтобы использовать её в задачах классификации. Его финальное состояние используется для агрегирования всей последовательности. Это позволяет избежать предвзятости, которая может возникнуть при выборе отдельного токена последовательности.

Часть III

Обучение на GPU

В этой главе мы отойдем от рассмотрения архитектур нейронных сетей и коснемся инженерных основ, которые во многом и определяют решения, которые принимаются при проектировании современных моделей. Рассмотрим обучение нейронных сетей на GPU. Но начнем с главного вопроса: а зачем нам вообще GPU?

Линейные слои, свёртки, механизм внимания, нормализации и другие операции в нейронных сетях работают не с одним числом, а с большими тензорами. Линейный слой, например, описывается матричным умножением — матрица объектов $X \in \mathbb{R}^{s \times D}$ (где s — длина последовательности) умножается на матрицу весов $W \in \mathbb{R}^{D \times D}$ и матричным сложением — к каждой строке x прибавляется вектор смещения b .

При перемножении или сложении матриц каждый элемент результирующей матрицы вычисляется независимо от других. При вычислении на CPU, эта информация не дает нам почти никакого преимущества, поскольку операции выполняются последовательно (по сравнению с GPU). Однако, GPU позволяют проводить вычисления параллельно и спроектированы для максимизации пропускной способности (**throughput**) — количества операций за единицу времени.

В свою очередь CPU позволяют проводить вычисления с минимальной задержкой (**latency**), однако будут значительно уступать GPU в пропускной способности.

Таким образом, чем больше вычислений мы можем выполнять параллельно, тем большее преимущество имеет GPU перед CPU. А как мы ранее рассматривали, нейронные сети, в частности трансформеры, очень хорошо распараллеливаются.

Однако не все операции нейронной сети одинаково хорошо ускоряются на GPU. Если операция выполняет мало арифметики, но много читает или пишет в память, то она может упираться не в скорость вычислений, а в скорость чтения и записи памяти. Например, при выполнении поэлементных операций, таких как сложения векторов или матриц.

Проблема усугубляется при появлении редукций: операций, “схлопывающих” массив чисел в одно число. Это могут быть, например, операции сложения, нахождения минимума, среднего или максимума. Например, при вычислении функции Softmax

$$\text{Softmax}_j(x) = \frac{\exp(x^{(j)})}{\sum_{k=1}^D \exp(x^{(k)})}$$

необходимо вычислить сумму $\sum_{k=1}^D \exp(x^{(k)})$ — выполнить редукцию. А функция Softmax необходима для вычисления механизма внимания. Также сложности возникают при вычислении нормализаций, поскольку они требуют подсчёта статистик, в ходе которых вызываются операции редукции

— нахождение среднего и суммы.

GPU также плохо справляется с выполнением множества мелких операций. Во-первых, потому что запуск вычислений на GPU происходит на CPU и требует некоторого времени. Во-вторых, потому что в начале вычисления GPU должен считать данные из памяти, а в конце вычислений — уже записать новые данные в память.

Перед тем, как решать проблемы вычисления нейронных сетей на GPU, необходимо рассмотреть то, как они устроены внутри.

6 Анатомия GPU

6.1 Streaming Multiprocessor

GPU состоит из большого числа вычислительных блоков, которые называются **Streaming Multiprocessors (SM)**. SM является основной единицей исполнения вычислений на GPU. Грубо говоря, GPU можно представить как набор множества SM, каждый из которых умеет независимо выполнять часть общей работы.

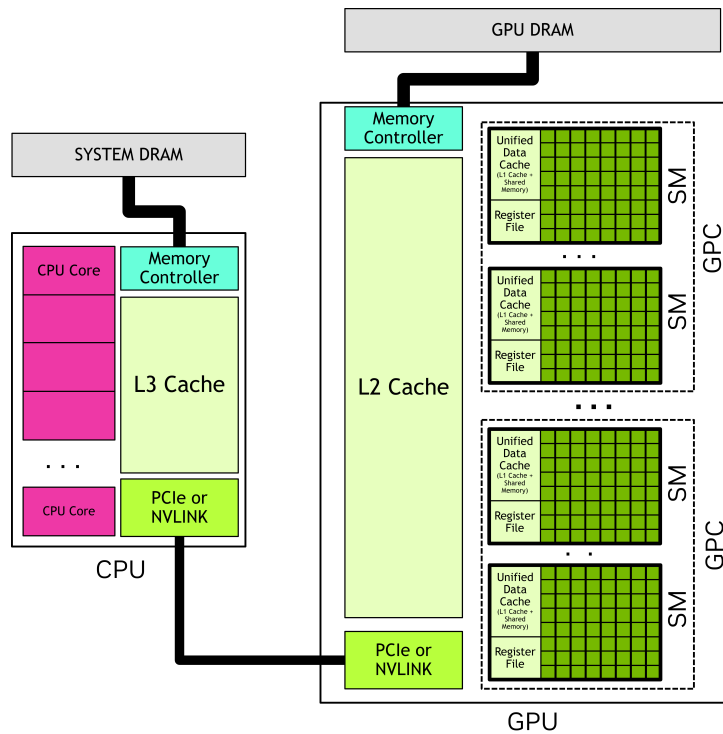


Рис. 15: Архитектура NVIDIA GPU

Внутри SM находятся основные аппаратные компоненты, которые используются при выполнении программ: CUDA-ядра, тензорные ядра, регистры, разделяемая память (shared memory) и варп-планировщики (warp schedulers).

6.2 Потоки и варпы

Когда мы запускаем вычисления на GPU, задача разбивается на большое число **потоков (threads)**. Каждый поток выполняет одну и ту же программу, но обычно работает со своими данными.

Например, при сложении двух векторов, один поток может отвечать за вычисление одного элемента итогового вектора:

$$c_i = a_i + b_i$$

В таком случае, разные потоки считают разные элементы массива независимо.

GPU, однако, не исполняет такие операции полностью независимо по одному элементу. Они группируются в **варпы (warps)**.

Варп — это группа из 32 потоков, которые исполняются вместе. Все потоки внутри варпа в каждый момент исполняют одну и ту же инструкцию, но могут работать с разными данными. Варп является базовой единицей исполнения инструкций на GPU.

Такой принцип исполнения называется **SIMT** — Single Instruction, Multiple Threads. Одна инструкция применяется сразу к группе потоков. Так, при сложении векторов, все потоки исполняют инструкцию сложения, но каждый поток работает со своим индексом.

Однако, если мы решим выполнить сложение с условием по маске, то задача становится сложнее. Допустим, при значении маски на текущем индексе равном 1, мы выполняем сложение, а при значении маски равном 0 — вычитание. Тогда, если в одном варпе для двух потоков попадутся разные значения маски по их индексу, то варпу придется выполнять 2 разные инструкции — выполнить сложение или вычитание. Такая ситуация называется **расхождением варпа (warp divergence)**.

В этом случае варпу придется разделить выполнение программы на 2 ветки: ту, в которой он выполняет сложение для одних индексов, и ту, в которой он выполняет вычитание для других индексов. Ветки будут вынуждены выполняться последовательно: во время выполнения такой ветки, варп отключает потоки, которые в ней не участвуют. Это плохо, поскольку мы хотим выполнять вычисления параллельно. Поэтому при написании программ на GPU стоит избегать ветвлений, которые могут привести к расхождению варпа.

6.3 CUDA-ядра

CUDA-ядра — это арифметико-логические устройства внутри SM, они выполняют обычные арифметические операции над целыми числами (int) и

над числами с плавающей точкой (float), такие как сложение и умножение, а также логические операции, операции сравнения и другие.

Важно, что один поток не соответствует одному CUDA-ядру. Поток — программная единица исполнения, а CUDA-ядро — аппаратное устройство, которое выполняет инструкции. Инструкции варпов уже исполняются аппаратными блоками внутри SM.

6.4 Тензорные ядра

CUDA-ядра хорошо подходят для простых арифметических операций, индексной арифметики, но для выполнения матричных умножений используются специализированные вычислительные блоки внутри SM — **тензорные ядра**.

Тогда как CUDA-ядра выполняют скалярные инструкции, тензорные ядра спроектированы таким образом, чтобы выполнять небольшие операции матричного умножения. Однако, они не предназначены для другой логики.

6.5 Варп-планировщики

Как мы уже обсудили, потоки на GPU не исполняются по одному, а варпами. Однако, сам по себе варп ещё не выполняет вычисления. Внутри SM есть специальные аппаратные блоки, которые выбирают, какой варп будет исполняться в данный момент — **варп-планировщики (warp schedulers)**. Обычно в одном SM четыре варп-планировщика.

В каждый момент времени работы программы на одном SM находится несколько активных варпов — варпов, занявших ресурсы SM. В зависимости от архитектуры, максимальное число активных варпов может быть 32, 48 или 64, однако в случае нехватки ресурсов на SM, это число может быть ниже.

Часть из них готова к выполнению, а часть может ждать данные из памяти или завершения предыдущих инструкций. Задача варп-планировщика — выбрать готовый активный варп и отправить его следующую инструкцию на выполнение.

Если варп запрашивает данные из глобальной памяти, результат может прийти не сразу. CPU в такой ситуации пытается уменьшить задержку за счёт сложных кэшей, предсказаний ветвлений и других инженерных хитростей.

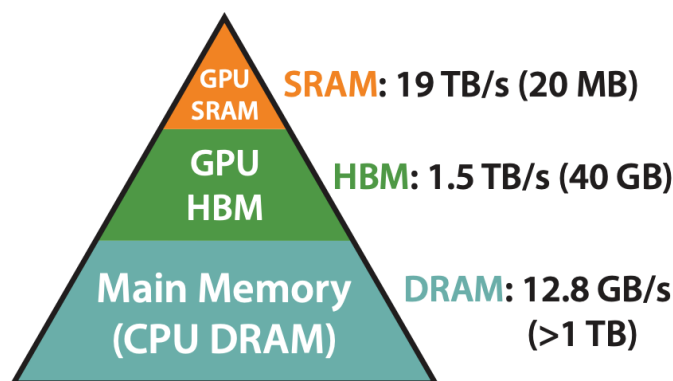
GPU в свою очередь не столько уменьшает задержку, сколько перекрывает ее другой полезной работой: если один варп все еще ждет данные, SM может выполнять инструкции другого варпа.

7 Иерархия памяти

Мы рассмотрели вычислительные блоки GPU. Однако вычисления проводятся над данными, которые где-то необходимо хранить до вычислений и куда-то записывать после.

Хранить данные надо в памяти. Но обращение к памяти не мгновенно и требует некоторого времени. Логично, что те данные, которые чаще всего используются, необходимо держать в наиболее быстрой памяти. При этом, при работе с нейронными сетями часто необходимо хранить гигабайты параметров.

К сожалению, невозможно сделать память одновременно и самой быстрой и при этом максимально объёмной.



**Memory Hierarchy with
Bandwidth & Memory Size**

Рис. 16: Иерархия памяти из статьи
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Таким образом формируется **иерархия памяти**: на одном уровне мы получаем наиболее быструю память, но жертвуем вместимостью, на другом уровне — имеем компромисс, на еще одном, наоборот, получаем очень объёмную память, но жертвуем её скоростью.

7.1 Обмен данными между хостом и девайсом

Наиболее объёмная и дешёвая память — это память диска. На диске мы можем хранить сотни терабайт данных. Однако обращение к такой памяти — самое дорогое. Такие обращения необходимо выполнять асинхронно с остальными операциями, подгружая “наперёд”.

После того, как мы считываем данные с диска, например текст для обучения или веса модели, мы помещаем их в оперативную память CPU.

Обычно, CPU работает со своей памятью, а GPU — со своей. Хотя бывают и исключения, при котором возможно общее физическое адресное пространство. В остальных же случаях, перед тем, как выполнять вычисления на GPU, данные нужно физически переместить из памяти CPU в память GPU, иначе у GPU просто не будет доступа к этим данным.

CPU также называют **хост (host)**, а GPU — **девайс (device)**.

Копирование данных с хоста на девайс не бесплатное. По возможности, если хватает памяти GPU, все параметры модели загружают на девайс до конца обучения, а данные — до конца итерации.

Если же данные, параметры или активации не помещаются в память девайса, их часть выгружают в память хоста. Такой метод называется **cpu offloading**.

Память CPU управляется операционной системой. Она разбита на **страницы (pages)** — небольшие непрерывные участки виртуальной памяти. Операционная система может перемещать страницы, выгружать их, отображать их на разные физические адреса и управлять ими независимо.

Для обычной CPU программы это удобно, но для копирования с CPU на GPU — нет. Девайс и контроллер передачи данных должны работать с физически доступными участками памяти. Если страница может быть перемещена операционной системой во время копирования, то перед копированием её необходимо разместить в **закрепленном буфере (pinned buffer)**.

Поэтому для эффективного копирования данных с хоста на девайс используется **закреплённая память (pinned memory)**. Мы буквально закрепляем страницу на время копирования и не даём операционной системе её переносить, тем самым уменьшая накладные расходы, которые возникают при копировании данных в закрепленный буфер (pinned buffer) (а также, что крайне важно, позволяем выполнять копирование асинхронно).

7.2 Глобальная память (GMEM)

Основная память GPU называется **глобальной памятью (GMEM)**.

Такая память имеет большой объём и высокую пропускную способность по сравнению с памятью хоста. Однако среди всех остальных уровней иерархии памяти GPU она же — самая медленная и может занимать сотни тактов. Часто задача оптимизации вычислений на девайсе сводится к минимизации количества обращений к глобальной памяти.

Однако записывать данные в GMEM необходимо, поскольку именно так происходит общение между разными программами на одном девайсе. Так, в глобальную память сохраняются активации слоёв во время фазы прямого распространения обучения нейронной сети. Эти активации слоёв потом используются при вычислении обратного распространения. Состояния оптимизатора и градиенты также хранятся в GMEM.

7.3 Разделяемая память (SMEM)

Разделяемая память (shared memory, SMEM) — следующий уровень иерархии. Она намного меньше глобальной памяти, но и доступ к ней значительно быстрее, примерно несколько десятков тактов. SMEM используется для хранения данных, которые нужны нескольким потокам, исполняющимся на одном SM.

Мы можем загрузить в SMEM данные из GMEM единожды на время работы программы, а переиспользовать их много раз, что особенно особенно понадобится для матричного умножения. Это что-то вроде кэша программы, который разработчик организует самостоятельно.

Важная проблема при использовании разделяемой памяти — **конфликты банков**. SMEM физически разбита на несколько банков. Если несколько потоков одновременно обращаются к адресам, попадающим в один и тот же банк, доступ может выполняться медленнее.

7.4 Память регистров (RMEM)

Самая быстрая память в иерархии — это регистровая память. Регистры находятся внутри SM. Обычно они используются для хранения локальных переменных потока: индексов, промежуточных значений и так далее.

Операции выполняются над данными, находящимися в регистрах. Если SMEM мы можем не использовать, читая из GMEM напрямую, то использовать RMEM мы обязаны.

Регистры очень быстрые, но их ресурсы ограничены и потоки не могут обмениваться друг с другом данными из RMEM. Если регистров не хватает, то компилятор может использовать более медленную память — **local memory**. Такая ситуация называется **register spilling**. Поэтому очень важно следить за регистровым давлением.

7.5 L1 и L2 кэш

Помимо явно управляемой памяти, в GPU есть кэши. Они делятся на два уровня: L1 и L2.

L1 кэш находится внутри SM. Он кэширует часть обращений к глобальной памяти. На современных видеокартах NVIDIA L1 кэш и SMEM часто используют общий физический ресурс. Соотношение размеров L1 кэша и SMEM не фиксировано. Чем больше используется SMEM, тем меньше остается места под L1.

L2 кэш является общим для GPU. Через него происходят обращения разных SM к глобальной памяти.

В отличие от SMEM, L1 и L2 кэши работают автоматически.

7.6 Согласованный доступ к памяти

Для эффективной работы с глобальной памятью важно не только сколько данных мы читаем, но и то, как именно мы их читаем.

Потоки исполняются варпами. Если потоки одного варпа читают соседние адреса памяти, GPU может объединить эти обращения в одну крупную **транзакцию** памяти, что кратно понижает стоимость обращения к памяти. Это называется **согласованным доступом к памяти (coalesced memory access)**.

То есть, если потоки читают элементы

$$x[i], x[i + 1], x[i + 2], \dots$$

то такой доступ согласованный (коалесцированный). Но если читаются элементы

$$x[i], x[i + D], x[i + 2D], \dots$$

то обращения к памяти будут хуже объединяться в одну транзакцию, вплоть до того, что на каждый такой элемент необходимо будет выделить отдельную транзакцию. Но зачем кому то читать данные таким странным образом?

Тензоры записаны в памяти в виде длинной последовательности элементов. Форма задается с помощью размерностей осей. Для матрицы есть две основных формы записи в памяти — **по строкам (row-major)** и **по колонкам (column-major)**. В первом случае элементы строки матрицы записываются по порядку. Во втором случае, по порядку записываются уже элементы колонки матрицы.

Во время матричного умножения, нам необходимо найти скалярное произведение строки и колонки матрицы. Чтобы доступ к памяти был коалесцированным, необходимо, чтобы левая матрица была записана в row-major форме, а правая — в column-major.

Также часто полезно, чтобы размеры тензоров и смещения в памяти были выровнены и кратны удобным значениям: размеру варпа, ширине шины памяти (векторизованной загрузки). Если не сделать тензоры кратными заранее, в GPU-программе его можно западдить до нужного значения (но это накладные расходы).

8 Метрики производительности GPU

Мы разобрались с устройством вычислений на GPU и иерархией памяти. Но перед тем, как оптимизировать вычисления нейронных сетей, разберемся, как оценивать какие решения выгодно использовать при оптимизации, а какие — нет.

Важно также понимать, чем именно ограничена та или иная операция: скоростью вычислений, скоростью работы с памятью или неудачным паттерном доступа к данным, инициализацией кернелов или синхронизациями.

Для этого используют метрики производительности GPU.

8.1 Задержка и пропускная способность

Первое что приходит в голову, когда сравнить производительность двух программ — давайте оценивать время, которое требуется для выполнения операции или программы.

Эта метрика называется **задержкой (latency)**. Мы можем оценивать как время выполнения всей программы на GPU, так задержку её составляющих: чтения памяти, различных вычислений и записи результата.

Другая метрика, которая часто используется для оценки производительности GPU — это **пропускная способность**. Грубо говоря, это тот объём работы, которая совершается за единицу времени. Например, при обучении или инференса трансформера, мы можем оценивать количество обработанных в секунду токенов.

Нам важно минимизировать задержку и максимизировать пропускную способность.

8.2 FLOP/s

Одна из основных характеристик вычислительной производительности — это количество операций с плавающей точкой, которые GPU способен выполнять за единицу времени, **FLOP/s — floating point operations per second**.

Чтобы оценить FLOP/s, достаточно вычислить количество операций с плавающей точкой FLOPs, необходимых для выполнения программы, и разделить на время работы программы.

$$\text{FLOP/s} = \frac{\text{FLOPs}}{t}$$

Важно отличать пиковое значение FLOP/s от достижимого. В характеристиках GPU обычно указывается пиковое значение — теоретический максимум, который на практике почти недостижим.

Также FLOP/s сильно зависит от типа данных. Один и тот же GPU может иметь разные значения для FP32, FP16, BF16, FP8 или FP4 типов. О них мы поговорим позднее.

8.3 Пропускная способность памяти (memory bandwidth)

Для памяти же одна из основных метрик — это **пропускная способность памяти (memory bandwidth)**. Она показывает, сколько байт данных можно прочитать из памяти или записать в нее за единицу времени.

Если программа читает R байт и записывает W байт за время t , то пропускную способность памяти можно оценить как

$$\text{MemoryBandwidth} = \frac{R + W}{t},$$

также, аналогично FLOP/s, важно различать пиковое значение и достижимое.

Пропускная способность памяти — особенно ценная метрика при выполнении операций, в которых мало арифметики, но много обращений к памяти. Например, при сложении двух векторов

$$c_i = a_i + b_i$$

для каждого элемента необходимо прочитать 8 байт (4 байта для a_i и 4 байта для b_i в FP32 типе) и записать 4 байта. То есть, ради одной операции сложения необходимо передать 12 байт.

Такая функция почти не нагружает вычисления, но нагружает память. Поэтому при оптимизации таких программ важно в первую очередь максимизировать пропускную способность памяти.

8.4 Арифметическая интенсивность

Какие то программы могут быть ограничены пропускной способностью памяти в большей степени, а какие то — в меньшей.

Для оценки, в чем программа ограничена сильнее — в вычислениях или в чтении памяти, используется такая метрика как **арифметическая интенсивность**. Давайте просто разделим FLOP/s на пропускную способность памяти:

$$I = \frac{\text{FLOP/s}}{\text{MemoryBandwidth}}$$

Однако, FLOP/s и MemoryBandwidth неудобно использовать, поскольку обе эти метрики зависят от времени. Поэтому упростим:

$$I = \frac{\frac{\text{FLOPs}}{t}}{\frac{W+R}{t}} = \frac{\text{FLOPs}}{W+R}$$

Теперь арифметическая интенсивность показывает, сколько арифметических операций выполняется на один байт данных, прочитанный или записанный.

8.5 Roofline-модель

Если арифметическая интенсивность низкая, то программа делает мало вычислений на каждый байт данных. Тогда она скорее всего будет ограничена скоростью памяти. Это типично для поэлементных операций (elementwise), редукций, нормализаций, функций активации. Такие операции могут стать бутылочным горлышком в производительности при обучении нейронных сетей.

Если же арифметическая интенсивность высокая, то операция проводит много вычислений над уже загруженными данными. Тогда она может быть ограничена уже скоростью вычислений. Это характерно, например, для больших матричных умножений (но только тех, которые хорошо оптимизированы).

Первый сценарий называется **memory-bound** — программа ограничена скоростью памяти. Второй же — **compute-bound**, когда программа ограничена скоростью вычислений.

Для грубой оценки производительности программы или операции часто используют **roofline-модель**. Производительность операции не может быть выше, чем позволяют мощности девайса (иначе упираемся в compute-bound сценарий), и не может быть выше, чем позволяет пропускная способность памяти (иначе будет memory-bound ограничение).

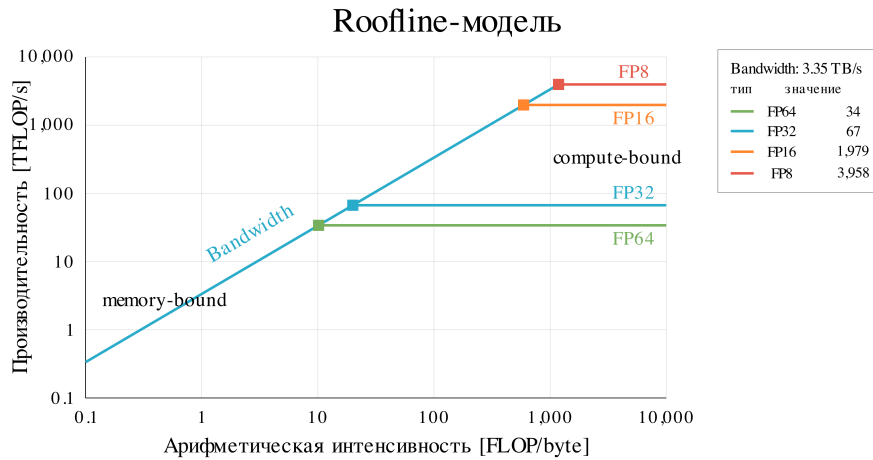


Рис. 17: Roofline-модель.

Максимальная производительность, которую можно получить при ограничении памятью равна:

$$\text{Performance} \leq \text{ArithmeticIntensity} \cdot \text{MemoryBandwidth}$$

В то же время, производительность не может превысить пиковую скорость вычислений PeakFLOP/s:

$$\text{Performance} \leq \text{PeakFLOP/s}$$

Поэтому поэтому максимальную производительность можно записать как:

$$\text{Performance} \leq \min(\text{ArithmeticIntensity} \cdot \text{MemoryBandwidth}, \text{PeakFLOP/s})$$

8.6 Occupancy

Roofline-модель позволяет оценить, в какой области применять оптимизации: в памяти или в вычислениях. Но не дает информации, какие ресурсы

GPU используются в текущей программе.

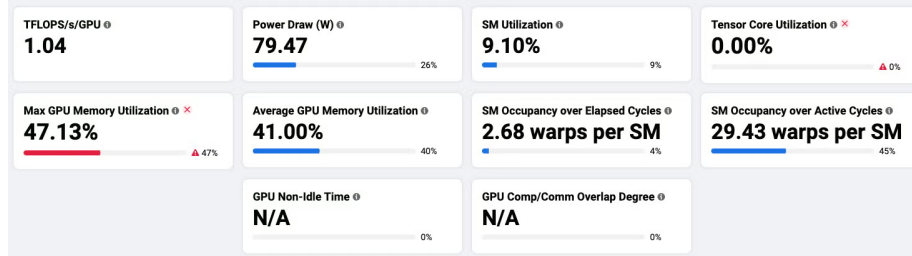


Рис. 18: Метрики производительности GPU в MAIProf.

А ресурсы внутри SM потребляются разные: регистры, тензорные ядра, варп-планировщики, SMEM. И при достижении лимита по одному из этих ресурсов приведет к тому, что остальные будут простаивать.

Например, пусть программа использует слишком много регистров на поток или слишком много SMEM. Тогда на одном SM сможет разместиться меньше потоков и, соответственно, меньше варпов. А при малом количестве варпов на программу планировщику тяжелее снижать задержку через перекрытие времени выполнения варпов. В результате SM будет частично простаивать во время работы программы.

Для описания этого эффекта используется метрика **occupancy**:

$$\text{Occupancy} = \frac{W_{\text{active}}}{W_{\text{max}}},$$

где W_{active} — количество активных варпов во время работы программы, а W_{max} — максимальное количество активных варпов для конкретной архитектуры.

Важно помнить, что occupancy — лишь одна из метрик, по которой можно судить о производительности и по которой можно делать выводы, как оптимизировать программу. Например, программа может иметь высокий occupancy, потому что выполняет матричные вычисления наивным образом, не нагружая SMEM и RMEM. Но такая программа будет memory-bound.

9 Типовые операции нейронных сетей на GPU

Теперь рассмотрим типовые операции нейронных сетей с точки зрения вычисления их на GPU, ведь разные операции по-разному ложатся на архитектуру GPU и, как мы поняли в предыдущем разделе, требуют разных оптимизаций.

9.1 Поэлементные операции

Самый простой тип операции — поэлементные операции. Например, такой операцией является сложение двух тензоров, как при использовании

residual connections:

$$c_i = a_i + b_i$$

или же поэлементно применяются некоторые функции активации, например ReLU:

$$y_i = \max(0, x_i)$$

причем в данном примере $\max$ не является редукцией в полном смысле слова, поскольку вычисляется всего над двумя элементами.

В поэлементных операциях каждый элемент результата зависит только от одного или нескольких элементов входных тензоров с тем же индексом.

Такие операции очень легко распараллеливаются, но в то же время обычно обладают низкой арифметической интенсивностью, что делает их memory-bound операциями.

9.2 Редукции

В нейронных сетях часто встречаются редукции — операции, которые преобразуют массив элементов в скаляр.

В отличие от поэлементных операций, редукции параллелизуются сложнее. В наивном случае, это приведет к тому, что несколько потоков будут пытаться писать значения в одну ячейку памяти. Поэтому редукции требуют дополнительных синхронизаций.

9.3 GEMV

GEMV-операция — это умножение вектора на матрицу. Например преобразование одного вектора с помощью линейного слоя:

$$y = xW$$

Подобная операция часто встречается при инференсе трансформеров с использованием KV-кэша (что мы обсудим позже).

С одной стороны, GEMV хорошо распараллеливаются по столбцам матрицы. С другой стороны, GEMV имеет значительно меньшую арифметическую интенсивность, чем GEMM.

9.4 GEMM

GEMM — это операция матричного умножения, самая важная операция для нейронных сетей:

$$Y = XW$$

Каждый элемент матрицы Y вычисляется независимо друг от друга, а значит параллельно. Таким образом, один поток программы может вычислять один элемент матрицы Y . В наивном виде работа одного потока выглядит так:

$$Y_{ij} = \sum_{k=1}^D X_{ik} W_{kj}.$$

Такой подход имеет крайне низкую арифметическую интенсивность. Но важно то, что данные в GEMM можно переиспользовать.

Так, например, элемент X_{ik} будет также использоваться при вычислении всех элементов i -ой строки матрицы Y , а элемент W_{kj} — при вычислении всех элементов j -го столбца матрицы Y . Эти элементы мы можем кэшировать в SMEM, значительно сокращая количество обращений к GMEM.

Чтобы в свою очередь сократить количество обращений к SMEM, мы можем вычислять одним потоком не 1 элемент матрицы, а, например, 2, 4, 8, или даже 128 элементов (при использовании FP4 или FP8 типов). Однако мы не можем наращивать это число до бесконечности, поскольку такой способ вычисления матриц колоссально увеличивает регистровое давление и снижает оверсапу.

Это позволяет значительно снизить задержку, вызванную чтением данных с GMEM. А чтобы увеличить вычислительную пропускную способность GEMM, используются тензорные ядра.

9.5 Нормализации

Нормализации используются почти во всех современных архитектурах.

Рассмотрим особенности вычисления нормализаций на примере LayerNorm:

$$y_j = \gamma_j \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_j,$$

где

$$\mu = \frac{1}{D} \sum_{j=1}^D x_j$$

и

$$\sigma^2 = \frac{1}{D} \sum_{j=1}^D (x_j - \mu)^2$$

Нормализация выполняется по одному токenu. Для каждого токена нам необходимо выполнить редукции: посчитать среднее и сумму квадратов. Самой арифметики в нормализации немного, что делает нормализацию — memory-bound операцией.

Однако, нормализация сложнее обычных поэлементных операций, потому что редукции также добавляют дополнительные синхронизации.

10 Программная модель CUDA

Перейдем к инструментам программирования на GPU. Начнем с основы работы с GPU NVIDIA — CUDA.

10.1 Технологический стек CUDA

CUDA — это целый стек технологий для работы с GPU.

На нижнем уровне находится **драйвер**. Он отвечает за взаимодействие с устройством: управление GPU, выделение памяти, запуск программ и работу с низкоуровневыми ресурсами.

Над драйвером находится **CUDA Runtime** — набор библиотек, инструментов и API. CUDA Runtime предоставляет более удобный интерфейс для работы по сравнению с драйвером. Так, например, есть функции для выделения памяти и её копирования между хостом и девайсом.

Над CUDA Runtime находится **CUDA C/C++** — расширение языка C++, которое позволяет писать программы, работающие с CUDA Runtime. Такие программы компилируются с помощью специального компилятора — **NVCC** (NVIDIA CUDA Compiler).

Еще выше находятся специализированные библиотеки, такие как cuBLAS и cuBLASLt для операций линейной алгебры, cuDNN с примитивами для нейронных сетей, NCCL для коммуникации между несколькими девайсами и многие другие. Из всех них мы в дальнейшем разберем только NCCL.

10.2 CUDA-кernels

Функция, выполняемая на девайсе, называется **кernels**. Kernel описывает работу одного потока. При запуске таких потоков создаётся много, поэтому каждый поток получает свой индекс и обрабатывает свой элемент массива.

Программа, которую мы пишем, обрабатывается на хосте. Хост также инициализирует запуск kernels. Запуск kernels — не бесплатный. Хост должен отправить работу на GPU, подготовить параметры запуска и поставить операцию в очередь выполнения. Все это приводит к **накладным расходам** — **kernel launch overhead**.

Если kernel большой и выполняет много работы, то эти накладные расходы будут несущественны. Но если программа состоит из большого числа маленьких kernels, то kernel launch overhead становится существенным.

10.3 Потоки, блоки, сетка

Вычисления в CUDA организованы иерархически:

$$\text{Grid} \rightarrow \text{Blocks} \rightarrow \text{Threads}$$

Поток (thread), как мы ранее обсуждали, это логическая единица выполнения программы, не аппаратная. Каждый поток в CUDA работает в

рамком исполнительной модели SIMT: он исполняет один и тот же код керна, но работает со своим индексом и со своими данными.

Потоки объединяются в варпы по 32 штуки, однако такой ступени иерархии в CUDA явно нет (хотя на аппаратном уровне вычисления все равно происходят именно варпами).

Зато есть **блоки (blocks)** — тоже группы потоков. В отличие от варпов, блоки не имеют фиксированный размер, он задается программистом (но обычно он кратен 32, поскольку иначе будет вызван варп, некоторые потоки которого будут отключены). Потоки внутри блока могут взаимодействовать друг с другом: обращаться к общей SMEM и синхронизироваться. В одном CUDA-блоке может максимум 1024 потока (то есть 32 варпа).

Набор блоков, которые запускаются при вызове керна, называется **сетка (grid)**.

10.4 Индексация

Каждый поток имеет свой индекс внутри блока, каждый блок имеет свой индекс внутри сетки. Чтобы определить глобальный индекс потока, в кернах также доступны размер блока и размер сетки.

Поскольку часто CUDA-кернам необходимо работать с многомерными массивами, для упрощения работы используется индексация потоков внутри блока и блоков внутри сетки по трём осям: x , y и z .

10.5 Синхронизации

Потоки асинхронны и могут запускаться в любом порядке. Если программа построена таким образом, что какой-то поток зависит от другого, то необходимо убедиться, что зависимый поток продолжит свое исполнение только после того, как основной поток пройдет некоторый этап программы.

В CUDA есть несколько синхронизаций: синхронизация на уровне девайса, синхронизация на уровне керна и синхронизация на уровне варпа.

Синхронизации стоит рассматривать как барьер, который ждет завершения выполнения всех участников группы, чтобы продолжить код.

В первом случае синхронизация проверяет, что все кернелы на всех стримах выполнены и безопасно начинать новую задачу.

Во втором случае синхронизация ставит барьер для работы потоков внутри керна. Это полезно, если работа на нескольких потоках идет с одним и тем же участком памяти и требуется “выравнивание” работы всех потоков, прежде чем вносить изменения, чтобы избежать такой проблемы, как race conditions.

Третий случай аналогичен второму, но синхронизация происходит не между всеми потоками, а между потоками одного варпа.

10.6 Race Conditions

Race conditions (гонки) — ситуация, когда результат зависит от недетерминированного порядка выполнения потоков, потому что несколько потоков без должной синхронизации читают или записывают в одни и те же ячейки памяти. Грубо говоря, разные потоки исполняются наперегонки, без необходимой синхронизации своих инструкций.

Чаще всего race conditions вызваны гонками данных, когда два потока пытаются изменить данные по одному адресу и между ними нет никакого барьера.

Другая ситуация — один поток должен прочесть результат, а второй поток этот результат еще не записал в нужную ячейку памяти.

Гонки опасны тем, что приводят к некорректному результату вычислений без явного падения программы.

Ранее мы говорили о конфликтах банков в SMEM. Конфликты банков не являются гонками. В отличие от гонок, конфликты банков приводят лишь к потере производительности.

10.7 Редукции

Редукции требуют более сложной организации параллелизма, чем обычные вычисления. Рассмотрим на примере суммы массива:

$$s = \sum_{i=1}^N x_i$$

При наивном подходе, все потоки одновременно добавляют свои значения в одну переменную, что приводит к гонкам и некорректному результату.

Воспользуемся ассоциативностью сложения, будем вычислять редукцию по дереву.

Разделим с округлением вверх количество элементов массива на размер блока. Полученное значение — это количество элементов массива, за которые ответственен 1 поток. Найдем частичные суммы s_i для каждого потока, последовательно сложив все элементы.

Внутри одного варпа мы можем разбить потоки (и соответствующие элементы) на пары.

$$\underbrace{s_0, s_1}_{\text{Пара 1}}, \underbrace{s_2, s_3}_{\text{Пара 2}}, \dots, \underbrace{s_{28}, s_{29}}_{\text{Пара 15}}, \underbrace{s_{30}, s_{31}}_{\text{Пара 16}}$$

Поток с меньшим индексом прибавит к своему элементу элемент соседа. После чего останется 16 частичных сумм:

$$\underbrace{s_0 + s_1}_{z_0}, \underbrace{s_2 + s_3}_{z_1}, \dots, \underbrace{s_{28} + s_{29}}_{z_{14}}, \underbrace{s_{30} + s_{31}}_{z_{15}}$$

На следующем шаге суммы снова объединяются попарно:

$$\underbrace{z_0, z_1}_{\text{Пара 1}}, \underbrace{z_2, z_3}_{\text{Пара 2}}, \dots, \underbrace{z_{12}, z_{13}}_{\text{Пара 7}}, \underbrace{z_{14}, z_{15}}_{\text{Пара 8}},$$

Повторим эти шаги в сумме 5 раз (поскольку $\log_2 32 = 5$) и в первом потоке варпа окажется сумма по всем потокам варпа. Между каждыми такими шагами необходимо синхронизировать потоки внутри варпа, чтобы избежать гонок.

Эту частичную сумму сохраним в SMEM. Редукция внутри варпа по дереву называется **варп-редукцией**.

Поскольку в одном блоке может быть максимально 32 варпа, то в SMEM может быть сохранено максимально 32 частичные суммы. А это значит, что мы можем повторить варп-редукцию по частичным суммам в SMEM и получить итоговую сумму.

10.8 Стримы и асинхронность

Кернелы запускаются с хоста, но обычно выполняются на девайсе асинхронно относительно хоста, что означает, что CPU может поставить операцию на выполнение и продолжить работу, не дожидаясь немедленного завершения кернела на девайсе.

Для организации таких операций используются стримы (**streams**) — очереди операций на девайсе. Операции внутри стрима выполняются в порядке добавления.

Если операции находятся в одном стриме, то следующая операция начнётся только после завершения предыдущей. Однако разные стримы могут выполняться асинхронно относительно друг друга.

Это важно для обучения нейронных сетей. Например, на одном стриме производятся вычисления прямого и обратного этапов, а также обновляются параметры сети, на втором стриме находятся коммуникации между разными девайсами, третий стрим загружает на девайс следующий батч и так далее.

Причем, второй и третий стримы можно “спрятать” под первым стримом.

Однако, асинхронность усложняет общение между стримами и оценку времени работы кернела. Чтобы избежать ненужных синхронизаций в первом случае и корректно оценивать время работы различных участков CUDA-программы во втором случае, используются CUDA-события.

10.9 CUDA-события

CUDA-события — это маркеры, которые можно записать в стрим. Когда девайс доходит до этого маркера, событие считается выполненным.

CUDA-события очень важны при работе с несколькими стримами. Например, при обучении нейронных сетей на нескольких девайсах.

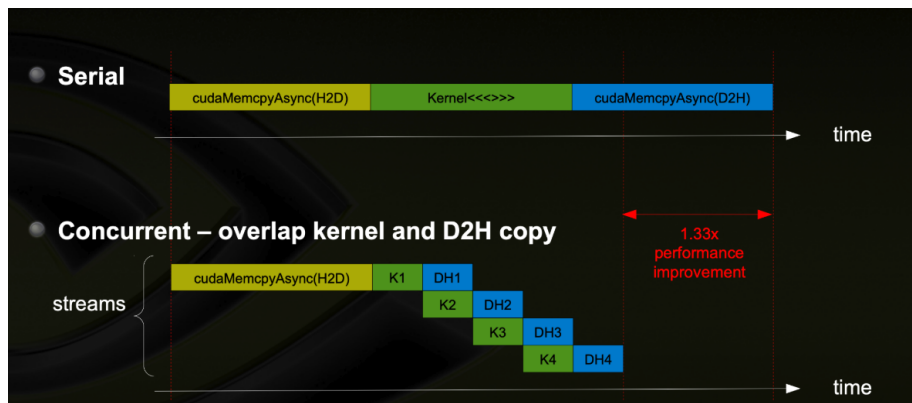


Рис. 19: На этом примере разделение выполнения ядра и копирования данных с девайса на хост на 4 стрима позволяет спрятать большую часть задержки выполнения копирования под выполнение ядра

Например, перед вычислением слоя, нам необходимо дождаться сборки необходимых данных с нескольких девайсов через ядро коммуникации. Ядра коммуникации выполняются на отдельном стриме. Если стрим вычислений начнет вычисление слоя до того, как завершится стрим коммуникаций и с других девайсов будут собраны необходимые данные, то вычисления пойдут.

Это поведение напоминает *race conditions*. Гонки мы решали тем, что добавляли синхронизации. Добавим синхронизацию девайса.

Однако если ядро коммуникации завершится до того, как начнется ядро вычисления слоя, то стрим коммуникаций будет простаивать из-за синхронизации, хотя он мог бы начать выполнение следующей коммуникации (например, собирать данные для еще одного слоя).

CUDA-события позволяют нам поставить маркер после завершения ядра коммуникации и запретить выполнение ядра вычислений до того, как ивент на стриме коммуникаций будет выполнен.

Другой пример — замер времени выполнения ядра. Поскольку код, который замеряет время, исполняет CPU, а ядра запускаются асинхронно, то он будет замерять лишь время инициализации ядра (что в некотором смысле тоже может быть полезно, но это уже другая задача).

Мы можем поставить CUDA-события в стриме сразу перед ядром и сразу после него, а затем посчитать время между ними.

11 Оптимизация вычислений нейронных сетей на GPU

Из roofline модели следует, что есть два основных направления оптимизации вычислений нейронных сетей на GPU: оптимизация вычислений и оптимизация памяти.

Причем, как мы ранее рассматривали, скорость вычисления многих слоёв нейронных сетей на GPU ограничена по памяти (memory-bound сценарий).

Рассмотрим основные методы оптимизации вычислений на GPU, применимые для нейронных сетей.

11.1 Тайлинг в GEMM

Основная операция в нейронных сетях — матричное умножение, GEMM:

$$C = AB,$$

где

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}$$

При наивном вычислении GEMM, мы считаем каждый элемент C как скалярное произведение строки матрицы A на столбец матрицы B .

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}$$

Это приводит к тому, что на одну операцию произведения и одну операцию сложения приходится чтение 8 байтов из GMEM, то есть арифметическая интенсивность такого алгоритма крайне низкая.

Как ранее обсуждалось, при таком подходе одни и те же элементы матриц A и B будут много раз загружаться из глобальной памяти.

Поэтому, снизим количество обращений к глобальной памяти за счёт того, что будем “кэшировать” некоторые элементы в разделяемой памяти, чтобы они были доступны всем потокам блока.

Для этого разобьём вычисление C на блоки меньшего размера $C_{I,J}$, где I — набор строк соответствующего блока, J — набор столбцов. Такой подход называется **тайлинг**.

Этот блок, называемый тайлом, теперь можно представить как сумму произведений тайлов:

$$C_{I,J} = \sum_T A_{I,T} B_{T,J},$$

где T пробегает блоки по внутренней размерности K .

За вычисления одного элемента тайла ответственен один поток, а за вычисление всего тайла $C_{I,J}$ — блок потоков.

Для вычисления одного слагаемого $C_{I,J}^1 = A_{I,1}B_{1,J}$ этой суммы мы можем загрузить тайлы $A_{I,1}$ и $B_{1,J}$ в SMEM. Тогда, чтобы вычислить $C_{I,J}^1$, можно читать данные не из медленной GMEM, а из гораздо более быстрой SMEM. А результат записывать практически бесплатно в RMEM.

Потом мы можем аналогичным образом вычислить $C_{I,J}^2 = A_{I,2}B_{2,J}$. Поскольку каждый поток блока ответственен только за свой элемент тайла $C_{I,J}$, то мы можем просто прибавить результат $C_{I,J}^2$ к $C_{I,J}^1$.

Так, пока в цикле выполняются вычисления $C_{I,J}^t$, каждый поток может хранить в RMEM кумулятивную сумму элемента $C_{I,J}$, за который он ответственен.

Арифметическая интенсивность тайлинга Выглядит все так, будто мы лишь раздули количество операций чтения и записи, а значит снизили арифметическую интенсивность.

Так и есть. Но только формально.

Вот арифметическая интенсивность для наивного GEMM:

$$I_{\text{naive}} = \frac{2 \text{ FLOP}}{2 \cdot 4 \text{ bytes}} = \frac{1}{4} \text{ FLOP/byte}$$

Посчитаем арифметическую интенсивность для тайлинга. Отдельно оценим обращения к GMEM и SMEM. Число операций для вычисления $C_{I,J}^t$ равно:

$$\text{FLOP}_{\text{tile}} = 2B_M B_N B_K,$$

где:

- B_M — размер тайла по размерности M ,
- B_N — размер тайла по размерности N ,
- B_K — размер тайла по размерности K .

Из GMEM нужно прочитать два тайла:

$$\text{Bytes}_{\text{GMEM}} = 4(B_M B_K + B_K B_N)$$

При этом из SMEM потоки читают элементы тайлов $A_{I,T}$ и $B_{T,J}$ для вычисления своих элементов $C_{I,J}$. Каждый поток читает B_K элементов из тайла A и B_K элементов из тайла B . Тогда:

$$\text{Bytes}_{\text{SMEM}} = 2 \cdot 4B_M B_N B_K$$

Тогда арифметическая интенсивность будет равна

$$I_{\text{tiled}} = \frac{2B_M B_N B_K}{\text{Bytes}_{\text{GMEM}} + \text{Bytes}_{\text{SMEM}}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N) + 2 \cdot 4B_M B_N B_K}$$

Если упростить, то

$$I_{\text{tiled}} = \frac{1}{2/B_N + 2/B_M + 4} \leq \frac{1}{4} \text{ FLOP/byte}$$

Видно, что арифметическая интенсивность тайлинга всегда будет хуже, чем арифметическая интенсивность наивного подхода.

Однако, это утверждение было бы верно только в том случае, если бы стоимость обращений к GMEM была равна стоимости обращений к SMEM.

На практике же, в современных архитектурах GPU задержка обращения к SMEM может быть на 2 порядка меньше, чем задержка обращения к GMEM. Поэтому при подсчёте арифметической интенсивности, мы можем не учитывать обращения к SMEM. А значит

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N)} = \frac{1}{2/B_N + 2/B_M}$$

Таким образом:

$$I_{\text{naive}} \leq I_{\text{tiled}}^{\text{GMEM}}$$

и равенство достигается только в том случае, если $B_N = 1$ и $B_M = 1$ (то есть когда тайлинг по сути отсутствует).

11.2 Повышение арифметической интенсивности

Из выражения

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{1}{2/B_N + 2/B_M}$$

можно сделать вывод, что чем больше B_N и B_M , то есть размерности тайла матрицы C , тем выше арифметическая интенсивность. А значит, чтобы ускорить вычисление GEMM, нужно увеличивать размер тайла.

Ранее мы выполняли GEMM с расчётом, что каждый поток ответственен за свой элемент итоговой матрицы, а блок потоков — за тайл. Если мы будем увеличивать размерность итогового тайла, то упрёмся в максимальный размер блока: 1024 потока.

Чтобы перешагнуть через этот блок, пусть каждый поток считает не один элемент, а небольшой набор — свой подтайл (subtile). Кумулятивные суммы в таком случае будут храниться в виде массива в RMEM.

Таким образом, итоговый тайл становится больше, а число потоков остаётся тем же. Однако, такое решение не является бесплатным, поскольку увеличивается регистровое давление и потребление SMEM. Это в свою очередь может привести к уменьшению оверсапу.

11.3 Вычисления в пониженной точности

Мы увеличили арифметическую интенсивность посредством уменьшения количества обращений к GMEM. Однако, в формуле арифметической интенсивности

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{1}{2/B_N + 2/B_M},$$

вернее, в немного другой ее версии

$$I_{\text{tiled}}^{\text{GMEM}} = \frac{2}{4(1/B_N + 1/B_M)}$$

можно заметить константу 4 — это количество байт, необходимое для хранения одного числа.

Обычно, при обучении нейронных сетей используются числа с плавающей точкой. Базовый формат — `float32`, где 32 означает число бит, то есть 4 байта. Такое число представляется следующим образом:

- 1 бит — знак s ,
- 8 бит — показатель степени e ,
- 23 бита — мантисса m .

Нормализованное значение (то есть не описывающее специальные случаи, такие как 0, NaN, inf и тд) вычисляется по формуле

$$(-1)^s \cdot 2^{e-\text{bias}} \cdot 1.m,$$

где для `float32` $\text{bias} = 127$, а $1.m$ означает, что перед дробной частью мантиссы неявно стоит ведущая единица (таким образом мы не храним ее явно, что позволяет использовать еще один бит точности).

Но, на практике такая высокая точность оказывается избыточной для некоторых операций нейронных сетей. Значения весов чаще всего лежат около нуля, а небольшая погрешность при их изменении может не оказывать существенного влияния на результат.

Разумным решением будет снизить точность и использовать для хранения весов тип `float16`. Такое число будет представлено в памяти уже следующим образом:

- 1 бит — знак,
- 5 бит — показатель степени,
- 10 бит — мантисса

Уменьшение количества байт на параметр сети само собой также уменьшит количество требуемой для модели памяти. А больший объем памяти в свою очередь позволит обучать большие модели.

Однако веса — не единственные тензоры, которые можно хранить в пониженной точности. Во время обучения значительный объем памяти занимают также активации.

Активации нужны не только во время вычисления прямого прохода — они также участвуют в вычислении обратного распространения ошибки: после вычисления слоя их приходится сохранять, поскольку они необходимы для вычисления градиентов. Объем памяти, занимаемый активациями, может превосходить объем памяти, занимаемый весами.

Таким образом, уменьшив количество байтов, необходимых для хранения данных, вдвое, мы увеличим вдвое арифметическую интенсивность. В частности, арифметическую интенсивность матричного умножения:

$$I_{\text{tiled, fp16}}^{\text{GMEM}} = \frac{2}{2(1/B_N + 1/B_M)} = \frac{1}{1/B_N + 1/B_M}$$

11.4 Переполнение и округление значений с плавающей точкой

Но у `float16` есть важный недостаток: у него меньше бит под показатель степени. Поэтому диапазон представимых значений существенно уже, чем у `float32`:

$$\text{float32: } \approx 1.18 \cdot 10^{-38} \dots 3.4 \cdot 10^{38},$$

$$\text{float16: } \approx 6.10 \cdot 10^{-5} \dots 6.55 \cdot 10^4$$

`float16` хуже представляет большие числа. Конечно, параметры модели в большинстве своем лежат в интервале $(-1; 1)$. Но при вычислении, например, суммы или экспоненты, возможно, что итоговое значение окажется вне этого диапазона. Если оно будет храниться в `float16`, то случится **переполнение типа с плавающей точкой (float overflow)**.

Чтобы избежать подобной ситуации, был разработан тип данных `bfloat16`, где:

- 1 бит — знак;
- 8 бит — показатель степени;
- 7 бит — мантисса.

Поскольку `bfloat16` сохраняет столько же бит под показатель степени, что и `float32`, то и диапазон значений у него будет близок к диапазону `float32`.

Цена этого — меньшая мантисса. Вещественная числовая прямая непрерывна, однако представление чисел конечным количеством бит дискретизирует множество значений, которые мы можем хранить.

В числах с плавающей точкой степень дискретизации зависит от масштаба числа и от количества бит мантиссы. Чем больше число — тем большая будет погрешность при округлении к наиболее близкому представлению, поскольку чем больше масштаб, тем больше становится расстояние между ближайшими числами, представимыми в такой записи. А это расстояние уже напрямую зависит от точности мантиссы.

Это приводит к тому, что при выполнении арифметики с большими по модулю числами, мы будем накапливать все большую и большую погрешность.

Уже при хранении данных в 16 битах у нас возникают проблемы с диапазоном и точностью. Но это не значит, что мы на этом остановимся! Существуют архитектуры GPU, поддерживающие аппаратные вычисления в 8-битной точности и в даже 4-битной точности. Однако, при таких вычислениях нам необходимо применять некоторые хитрости, о которых мы поговорим позднее.

11.5 Тензорные ядра в GEMM

Рассмотрим арифметическую интенсивность вычисления GEMM в `bfloat16` и с тайлингом, блоками 128×128 элементов, где каждый поток считает участок в 4×4 элементов:

$$I_{\text{tiled, bf16}}^{\text{GMEM}} = \frac{1}{1/128 + 1/128} = 64$$

Это достаточно высокий результат. Возможно теперь стоит сместить фокус внимания с оптимизации памяти на оптимизацию вычислений? Действительно, в погоне за арифметической интенсивностью мы заставили один поток считать сразу 16 элементов, что, откровенно говоря, ломает саму идею GPU — параллелизм вычислений.

Как мы ранее рассматривали, в современных GPU для матричных умножений есть специальные аппаратные блоки — тензорные ядра. Они предназначены для быстрого выполнения операций матричного умножения и аккумуляции результата на небольших матричных блоках:

$$C_{I,J}^R \leftarrow C_{I,J}^R + A_{I,T}^R B_{T,J}^R,$$

здесь $C_{I,J}^R$, $A_{I,T}^R$ и $B_{T,J}^R$ — блоки, хранящиеся в памяти регистров. То есть работа с данными остаётся прежней:

$$\text{GMEM} \rightarrow \text{SMEM} \rightarrow \text{RMEM},$$

Однако, вычисления смещаются с уровня потоков на уровень варпов. То есть вычисления внутри регистрового тайла выполняют не CUDA-ядра,

последовательно находя значения каждого из, например, 16 элементов. Теперь фрагмент блока, за который ответственен один **варп**, вычисляется параллельно с помощью тензорных ядер.

Тензорные ядра наиболее эффективны для вычислений в пониженной точности, аккумулируя при этом значения в полной точности, чтобы минимизировать накапливаемую погрешность.

Арифметическая интенсивность при этом не меняется. Зато, с точки зрения гоofline-модели, тензорные ядра позволяют поднять вычислительный потолок. Так что если операция была compute-bound, то использование тензорных ядер может дать большое ускорение (в отличие от memory-bound сценариев).

При этом тензорные ядра не ускоряют любую операцию в нейронной сети. Они предназначены прежде всего для матричных операций или близких к ним. Но поскольку GEMM — одна из основных операций в нейронных сетях, тензорные ядра могут быть эффективны при вычислении:

- FFN-модулей;
- механизма внимания;
- свёрток

но вряд ли будут полезны при выполнении редукций или поэлементных операций.

11.6 Слияние кернелов (kernel fusion)

Такие оптимизации, как тайлинг, повышающий арифметическую интенсивность, и использование тензорных ядер, не применимы к поэлементным операциям и нормализациям.

Основная проблема этих операций — слишком низкая арифметическая интенсивность, поскольку каждый раз необходимо загружать данные из глобальной памяти.

Однако, слои нейронной сети, в основном, выполняются последовательно. И те результаты, которые мы получили на предыдущем слое, необходимо загрузить в глобальную память. А для следующего слоя, эти же самые результаты необходимо снова достать из глобальной памяти.

При выполнении кернела требуется сохранить результат в глобальную память в конце его работы, иначе этот результат исчезнет. Поэтому, чтобы избежать лишних перемещений данных, пусть один кернел ответственен за вычисление сразу нескольких слоёв.

Например, кернел, в котором мы выполняли матричное умножение, пусть до выгрузки результата в GMEM, например, прибавит к этому результату смещение и применит функцию активации.

Это уменьшает проблему низкой арифметической интенсивности таких операций, поскольку они теперь либо требуют значительно меньшего чтения данных из GMEM, либо не требуют их вовсе.

Этот подход называется **слиянием ядер** (**kernel fusion**).

Однако, слияние ядер — не бесплатная оптимизация. Объединение слишком большого количества операций может увеличить регистровое давление и давление на SMEM, что в свою очередь может привести к падению оверхеда.

К сожалению, по этой причине мы не можем сделать один большой ядро для всей модели. Однако слияние ядер крайне полезно в следующих сценариях (в том числе после GEMM):

- несколько последовательных поэлементных операций подряд;
- последовательные операции смещения и применения функции активации;
- последовательные операции применения остаточных связей и нормализации

Также мы можем объединить операции механизма внимания в один ядро. Такой ядро называется **Flash Attention**.

12 Flash Attention

12.1 Проблемы наивного внимания

Рассмотрим одну голову внимания. Механизм внимания внутри этой головы вычисляется следующим образом:

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right),$$

$$O = AV$$

При наивной реализации, вычисление внимания будет разбито на несколько этапов:

1. Вычислить $S = QK^T / \sqrt{d}$, после чего результат записать в GMEM;
2. Вычислить $A = \text{Softmax}(S)$, после чего результат также записать в GMEM;
3. Вычислить $O = AV$, после чего результат также записать в GMEM.

Кроме постоянного чтения и записи данных в GMEM, которые как мы уже обсуждали, очень дорогие, есть проблема гораздо серьезнее: материализация матриц S и A в глобальной памяти.

Эти матрицы имеют размер $N \times N$, где N — длина обрабатываемой последовательности токенов. Таким образом, нам необходимо хранить $2 \cdot N^2$

активаций. Если длина последовательности большая, например 10^5 элементов, то нам придется хранить только на одном слое $2 \cdot 10^{10}$ значений, которые займут примерно 40 гигабайт памяти. То есть для трансформера со всего восемью слоями, только эти *промежуточные* активации будут занимать объем памяти, равный всему объему памяти четырех GPU H100 по 80 гигабайт.

12.2 Идея Flash Attention

Flash Attention, в свою очередь, не материализует матрицы S и A . Вместо этого воспользуемся идеей тайлинга: будем вычислять итоговую матрицу по O по блокам, при этом объединив все операции в один кернел.

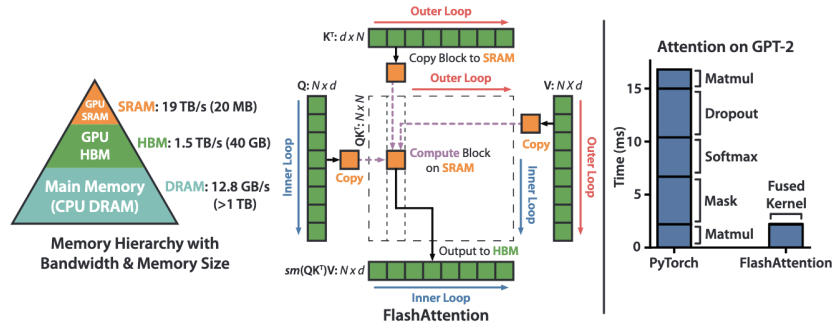


Рис. 20: Схема Flash Attention.

Такой подход позволит не только избавиться от хранения промежуточных активаций, но и применить оптимизации, которые мы рассматривали ранее: 'кэшировать' данные в SMEM и использовать тензорные ядра.

12.3 Численно стабильный Softmax (Safe Softmax)

Перед тем, как перейти к блочному вычислению внимания, разберем вычисление Softmax.

В обычном случае мы применяем Softmax к одной строке следующим образом:

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)}$$

Мы бы хотели проводить вычисления в пониженной точности — в bf16 типе. Однако, ранее мы пришли к выводу, чем больше масштаб значений, тем выше погрешность вычислений. Наиболее неустойчивы операции к таким погрешностям — вычисление экспоненты и суммирование длинных последовательностей.

К сожалению, так совпало, что при вычислении Softmax нам необходимо делать и то, и другое одновременно — вычислить сумму экспонент.

Если же мы будем использовать `fp16` тип, то рискуем получить переполнение.

Чтобы избежать этих проблем, используется **численно стабильная версия Softmax (Safe Softmax)**. Воспользуемся инвариантностью Softmax к сдвигу:

$$\text{Softmax}(x - m)_i = \frac{\exp(x_i - m)}{\sum_{j=1}^N \exp(x_j - m)} = \frac{\exp(-m) \exp(x_i)}{\exp(-m) \sum_{j=1}^N \exp(x_j)} = \frac{\exp(x_i)}{\sum_{j=1}^N \exp(x_j)},$$

то есть

$$\text{Softmax}(x)_i = \text{Softmax}(x - m)_i$$

Чтобы избежать большого масштаба элементов, будем вычитать из каждого элемента максимум по строке:

$$m = \max_j x_j$$

таким образом, все элементы становятся неположительными, а значит значения экспонент будут находиться в полуинтервале $(0; 1]$.

В наивной реализации это сделать просто, потому что вся строка S_i уже материализована в GMEM. Мы можем пройти по ней, найти максимум, затем еще раз пройти и посчитать сумму экспонент.

Однако, во Flash Attention мы не хотим считать всю строку целиком, мы хотим обрабатывать матрицы по блокам. Поэтому обычного safe softmax недостаточно: нужно научиться пересчитывать максимум и сумму экспонент постепенно, по мере обработки блоков.

12.4 Online Softmax

Во Flash Attention используется версия численно стабильного Softmax, которую удобно применять при тайлинге — **online-softmax**.

Пусть строка x разбита на несколько блоков:

$$x = [x^{(1)}, x^{(2)}, \dots, x^{(T)}]$$

Нам нужно посчитать softmax по всей строке x , но при этом не хранить всю строку целиком. Для этого будем обрабатывать блоки последовательно.

Для численно стабильного softmax нам нужны две величины:

$$m = \max_j x_j, \quad l = \sum_j \exp(x_j - m)$$

Тогда:

$$\text{Softmax}(x)_i = \frac{\exp(x_i - m)}{l}.$$

В online-softmax мы будем поддерживать эти величины постепенно. Пусть после обработки $t - 1$ блоков у нас уже есть максимум $m^{(t-1)}$ среди уже обработанных элементов, и

$$l^{(t-1)} = \sum_{\text{old}} \exp(x_j - m^{(t-1)}),$$

сумма экспонент, посчитанная относительно этого максимума.

Обработаем новый блок $x^{(t)}$. Сначала найдем максимум внутри этого блока:

$$m_t = \max_j x_j^{(t)}$$

Новый максимум по всем обработанным элементам будет равен

$$m^{(t)} = \max(m^{(t-1)}, m_t)$$

Теперь нужно обновить сумму экспонент. Посчитаем сумму по блоку с новым максимумом:

$$\hat{l}^{(t)} = \sum_{x_j \in x^{(t)}} \exp(x_j - m^{(t)})$$

Но старую сумму нельзя просто сложить с суммой по новому блоку, потому что старые значения были нормированы относительно другого максимума. Поэтому нужно перенормировать значения

$$l^{(t)} = \frac{\exp(m^{(t-1)})}{\exp(m^{(t)})} l^{(t-1)} + \hat{l}^{(t)}$$

или если упростить

$$l^{(t)} = \exp(m^{(t-1)} - m^{(t)}) l^{(t-1)} + \hat{l}^{(t)}$$

Теперь мы можем применить softmax к тайлингу механизма внимания.

12.5 QKV-тайлинг

Разобьем матрицы Q , K и V на блоки. Пусть мы хотим вычислить блок строк O_i , соответствующий блоку строк запросов Q_i . Для этого нужно учесть все блоки ключей и значений:

$$(K_1, V_1), (K_2, V_2), \dots, (K_T, V_T)$$

На каждой итерации мы берем текущий блок K_t и вычисляем блок матрицы оценок внимания $S_{i,t}$:

$$S_{i,t} = \frac{Q_i K_t^T}{\sqrt{d}}.$$

Далее сразу используем этот блок для обновления Softmax. Найдём максимумы по строкам:

$$m_t = \max_j S_{i,t}^{(:,j)}$$

m_t в данном случае — не число, а вектор. Обновим максимум:

$$m^{(t)} = \max(m^{(t-1)}, m_t).$$

Затем посчитаем сумму экспонент по текущему блоку относительного нового максимума:

$$\hat{l}^{(t)} = \sum_j \exp(S_{i,t}^{(j)} - m^{(t)})$$

и обновим сумму экспонент:

$$l^{(t)} = \exp(m^{(t-1)} - m^{(t)})l^{(t-1)} + \hat{l}^{(t)},$$

$l^{(t)}$ — тоже вектор.

В механизме внимания нам нужна не сама матрица весов A , а результат умножения на V :

$$O_i = A_i V$$

Матрицу весов внимания можно переписать таким образом:

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{\sum_k \exp(S_{ik} - m_i)},$$

где знаменатель — наша накапливаемая сумма экспонент. То есть

$$A_{ij} = \frac{\exp(S_{ij} - m_i)}{l^{(t)}},$$

Не будем сразу делить на $l^{(t)}$ — всё-таки, это не конечный результат. Поэтому вместе с m и l будем поддерживать ненормированную сумму текущего блока $U^{(t)}$, где значение по старым блокам равно

$$U^{(t-1)} = \sum_{\text{old } j} \exp(S_{ij} - m^{(t-1)})V_j$$

$U^{(t)}$ напоминает $l^{(t)}$. Вычислим для текущего блока вклад в эту сумму:

$$\hat{U}^{(t)} = \exp(S_{i,t} - m^{(t)})V_t$$

Тогда старую сумму нужно нормировать относительно нового максимума:

$$U^{(t)} = \exp(m^{(t-1)} - m^{(t)})U^{(t-1)} + \hat{U}^{(t)}.$$

Тогда после обработки всех блоков итоговый выход получается равным:

$$O_i = \frac{U^{(T)}}{l(T)}$$

В глобальную память записываем только результат O_i — несколько строк итоговой матрицы.

12.6 Сравнение наивного внимания и Flash Attention

Важно, что Flash Attention не меняет математическую формулу внимания. Мы по-прежнему считаем

$$O = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

только последовательность действий не как в наивном механизме внимания, где мы выполняем все эти шаги по порядку. Вместо этого, каждый блок потоков сразу считает O_i , постепенно обновляя результат, по мере прохождения по блокам ключей и значений

$$(K_1, V_1), (K_2, V_2), \dots, (K_T, V_T).$$

12.7 Маскированное внимание в Flash Attention

Пока мы рассмотрели только полное внимание — когда каждый токен может смотреть на каждый другой токен. Однако на практике почти всегда используется маска внимания.

В наивной реализации маска применяется к матрице оценок внимания S перед softmax. Если элемент запрещен маской, то ему присваивается значение $-\infty$. Тогда после softmax этот элемент получит нулевой вес, а значит не будет участвовать во взвешенной сумме

$$O_i = \sum_j A_{ij} V_j$$

Во Flash Attention полной матрицы S нет. Есть только текущий блок:

$$S_{i,t} = \frac{Q_i K_t^T}{\sqrt{d}}$$

Поэтому маску нужно применять сразу к блоку $S_{i,t}$ до обновления online-softmax.

Обозначим замаскированный блок через $\tilde{S}_{i,t}$, где каждый элемент:

$$\tilde{s}_{i,t} = \begin{cases} s_{i,t}, & \text{если позиция разрешена,} \\ -\infty, & \text{если позиция запрещена} \end{cases}$$

Дальше все вычисления выполняются не с $S_{i,t}$, а с $\tilde{S}_{i,t}$. Поскольку $\exp(-\infty) = 0$ и $-\infty$ точно не будет больше максимума, то замаскированные элементы не влияют ни на максимум, ни на сумму экспонент, ни на $U^{(t)}$.

В авторегрессионной задаче мы маскируем все элементы выше главной диагонали. При блочном вычислении для каждого такого блока важно понимать, где он расположен:

- если блок полностью ниже главной диагонали, то все элементы разрешены, мы можем считать его как обычный блок внимания;
- если блок полностью выше главной диагонали — все элементы запрещены, его можно не считать вообще;
- если блок пересекает диагональ — часть элементов разрешена, а часть запрещена, значит необходимо применить маску.

Возникает вопрос: но ведь мы считаем блок из нескольких строк матрицы O_i . Но для его вычисления мы всё равно идём по блокам ключей K_t , и блокам значений V_t . На каждой итерации возникает прямоугольный блок оценок внимания. И если этот блок полностью замаскирован — значит мы можем не проводить вычисления с соответствующими K_t и V_t .

12.8 Обратное распространение в Flash Attention

Все это время мы рассматривали только кернел, соответствующий прямому распространению в механизме внимания. Теперь необходимо взяться за самую сложную часть. Механизм внимания необходимо обучать, а значит нам нужен алгоритм Flash Attention для обратного распространения.

В наивной реализации выполнить обратное распространение в механизме внимания было достаточно просто, потому что во время прямого прохода мы материализовали матрицу весов внимания A .

Проблема в том, что Flash Attention не сохраняет матрицу A в глобальной памяти, а значит мы не можем использовать ее для обратного распространения.

Поэтому давайте пересчитаем нужные блоки матрицы A . Пусть мы временно сохранили необходимые статистики m_i и l_i во время прямого прохода.

При обратном распространении мы снова берем блок строк Q_i , а также текущие блоки ключей и значений (K_t, V_t) . Затем пересчитываем блок оценок внимания:

$$S_{i,t} = \frac{Q_i K_t^T}{\sqrt{d}}$$

Если используется маска, то применяем ее также, как и при прямом распространении:

$$S_{i,t} \rightarrow \tilde{S}_{i,t}$$

После этого можно восстановить соответствующий блок матрицы A :

$$A_{i,t} = \frac{\exp(\tilde{S}_{i,t} - m_i)}{l_i}$$

Матрица A не хранится в памяти целиком, но отдельные её блоки мы восстанавливаем по мере необходимости.

Пусть со следующего слоя к механизму внимания пришла производная функции потерь по выходу $\frac{\partial L}{\partial O_i}$. Начнем с выражения

$$O_i = A_i V$$

Для одной строки O_i можно записать

$$O_{ic} = \sum_j A_{ij} V_{jc},$$

где c — индекс компоненты вектора значения. Попробуем найти производную функции потерь по элементу V_{jc} :

$$\frac{\partial L}{\partial V_{jc}} = \sum_i \frac{\partial L}{\partial O_{ic}} \frac{\partial O_{ic}}{\partial V_{jc}}$$

Поскольку

$$\frac{\partial O_{ic}}{\partial V_{jc}} = A_{ij},$$

то получаем, что

$$\frac{\partial L}{\partial V_{jc}} = \sum_i A_{ij} \frac{\partial L}{\partial O_{ic}}$$

или же в матричном виде:

$$\frac{\partial L}{\partial V_t} = A_{i,t}^T \frac{\partial L}{\partial O_i}$$

Один и тот же блок V_t участвует в вычислении разных блоков выхода O_i , поэтому производная по V_t накапливает вклады от разных блоков строк Q_i .

Теперь найдем производную по весам внимания $A_{i,t}$. Из того же выражения

$$O_{ic} = \sum_j A_{ij} V_{jc}$$

получаем

$$\frac{\partial O_{ic}}{\partial A_{ij}} = V_{jc}$$

Тогда:

$$\frac{\partial L}{\partial A_{ij}} = \sum_c \frac{\partial L}{\partial O_{ic}} V_{jc}$$

или же в матричном виде:

$$\frac{\partial L}{\partial A_{i,t}} = \frac{\partial L}{\partial O_i} V_t^T$$

Теперь нужно пройти через Softmax. Для одной строки производная функции потерь по элементу S_{ij} записывается следующим образом:

$$\frac{\partial L}{\partial S_{ij}} = A_{ij} \left(\frac{\partial L}{\partial A_{ij}} - \sum_k \frac{\partial L}{\partial A_{ik}} A_{ik} \right)$$

На первый взгляд кажется, что для вычисления суммы $\sum_k \frac{\partial L}{\partial A_{ik}} A_{ik}$ нам снова потребуется вся строка A_i . Но это выражение можно переписать через уже известные значения O_i и $\frac{\partial L}{\partial O_i}$:

$$\sum_j \frac{\partial L}{\partial A_{ij}} A_{ij} = \sum_j \left(\sum_c \frac{\partial L}{\partial O_{ic}} V_{jc} \right) A_{ij}$$

Поменяем порядок суммирования:

$$\sum_j \frac{\partial L}{\partial A_{ij}} A_{ij} = \sum_c \frac{\partial L}{\partial O_{ic}} \left(\underbrace{\sum_j A_{ij} V_{jc}}_{O_{ic}} \right) = \sum_c \frac{\partial L}{\partial O_{ic}} O_{ic}$$

Обозначим эту величину через D_i . Если O_i — это блок строк, то D_i не число, а вектор: по одному значению на каждую строку блока.

Теперь производную по блоку оценок внимания можно записать так:

$$\frac{\partial L}{\partial S_{i,t}} = A_{i,t} \odot \left(\frac{\partial L}{\partial A_{i,t}} - D_i \right)$$

Осталось найти производные по Q_i и K_t . Для элемента матрицы оценок внимания верно

$$S_{ab} = \frac{1}{\sqrt{d}} \sum_c (Q_i)_{ac} (K_t)_{bc}$$

Найдем производную по элементу $(Q_i)_{ac}$:

$$\frac{\partial S_{ab}}{\partial (Q_i)_{ac}} = \frac{(K_t)_{bc}}{\sqrt{d}}$$

Тогда

$$\frac{\partial L}{\partial (Q_i)_{ac}} = \sum_b \frac{\partial L}{\partial S_{ab}} \frac{(K_t)_{bc}}{\sqrt{d}}$$

или в матричном виде:

$$\frac{\partial L}{\partial Q_i} += \frac{1}{\sqrt{d}} \frac{\partial L}{\partial S_{i,t}} K_t$$

Аналогично для K_t :

$$\frac{\partial L}{\partial K_t} += \frac{1}{\sqrt{d}} \left(\frac{\partial L}{\partial S_{i,t}} \right)^T Q_i$$

На первый взгляд это может показаться странным: мы специально пересчитываем то, что уже считали во время прямого распространения.

Однако, на практике оказывается, что на запись в глобальную память и чтение из нее промежуточных результатов уходит больше времени, чем на пересчитывание такого блока. Такая идея называется **activation checkpointing**.

13 Программная модель Triton

CUDA предоставляет программисту достаточно низкоуровневую модель: потоки, блоки, работу с SMEM и RMEM. Это удобно, когда нужен полный контроль над поведением программы, но писать кернелы с производительностью на уровне cuBLAS крайне сложно.

При работе с нейросетями и, например, тестировании новых архитектур, хотелось бы работать на более высоком уровне.

Обычно мы не хотим вручную описывать поведение каждого потока. Вместо этого проще думать о блоках данных: фрагментах векторов, тайлах матриц и так далее.

Именно эту идею использует **тритон (Triton)** — Python-подобный **DSL (Domain Specific Language)**. Пользователь описывает вычисления на уровне блоков данных, а Triton затем компилирует этот код в низкоуровневое представление для выполнения на GPU.

13.1 Зачем нужен Triton?

На данный момент мы знаем только два инструмента для работы с нейронными сетями на GPU.

Первый вариант — вызывать готовые кернелы на **PyTorch**. Это одна крайность: удобно, но иногда это может быть малоэффективно, поскольку без слияния кернелов это может значительно замедлить работу модели и раздуть активации.

Второй вариант — писать собственные CUDA-кернелы. Это другая крайность: мы можем достичь максимальной эффективности, но это потребует огромных сил.

Тритон занимает промежуточное место: мы можем реализовывать собственные ядра малыми силами. В некотором смысле использование тритона напоминает закон Парето: 20% усилий дают 80% результата.

13.2 Ядро в Triton

Тритон-ядро аналогично ядру в CUDA. Это функция, которая будет выполняться на GPU. Такую функцию мы помечаем декоратором `@triton.jit`.

Синтаксис этой функции похож на Python, но она не выполняется интерпретатором Python построчно, а работает иначе. Тритон-ядро при запуске сначала проходит через компилятор. Код компилируется не заранее, а во время первого запуска.

Когда мы впервые вызываем Тритон-ядро, Тритон анализирует функцию, строит промежуточное представление программы, оптимизирует его и генерирует низкоуровневый код для GPU. Такой подход называется **just-in-time (JIT)** компиляцией.

После компиляции результат обычно кэшируется и переиспользуется во время следующих запусков. Поэтому первый запуск ядра может быть значительно медленнее последующих.

Запуск тритон ядра похож на запуск CUDA-ядра. Однако в отличие от CUDA, где мы задавали сетку блоков и потоков, в тритон мы задаем сетку из **program instance** — экземпляров программы.

13.3 Program Instance

В CUDA основная исполнительная единица — поток. Основная единица исполнения в Тритон — **экземпляр программы (program instance)**. Этот экземпляр программы отвечает не за один элемент, а за целый блок данных: фрагмент вектора, строку матрицы, тайл матрицы и так далее. Размер этого блока задается с помощью `BLOCK_SIZE`.

Чтобы понять, какой блок данных должен обработать текущий экземпляр программы, используется `pid` — идентификатор экземпляра программы.

Далее нужно построить глобальные индексы элементов, которые относятся к текущему блоку

$$\text{offset}_j = \text{pid} \cdot \text{BLOCK_SIZE} + \text{local_id}_j,$$

где `local_id` — это локальные индексы внутри блока. Их можно получить с помощью `tl.arange(0, BLOCK_SIZE)`.

Если `BLOCK_SIZE = 4`, то для разных значений `pid` получим

```
pid = 0 :   offset = (0  1  2  3) ,  
pid = 1 :   offset = (4  5  6  7) ,  
pid = 2 :   offset = (8  9 10 11) ,
```

...

Далее вычисления внутри кернела выполняются сразу для всего блока. Например, сложение векторов будет выглядеть следующим образом:

```
x = tl.load(x_ptr + offsets)  
y = tl.load(y_ptr + offsets)  
  
z = x + y  
  
tl.store(z_ptr + offsets, z)
```

Один экземпляр программы вычисляет сразу блок итогового вектора, в отличие от CUDA, где один поток вычислял бы один элемент. Экземпляр программы тритона — это более крупная логическая единица. При выполнении, компилятор сам отобразит операции внутри экземпляра программы на более низкоуровневое выполнение на GPU.

Иными словами, Тритон использует блочную векторизованную модель программирования. По форме она похожа на **single instructions multiple data (SIMD)** модель: одна программа применяется для блока данных. Но по итогу она исполняется на SIMT-подобной архитектуре GPU.

Как и в CUDA, в Тритон мы можем задать несколько осей при задании сетки. Тогда по каждой из осей нужно будет вычислить свой `pid`.

14 Профилирование

Оптимизация без явных измерений и оценок каких то метрик превращается в угадывание. Даже если некоторая оптимизация кажется очевидно, она может не ускорить программу.

Например, мы можем ускорить один кернел, но весь шаг обучения при этом остается таким же медленным, потому что узкое место на самом деле находится, например, в загрузке данных и синхронизации.

Поэтому любая оптимизация должна проверяться профилированием.

Профилирование можно разделить на несколько уровней: бечмарк, профилирование отдельных кернелов и профилирование всего шага обучения.

14.1 Бенчмарк

Бенчмарк — это измерение времени выполнения операций, по отдельности или вместе.

Ранее, в контексте стримов, мы говорили о том, что кернелы выполняются асинхронно. Это значит, что когда мы вызываем GPU-операцию, CPU обычно не ждет её завершения. Поэтому если мы в CPU-коде решим замерить выполнение программы на GPU, то мы получим не то что хотим, ведь таким образом мы замеряем именно время инициализации кернела на CPU.

CPU ставит операцию в очередь стрима. Поэтому, чтобы измерить время выполнения этой операции, необходимо до и после неё поставить CUDA-события. Далее мы можем оценить время между срабатыванием обоих событий.

Нельзя делать выводы по первому запуску. Первые итерации часто медленные: может происходить инициализация CUDA Runtime, JIT компиляции, аллокации буферов памяти, работа с кэшами и так далее.

Поэтому перед измерениями обычно делают несколько разогревочных запусков (**warmup-запусков**).

Оценивать время работы по одному замеру тоже ненадежно: время может колебаться из-за фоновых процессов, загрузки CPU, температуры GPU и так далее. Поэтому операцию запускают много раз и смотрят не одно число, а статистику: медиану и, например 25%, 75% перцентили (иногда еще и 90% перцентили).

Важно различать бенчмарк одной отдельной операции и полный замер всего шага обучения. Ускорение в первом случае может быть почти не заметно на всём масштабе: если операция занимала малую долю времени, то её оптимизация почти ничего не изменит.

Это явление описывает закон Амдала:

$$S = \frac{1}{(1 - p) + \frac{p}{s}},$$

где p — доля оптимизированной части, а s — ускорение этой части.

Например, если операция занимала 10% времени шага, а мы ускорили её в 10 раз (что на самом деле очень хороший результат), то общее ускорение будет равно:

$$S = \frac{1}{0.9 + 0.01} \approx 1.1,$$

то есть примерно 10%.

14.2 Профилирование кернелов

Бенчмарк показывает сколько времени заняло выполнение операции. Однако не объясняет, почему операция заняла именно столько времени. Для этого нужно смотреть, сколько времени занимают те или иные составляющие самого кернела.

Для профилирования кернелов используются такие инструменты как **Nsight Compute**. Такие профилировщики позволяют смотреть, как кернел использует вычислительные блоки, тензорные ядра и памяти (а **Nsight**

`Compute` также позволяет построить диаграмму обращений к разным уровням памяти).

Если кернел — `compute-bound`, то важно смотреть на достигнутое значение FLOP/s.

Если же ситуация иная, и кернел `memory-bound`, то полезно оценивать пропускную способность памяти.

14.3 Профилирование обучения

Отдельный кернел может быть быстрым, но весь шаг обучения всё равно может оставаться медленным. Поэтому также важно профилировать не только кернелы, но и всю итерацию обучения. Или по крайней мере основные шаги: прямое распространение, вычисление лосса, обратное распространение и шаг оптимизатора.

Также, важно учитывать работу всех стримов, поскольку задержка на одном стриме может блокировать все остальные стримы.

Для такого анализа используются таймлайн-профилировщики: `torch.profiler` и `Nsight Systems`. Они показывают не только отдельные операции, но и их расположение во времени: когда работает хост, когда работает девайс, где на таймлайне девайса находится запускаемый на таймлайне хоста кернел, какие стримы используются, где есть пустые промежутки (когда девайс простаивает) и так далее.

Для распределенного обучения также важно смотреть на то, как хорошо стрим коммуникаций перекрывается стримом вычислений, но об этом мы поговорим в следующем разделе.

Часть IV

Введение в распределенное обучение

При обучении на одном девайсе мы ограничены аппаратными возможностями одного GPU.

Во-первых, мы ограничены по памяти. При обучении нам необходимо хранить параметры модели, градиенты, состояния оптимизатора и активации. Это очень сильно ограничивает размер моделей.

Во-вторых, мы ограничены по вычислительной мощности. Допустим, у нас нет проблемы памяти и мы собрались обучать LLM. Для предобучения модели нам потребуется $4.8 \cdot 10^{26}$ FLOPs. При обучении на одной H100 на пике производительности в точности `float16` это займёт ~ 7600 лет. К сожалению, мы слишком непоседливы, чтобы ждать столько времени.

Чтобы решить эти проблемы, и обучать по настоящему большие модели за адекватное время, необходимо использовать тысячи видеокарт.

Однако, несколько GPU сами по себе не собираются в одну большую видеокарту. У каждого девайса — своя память и свои вычислительные блоки. Если тензор находится в памяти одного девайса, то другой девайс не может читать его прямым образом как из своей памяти. Нам необходимо передавать данные между GPU. Передача данных между GPU называется **коммуникациями**.

15 Связь нескольких GPU

Некоторые коммуникации могут быть быстрыми, а некоторые — медленными. При этом, их необходимо выстроить таким образом, чтобы нагрузка на GPU была распределена равномерно и не было простоя в вычислениях из-за ожидания необходимых данных.

Чтобы понять, почему коммуникации могут отличаться по скорости, необходимо сначала разобраться, как GPU физически связаны между собой.

15.1 Устройство GPU-ноды

На первом уровне иерархии кластера GPU находится нода — небольшой сервер, внутри которого установлено несколько GPU, обычно 8. Такая нода содержит CPU (обычно даже несколько CPU-сокетов), оперативную память, GPU и сетевые карты.

CPU управляет запуском вычислений, подготавливает данные, вызывает кернелы и коммуникационные операции. Оперативная память CPU — это хост-память. Она также может использоваться, например, для выгрузки

тех данных, которые не способны в текущий момент времени поместиться в GPU.

Сетевая карта, NIC, соединяет ноду с остальным кластером. Для распределенного обучения важно, где находится NIC относительно GPU.

Важна также NUMA-локальность. Если в ноде несколько CPU-сокетов, то память и девайсы могут быть ближе к одному сокету и дальше от другого. Поэтому два девайса одной ноды не всегда эквивалентны с точки зрения доступа к CPU памяти и NIC.

15.2 Внутринодовая связь GPU

Внутринодовая связь — это связь между GPU внутри одного сервера. Обычно она быстрее и дешевле, чем связь между разными нодами.

Самый базовый способ соединения GPU внутри ноды — PCIe шина. Через него GPU может обмениваться данными с CPU, памятью хоста, сетевой картой и другим GPU. Это универсальное соединение, но для интенсивного обмена между GPU его пропускной способности может быть недостаточно.

Для более быстрого обмена между GPU используется соединение NVLink. Он дает более высокую пропускную способность и лучше подходит для частых коммуникаций между девайсами. NVLink подключается к GPU отдельно от PCIe и соединяет GPU только попарно.

Возникает проблема топологии: одна GPU может быть соединена с несколькими соседями, но не обязательно с несколькими GPU в ноды. Тогда передача между некоторыми памятью GPU может идти через промежуточные устройства.

На помощь приходит NVSwitch, его можно рассматривать как коммутатор для NVLink. Мы можем подключить GPU к NVSwitch через NVLink и обмениваться между всеми девайсами внутри ноды.

Кроме физического соединения важно использовать P2P (peer-to-peer) связь. Без использования P2P, данные будут проходить через CPU RAM, что очень дорого:

$$\text{GPU}_0 \rightarrow \text{CPU} \rightarrow \text{GPU}_1,$$

а при использовании P2P:

$$\text{GPU}_0 \rightarrow \text{GPU}_1$$

15.3 Межнодовая связь

Обычно, ноды объединяют по 8 GPU. Этого всё еще мало, нам нужен кластер из тысяч GPU. Чтобы этого достичь, необходимо для начала наладить межнодовую связь.

Упрощенно межнодовую связь можно представить так:

$$\text{GPU}_0 \rightarrow \text{CPU}_0 \text{ memory} \rightarrow \text{NIC}_0 \rightarrow \text{Сеть} \rightarrow \text{NIC}_1 \rightarrow \text{CPU}_1 \text{ memory} \rightarrow \text{GPU}_1$$

Как мы уже обсуждали, передавать данные через CPU память очень дорого. Поэтому, чтобы избежать этого, будем передавать данные напрямую от девайса к NIC, используя **удаленный прямой доступ к памяти (remote direct memory access, RDMA)**. Тогда схема будет выглядеть короче:

$$\text{GPU}_0 \rightarrow \text{NIC}_0 \rightarrow \text{Сеть} \rightarrow \text{NIC}_1 \rightarrow \text{GPU}_1$$

Сетевые интерфейсы NIC — это аппаратные компоненты ноды, которые являются выходом в сеть. При перемещении данных по сети при обучении LLM обычно используют Infiniband.

15.4 Топология кластера

Объединение нод в одну сеть, **кластер**, можно рассматривать как граф. Вершины — это ноды, а ребра — сетевые соединения между ними.

Если рассматривать более подробно, то можно заметить, что каждая нода — тоже граф, только на этот раз вершины — GPU.

Расстояние между двумя GPU внутри ноды значительно меньше, чем расстояние между двумя GPU в разных нодах. Межнодовая связь значительно сложнее, а значит дороже, чем внутринодовая.

Но даже между нодами расстояние может быть разное. Некоторые ноды могут физически находиться на одном стеллаже и быть подключены к одному сетевому коммутатору. Другие ноды могут обмениваться данными через несколько слоёв сетевой инфраструктуры.

В больших кластерах важна топология сети. Например:

- fat-tree,
- dragonfly,
- torus

Мы не будем их разбирать, однако важно понимать, что топология определяет, какие коммуникации будут дешевыми, а какие — дорогими, потому что это напрямую влияет на распределенное обучение.

Если стратегия параллелизма требует частого обмена большими тензорами между удаленными нодами, то обучение может упереться в стоимость коммуникаций.

15.5 Стоимость коммуникаций

Основные метрики, которые оценивают стоимость коммуникаций — это задержка передачи α и пропускная способность B .

Простейшая модель времени коммуникации имеет вид:

$$T_{\text{comm}}(n) \approx \alpha + \frac{n}{B},$$

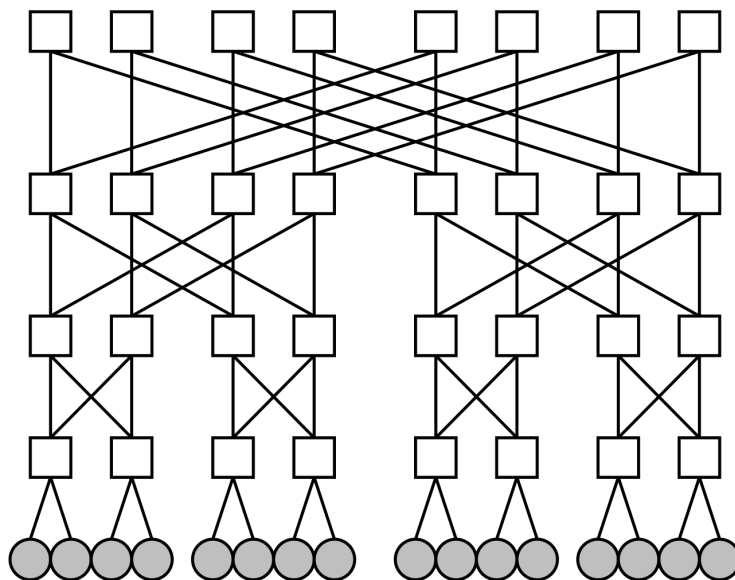


Рис. 21: Схема fat-tree топологии.

где n — размер передаваемых данных. Из этой формулы видно, что при малых сообщениях в стоимости коммуникации доминирует задержка, а при больших — скорость пропускной способности. Запускать много мелких коммуникаций — плохо, лучше объединять их в большие пакеты.

16 Коммуникации

В распределенном обучении говорят не просто про девайсы, а про ранги. Ранг — это отдельный участник распределенного обучения. Часто (но не всегда) один ранг соответствует одному GPU.

Существуют глобальные ранги и локальные ранги. Глобальный ранг — это ранг во всем кластере. Локальный ранг — это ранг внутри ноды.

Каждый ранг хранит свои локальные тензоры и выполняет свою часть вычислений, в зависимости от стратегии параллелизма. Паттерны, которые устанавливают, какие данные должны быть переданы между рангами, называются коллективными коммуникациями.

16.1 Point-to-point коммуникации

Перед рассмотрением коллективных коммуникаций, разберем простейшую схему обмена данными — между двумя GPU.

Такие операции называются point-to-point операциями: один ранг отправляет буфер данных с помощью операции `send`, а второй принимает его с помощью операции `recv`.

Point-to-point операции просты по смыслу, но требуют аккуратной синхронизации. Если один ранг ждет данные, а другой их не отправил, то этот ранг будет ждать их вечно. Такая проблема называется deadlock.

Поэтому обычно point-to-point операции используют внутри коллективных коммуникационных операций, где порядок действий заранее фиксирован (хотя, такие операции также могут использоваться в некоторых типах параллелизма).

16.2 Коллективные коммуникации

В коллективных коммуникациях участвует уже не пара рангов, а целая группа (называемая процесс-группой). Эти операции — шаблоны. Но именно такие шаблоны часто и повторяются в различных стратегиях распределенного обучения.

Рассмотрим некоторые из этих операций.

AllReduce AllReduce — операция, которая выполняет редукцию данных между рангами в группе.

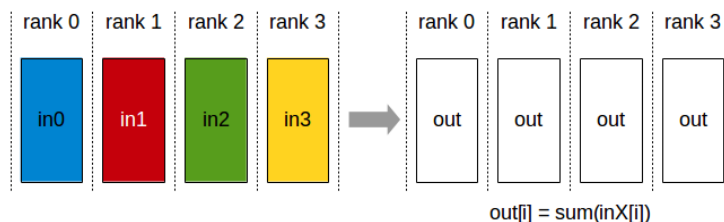


Рис. 22: Схема AllReduce.

Каждый ранг группы получает результат редукции.

Broadcast Broadcast — операция, которая копирует тензор из корневого ранга на остальные ранги.

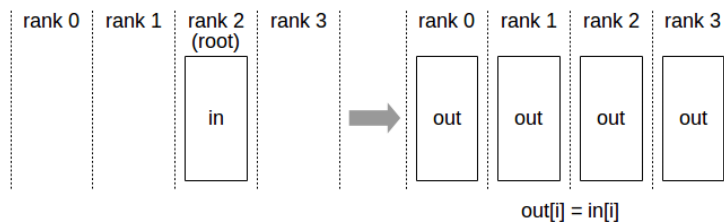


Рис. 23: Схема Broadcast.

AllGather AllGather собирает данные с каждого ранга и раздает полный результат каждому рангу.

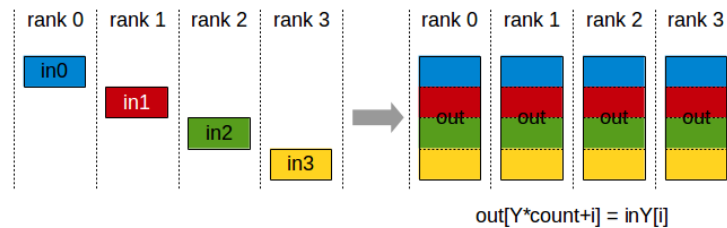


Рис. 24: Схема AllGather.

В итоге каждый ранг получает тензор, собранный из отдельных частей со всех рангов.

Reduce Операция Reduce выполняет ту же операцию, что и AllReduce, но итоговый результат помещается только на один ранг.

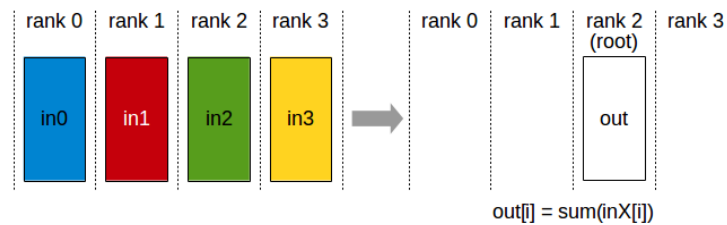


Рис. 25: Схема Reduce.

ReduceScatter ReduceScatter объединяет редукцию и разбиение результата.

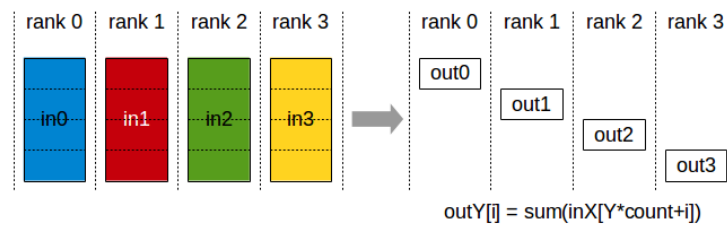


Рис. 26: Схема ReduceScatter.

Сначала локальные тензоры редуцируются, например, суммируются

$$z = \sum_{i=0}^{N-1} x_i$$

после чего z разбивается на несколько частей

$$z = [z_0, z_1, \dots, z_{N-1}]$$

и каждый ранг i получает только свою часть $y_i = z_i$.

AlltoAll AlltoAll — это операция, при которой каждый ранг отправляет разные части своего тензора всем остальным рангам в группе.

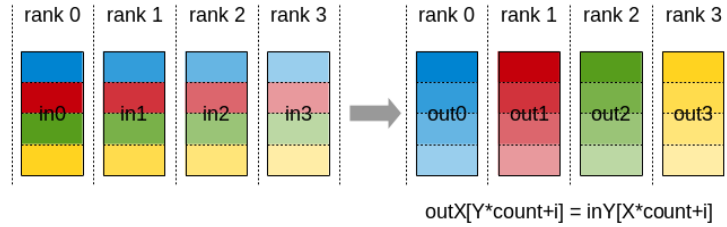


Рис. 27: Схема AlltoAll.

Пусть на ранге i хранится тензор, разбитый на N частей:

$$x_i = [x_{i,0}, x_{i,1}, \dots, x_{i,N-1}]$$

Часть $x_{i,j}$ предназначена для ранга j . После AlltoAll ранг j получает тензор, состоящий из следующих частей:

$$y_j = [x_{0,j}, x_{1,j}, \dots, x_{N-1,j}]$$

AlltoAll — одна из самых тяжелых коллективных операций, потому что количество обменов быстро растет с числом рангов. Схему этой коллективной операции можно представить в виде полного графа, где каждая вершина связана с каждой другой.

16.3 Объединение коллективных коммуникаций

Некоторые коллективные коммуникации, выполняемые подряд, можно объединить в одну. Например, последовательное выполнение операций Reduce и Broadcast можно заменить на AllReduce.

Причем такая замена будет выгодна, поскольку AllReduce выполняется быстрее, чем последовательное выполнение сначала Reduce и следом Broadcast.

Другое полезное объединение коммуникаций:

$$\text{AllReduce} = \text{ReduceScatter} + \text{AllGather}$$

`ReduceScatter` считает редуцированный результат, но оставляет на каждом ранге только его часть. Затем `AllGather` собирает эти части обратно и раздает полный результат всем рангам.

17 NCCL

Всё, что мы рассмотрели в предыдущем разделе, является лишь теоретическими моделями операций коммуникации. Для их эффективного выполнения необходим инструмент, который будет способен настроить общение между GPU по заданному шаблону, учитывая топологию кластера, скорость внутринодовой и межнодовой связи.

Для этого используется библиотека коммуникаций **NVidia Collective Communication Library (NCCL)**. Она используется как бэкенд в системах распределенного обучения, реализуя рассмотренные ранее коллективные коммуникации (но не ограничиваясь только на них).

Одна и та же операция, например `AllReduce`, может выполняться разными способами даже в одном кластере, и в зависимости от размера пакета, NCCL будет минимизировать модель стоимости коммуникаций.

Так, для выполнения такой операции, NCCL берет на себя задачу выбора протокола передачи данных, определяет путь передачи от девайса к девайсу, разбивает сообщение на части, запускает кернелы коммуникации и синхронизирует передачу данных каждым шагом, чтобы не допустить ошибок или дедлоков.

NCCL — высокоуровневая библиотека коммуникаций, API которой мы можем использовать напрямую или, например, через PyTorch в виде модуля `torch.distributed`.

17.1 Модель исполнения

Основные сущности в NCCL — это ранги, стримы и коммуникатор. Что такое ранг мы уже знаем.

Коммуникатор — это объект, который помогает процесс-группе рангов выполнять коммуникационные операции друг с другом. Если `AllReduce` вызывается на коммуникаторе 8 рангов, то в операции участвуют только эти ранги.

Это может быть удобно, например, для того, чтобы явным образом отделить использование внутринодовых и межнодовых коммуникаций, что особенно важно при построении сетки стратегий параллелизма.

Стрим — это обычный CUDA стрим, который задает порядок исполнения операций на GPU. Операции коммуникации — это тоже кернелы, которые запускаются на GPU и требуют как ресурсы памяти, так и SM.

Мы запускаем коммуникации в отдельном стриме для того, чтобы выполнять их асинхронно и скрыть задержку коммуникаций под стримом вычислений. Если бы мы запускали все операции в одном стриме, то они бы

выполнялись последовательно, что привело бы к простоя или недоиспользованию GPU.

За счёт CUDA-событий мы можем синхронизировать работу стрима вычислений и стрима коммуникаций: очевидно, что нам не стоит начинать вычисление слоя до того, как мы полностью соберем веса этого слоя.

17.2 Коммуникационные каналы

NCCL не обязан передвигать большой тензор как один монолитный блок. Сообщения разбиваются на части, которые могут передаваться параллельно. Эти потоки передачи данных называются **коммуникационными каналами**.

Важно: коммуникационный канал — это логическая единица, а не физический кабель. Через один NVLink может проходить несколько коммуникационных каналов, также и один коммуникационный канал может передавать данные по нескольким физическим кабелям.

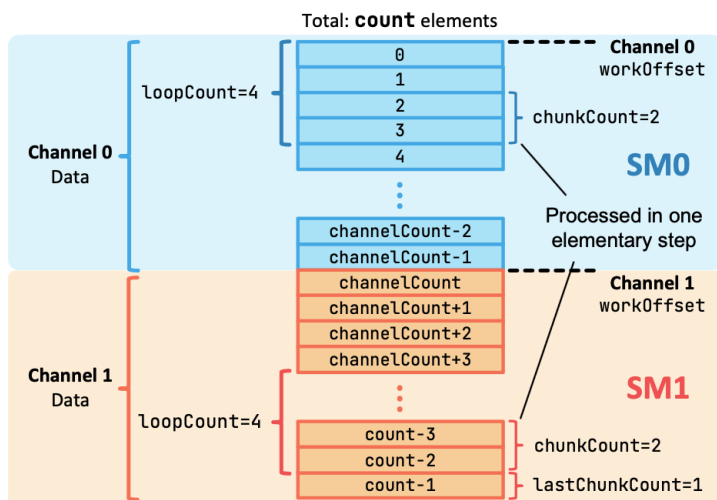


Рис. 28: Визуализация стратегии разделения данных NCCL между каналами.

Для больших сообщений большое число каналов может насытить пропускную способность. Но каждый канал требует вычислительных ресурсов GPU. Увеличивая число каналов, мы можем уменьшить время операций коллективных коммуникаций, но взамен замедлим выполнение кернелов на вычислительном стриме.

17.3 Протоколы передачи данных

Чтобы увеличить пропускную способность при передаче больших тензоров, мы можем передавать данные крупными блоками. Для обеспечения корректного порядка и видимости данных необходимо использовать барьеры памяти (memory fences).

Такой протокол передачи данных называется **Simple**. Он увеличивает утилизацию пропускной способности, но взамен также увеличивает задержку выполнения операции из-за барьеров-памяти.

Вспомним модель стоимости коммуникации:

$$T = \alpha + \frac{n}{B}$$

При большом значении n , то есть размера пакета, это выгодный протокол. При малом — нет.

Для работы с сообщениями малого размера используется протокол **LL** (**low latency**). Вместо использования барьеров памяти, LL полагается на синхронизации на основе передаваемых флагов. Они показывают, что данные валидны и получатель может продолжать работу. На каждые 4 байта данных передается 4 байта флага.

Simple — это одна крайность, полезная при пересылке больших пакетов, но увеличивающая задержку выполнения операции. LL — другая крайность, полезная при пересылке малых пакетов, но снижающая полезное использование пропускной способности по сути в два раза.

Компромиссный вариант — протокол **LL128**. Как и LL, этот протокол использует синхронизацию на основе флагов, однако на этот раз передает не 8, а 128 байт (отсюда и название). Из этих 128 байт, 8 байт занимает флаг, а 120 — полезная нагрузка.

Однако LL128 не всегда возможно использовать из-за высоких требований к оборудованию.

NCCL выбирает протоколы за нас, однако полезно понимать ограничения своей системы, чтобы оценивать какие операции будут занимать много времени и оптимизировать их до использования коммуникаций.

17.4 Алгоритмы коммуникаций

Одна и та же коллективная операция может быть выполнена разными алгоритмами. Порядок обмена данными может быть разным. Рассмотрим основные алгоритмы коммуникаций.

Ring При использовании алгоритма **Ring** ранги образуют кольцо. Например, для четырех девайсов такое кольцо будет выглядеть следующим образом:

$$\text{GPU}_0 \rightarrow \text{GPU}_1 \rightarrow \text{GPU}_2 \rightarrow \text{GPU}_3 \rightarrow \text{GPU}_0.$$

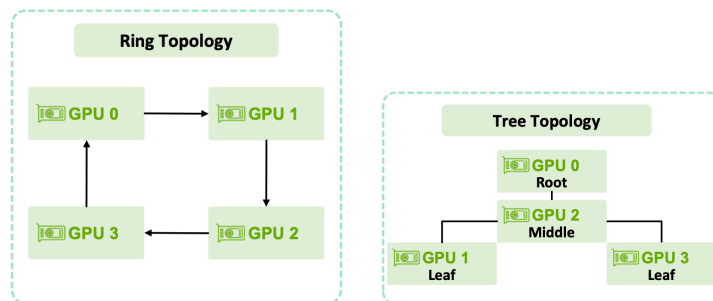


Рис. 29: Схемы алгоритма коммуникации по кольцу (слева) и коммуникации по дереву (справа).

Каждый ранг передает данные следующему рангу и получает данные от предыдущего. Большой тензор разбивается на части и эти части двигаются по кольцу, пока не совершат один полный обход.

Кольцевой алгоритм позволяет очень удачно утилизировать пропускную способность и при этом не раздувать память, которая выделяется под буферы.

Однако, количество шагов растет с увеличением количества рангов. Поэтому при большом количестве рангов возможна высокая задержка коммуникации.

Tree В древовидном алгоритме ранги организуются в дерево. Данные сначала поднимаются от листьев к корню, где выполняется редукция, а следом результат распространяется обратно вниз по дереву.

Такой алгоритм может иметь меньше последовательных шагов, чем кольцо, а глубина дерева растет как $O(\log N)$, поэтому в операциях, где участвует большое количество рангов, дерево может быть эффективнее, чем кольцо.

Иерархический алгоритм В реальных кластерах топология обычно иерархическая. GPU внутри одной ноды имеют большую скорость передачи данных, чем GPU в разных нодах. Поэтому стоит учитывать это при выборе алгоритмов коммуникаций.

Например, внутри одной ноды мы можем построить два кольца по четыре девайса. А коммуникации между нодами можем выстроить как дерево, что позволит значительно снизить объем междоузовой передачи данных.

18 Параллелизм данных (DP)

18.1 Распределенный параллелизм данных (DDP)

Самый простой способ организации параллелизма вычислений на GPU — это разбить батч данных между GPU. Такая стратегия называется **параллелизмом данных**.

Каждая GPU получает свою часть батча и выполняет прямое и обратное распространение независимо друг от друга. А параметры модели копируются (реплицируются) между девайсами.

На каждом ранге выполняется вычисление локального лосса по своим объектам:

$$\mathcal{L}_i^{(t)} = \mathcal{L}_{w_t}(B_i)$$

Каждый девайс выполняет обратное распространение в конкретном слое модели на основе её входа и на основе своего лосса. Это значит, что градиенты на каждом ранге будут разные. А веса — одни.

$$g_i^{(t)} = \nabla_{w_t} \mathcal{L}_i^{(t)}$$

Чтобы все копии модели оставались одинаковыми, нужно синхронизировать градиенты. Для этого применяется **AllReduce** по градиентам:

$$g^{(t)} = \frac{1}{N} \sum_{j=1}^N g_j^{(t)}$$

После этого на каждом ранге будет лежать одинаковый усредненный градиент, поэтому мы можем на каждом девайсе независимо выполнить шаг оптимизатора.

Как видно, главная коммуникация в DDP — это **AllReduce**. Размер передаваемых данных примерно равен размеру градиентов модели. При обучении больших моделей это может занимать много времени.

Поэтому на практике DDP не ждет завершения всего обратного распространения, чтобы начать коммуникацию. Градиенты ведь считаются последовательно по слоям. Таким образом, мы можем вычислить градиенты одного трансформерного слоя и выполнять **AllReduce** над ними, а в это время асинхронно начать вычислять градиенты следующего трансформерного слоя. Таким образом мы можем прятать коммуникации под вычислениями.

Это называется **bucketing**. Мы не ждём вычисления всех градиентов. Как только мы вычисляем достаточно градиентов, что их размер превышает заданный нами размер бакета, мы начинаем коммуникацию.

Размер бакета не обязательно равен размеру вектора градиентов одного трансформерного слоя, как было в примере. Это настраиваемый гиперпараметр, поскольку большие бакеты приводят к более высокой пропускной способности, но увеличивают задержку из-за более поздней коммуникации.

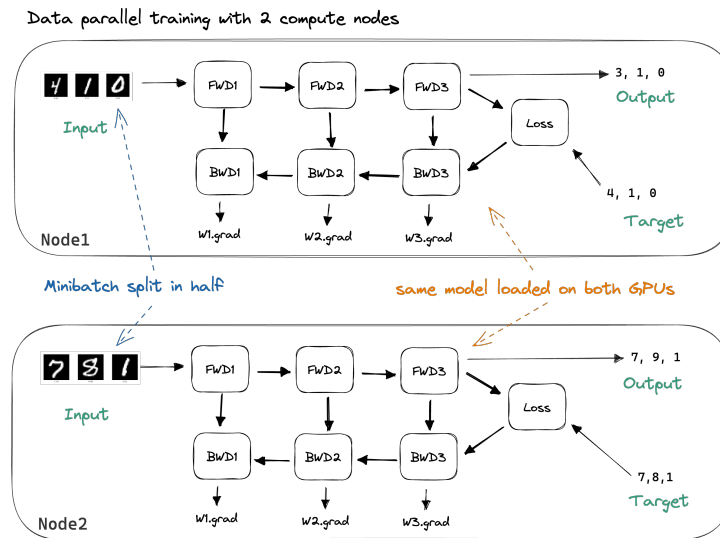


Рис. 30: Схема распределенного параллелизма данных между двумя нодами.

Малые пакеты наоборот, снижают пропускную способность, зато коммуникации стартуют раньше.

Для моделей с большими слоями полезно использовать большие пакеты, а для малых — меньшие.

18.2 ZeRO

DDP хорошо масштабирует вычисления, но плохо масштабирует память. На каждом ранге нам приходится хранить активации и реплики параметров модели, градиентов и состояний оптимизатора.

При использовании оптимизаторов, таких как, например, Adam или AdamW, для одного параметра в fp32 необходимо хранить 16 байт данных: 4 байта на сам параметра, 4 байта на его градиента, 8 байта на состояние оптимизатора. Это сильно раздувает потребление памяти. Таким образом, хоть мы и выигрываем от увеличения вычислительных мощностей, мы почти не получаем выигрыша от увеличения памяти GPU, поскольку храним реплики.

Эту проблему решает **оптимизация нулевой избыточности (ZeRO, Zero Redudancy Optimizer)**. Основная идея заключается в том, чтобы не реплицировать градиенты, состояния оптимизатора и параметры модели между всеми рангами, а шардировать их (разделять на равные части) между всеми рангами и собирать по мере необходимости.

Существует три стадии ZeRO. Рассмотрим первые две из них.

ZeRO-1 В этой стадии мы шардируем только состояния оптимизатора. Параметры и градиенты всё ещё хранятся на своих рангах.

После обратного распространения градиенты синхронизируются как в DDP с помощью **AllReduce**. Затем каждый ранг обновляет только свою часть состояния оптимизатора и соответствующую ей часть параметров. После этого обновленные параметры нужно снова сделать доступными всем рангам с помощью **AllGather**.

ZeRO-2 В ZeRO-2 шардируются и состояния оптимизатора, и градиенты. Теперь после обратного распространения слоя нам не нужно хранить полную копию всех градиентов на каждом ранге

$$g_i^{(t)} = \nabla_{w_i} \mathcal{L}_i^{(t)}$$

Вместо этого мы шардируем градиенты между рангами

$$g_i^{(t)} = [g_{i1}^{(t)}, g_{i2}^{(t)}, \dots, g_{iN}^{(t)}]$$

Ранг i хранит только $g_{ii}^{(t)}$. Вместо **AllReduce** нужно использовать **Reduce-Scatter** после вычисления обратного распространения каждого слоя. Так каждый ранг i получит все градиенты $\{g_{ji}^{(t)}\}_{j=1}^N$, соответствующие его шардированной части и выполнит редукцию только по ней:

$$g_i^{(t)} = \frac{1}{N} \sum_{j=1}^N g_{ij}^{(t)}$$

18.3 FSDP

ZeRO-3 требует особого внимания. Эта стадия также называется **полностью шардированным параллелизмом данных (fully sharded data parallel, FSDP)**. Как понятно из названия, FSDP шардирует всё: и градиенты, и состояния оптимизатора, и параметры.

Если мы используем N рангов, то в идеальном случае можем уменьшить потребление памяти в N раз: в N раз уменьшается объем активаций на девайсе за счёт уменьшения количества объектов в батче и в N раз уменьшается объем хранимых параметров, градиентов и состояний оптимизатора за счёт шардирования.

Это самая агрессивная стадия шардирования и она требует большого количества постоянных коммуникаций всех рангов в группе DP-параллелизма.

Если параметры зашардированы, то перед вычислением прямого распространения слоя ранг должен собрать все шарды со всех остальных рангов с помощью **AllGather**. Чтобы не раздувать потребление памяти репликами параметров, после завершения вычисления слоя мы освобождаем память тех параметров, которые не являются шардом на текущем ранге.

В обратном распространении нам также необходимы все параметры слоя, поэтому мы также вынуждены сначала собрать их с помощью **AllGather**.

После вычисления обратного распространения, вычисленные градиенты нужно разложить по рангам с помощью **ReduceScatter**, как мы делали в ZeRO-2. После вычисления слоя, нужно освободить память шардов, не относящихся к текущему рангу.

Выполнение **AllGather** коммуникации для сбора следующего слоя мы можем прятать под вычисления текущего слоя. Аналогично и с **ReduceScatter**.

Скрытие стрима коммуникаций под стримом вычислений — основная оптимизация FSDP. Степень скрытия коммуникаций оценивает метрика communication-computation overlap:

$$\text{Overlap}_{\text{comm-comp}} = \frac{T_{\text{comp}}}{T_{\text{comp}} + T_{\text{idle}}},$$

где T_{idle} означает время, которое ранг находится в простое из-за ожидания завершения коммуникации.

Дальнейшие оптимизации FSDP уменьшают время коммуникации за счёт снижения накладных расходов на реаллокацию памяти, изменения методов шардирования и использования двойной буферизации. Это — тонкие инженерные детали, которые мы не будем рассматривать.

19 Тензорный параллелизм (TP)

FSDP уменьшает потребление памяти и ускоряет вычисления, разделяя батч на части.

Однако, мы не можем ускорить таким образом вычисление слоя, обрабатывающего один объект в батче. Мы всё ещё можем упереться в хранение активаций.

Попробуем параллелизовать вычисления внутри одного слоя, разделяя вычисления самих тензоров между разными рангами. Такой метод параллелизма называется **тензорным параллелизмом (TP)**

19.1 Параллелизм по столбцам и строкам

Тензорный параллелизм использует математические свойства матричного умножения. Во-первых:

$$A \cdot B = A \cdot \begin{bmatrix} B_1 & B_2 & \dots \end{bmatrix} = \begin{bmatrix} A \cdot B_1 & A \cdot B_2 & \dots \end{bmatrix}$$

И во-вторых:

$$A \cdot B = \begin{bmatrix} A_1 & A_2 & \dots \end{bmatrix} \cdot \begin{bmatrix} B_1 \\ B_2 \\ \dots \end{bmatrix} = \sum_{i=1} A_i B_i$$

Первый способ позволяет организовать **параллелизм по столбцам**: каждый ранг держит полную матрицу A и несколько столбцов матрицы B . Этот ранг вычисляет $A \cdot B_i$.

Например, при вычислении линейного слоя, это позволяет разделять веса между рангами

$$Y_i = XW_i,$$

увеличив скорость вычисления этого слоя. Однако параллелизм по столбцам не уменьшает объем хранимых активаций в памяти.

Попробуем воспользоваться вторым свойством матричного умножения, которое мы рассматривали ранее: разрежем матрицу X по столбцам, а матрицу W — по строкам.

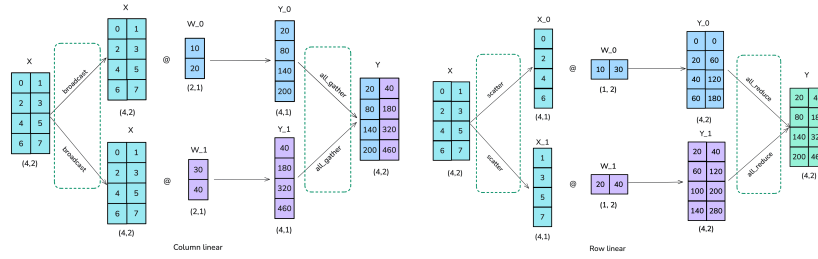


Рис. 31: Схема параллелизма по столбцам (слева) и по строкам (справа). Взято из Ultra-Scale Playbook

Этот метод называется **параллелизмом по строкам**. Каждый девайс хранит лишь часть активаций и часть весов. Однако взамен мы платим очень долгими и тяжелыми **AllReduce** коммуникациями после каждого слоя, необходимыми для вычисления суммы

$$Y = \sum_{i=1} X_i W_i$$

При вычислении FFN-блока мы можем избежать лишних коммуникаций, вычисляя первый линейный слой с помощью параллелизма по столбцам, а следующий — с помощью параллелизма по строкам.

Поскольку внутренняя размерность FFN-блока обычно больше, чем входная и выходная, то такой подход позволяет также значительно выиграть в хранении активаций.

Теперь рассмотрим блок многоглавого внимания. Головы внимания вычисляют результат независимо друг от друга. А это значит, мы можем попробовать вычислять их параллельно на разных девайсах.

Рассмотрим примере двух голов внимания и двух девайсов. Пусть на одном девайсе — одна тройка матриц Q_1 , K_1 и V_1 , а на втором девайсе другая тройка Q_2 , K_2 и V_2 .

Вычисление выхода механизма внимания одной головы тогда будет происходить также, как при использовании параллелизма по столбцам. Важно: при этом мы никак не ломаем кернел **FlashAttention**, на каждом девайсе мы просто запускаем кернел над разными данными.

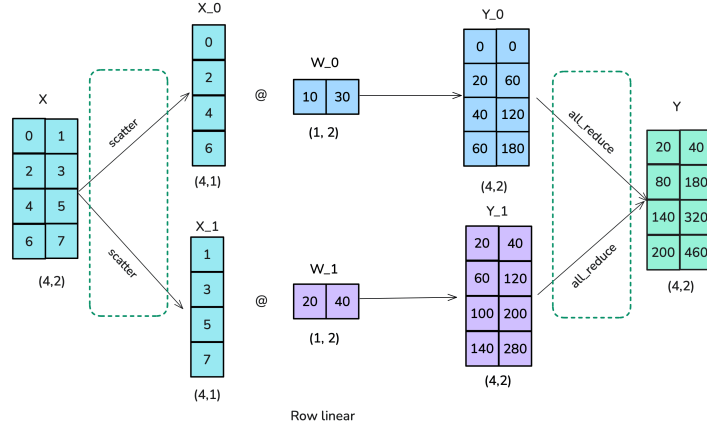


Рис. 32: Схема вычисления FFN-блока с использованием сначала параллелизма по столбцам, а следом — параллелизма по строкам. Взято из Ultra-Scale Playbook.

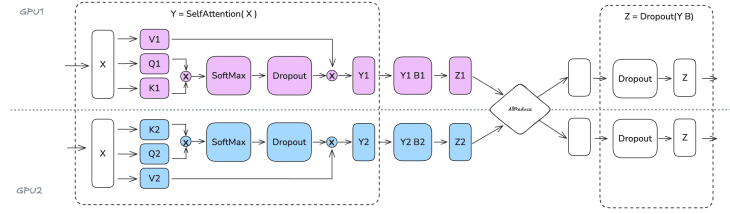


Рис. 33: Применение тензорного параллелизма при вычислении многоглавого внимания. Взято из Ultra-Scale Playbook.

После вычисления результата голов O_1 и O_2 , их необходимо сконкатенировать и спроецировать с помощью матрицы W_O :

$$O = [O_1 \quad O_2] W_O$$

Разобьём матрицу W_O по строкам. Пусть W_{O_1} лежит на первом девайсе, а W_{O_2} — на втором. Тогда

$$O = [O_1 \quad O_2] \cdot \begin{bmatrix} W_{O_1} \\ W_{O_2} \end{bmatrix}$$

Это значит, что каждый ранг сначала первый ранг может вычислить $O_1 W_{O_1}$, а второй ранг — $O_2 W_{O_2}$, после чего каждый из рангов может получить результат

$$O = \sum_{i=1}^2 O_i W_{O_i}$$

с помощью AllReduce.

19.2 Параллелизм последовательностей (SP)

Мы применили параллелизм к FFN-блоку и к механизму внимания. Однако, еще остались Dropout и LayerNorm слои. Поскольку преобразования токенов внутри этих операций применяются независимо друг от друга, мы можем разделить последовательность между различными девайсами.

Такой способ параллелизма называется **параллелизмом последовательностей (SP)**. Параллелизм последовательностей также позволяет сократить объем хранимых активаций на одном ранге.

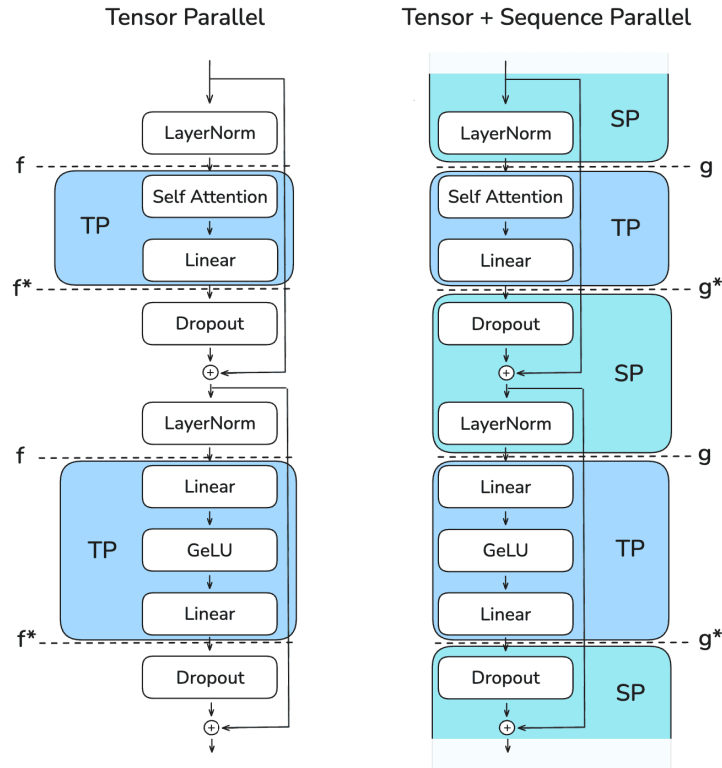


Рис. 34: TP+SP параллелизм. Взято из Ultra-Scale Playbook.

Параллелизм последовательностей идет в паре с тензорным параллелизмом. Чтобы не было простоя GPU, важно чтобы

$$\text{rank}(\text{TP}) = \text{rank}(\text{SP})$$

После вычисления, например, нормализации, необходимо вычислять механизм внимания. Для вычисления механизма внимания с тензорным параллелизмом, необходимо с помощью **AllGather** собрать всю последовательность.

Тензорный параллелизм в паре с параллелизмом последовательностей — мощный инструмент параллелизма вычислений, который позволяет уменьшить объем хранимых активаций и при этом увеличить скорость вычислений слоя. Но из-за тяжелых **AllReduce** и **AllGather**-коммуникаций, которые не получается скрыть под стримом вычислений, необходимо держать коммуникации строго внутри одной ноды.

20 Контекстный параллелизм (СП)

При использовании тензорного параллелизма в механизме внимания нам необходимо на каждом девайсе собирать всю последовательность и хранить активации по ней. Это может привести к нехватки памяти при обучении больших моделей на длинном контексте.

Для решения этой проблемы применяется **контекстный параллелизм (СП)**. Также как и SP, контекстный параллелизм делит последовательность между рангами, при этом сохраняя возможность вычислять внимание по всему контексту.

20.1 Кольцевое внимание

Для реализации вычисления внимания по всему контексту при разделении последовательности между девайсами используется механизм **кольцевого внимания**.

Каждый ранг хранит свой блок запросов Q_i и постепенно получает блоки K и V от других рангов по кольцу GPU.

Пусть есть N рангов. Разобьём последовательность на N блоков:

$$X = [X_1, X_2, \dots, X_N]$$

На ранге i хранятся локальные Q_i , K_i , V_i . Чтобы вычислить O_i , рангу i нужно применить внимание между Q_i и всеми блоками $\{(K_i, V_i)\}_{i=1}^N$.

Вместо того, чтобы сразу собирать все K и V на каждом ранге, блоки передаются по кольцу. На первом этапе так:

$$\text{Rank}_1 \xrightarrow{(K_1, V_1)} \text{Rank}_2 \xrightarrow{(K_2, V_2)} \dots \xrightarrow{(K_{N-1}, V_{N-1})} \text{Rank}_N \xrightarrow{(K_N, V_N)} \text{Rank}_1$$

На следующем этапе:

$$\text{Rank}_1 \xrightarrow{(K_N, V_N)} \text{Rank}_2 \xrightarrow{(K_1, V_1)} \dots \xrightarrow{(K_{N-2}, V_{N-2})} \text{Rank}_N \xrightarrow{(K_{N-1}, V_{N-1})} \text{Rank}_1$$

И так $N - 1$ раз.

Однако, в этом случае на одном ранге мы вычисляем лишь часть матрицы S . Пусть, например, на одном ранге на текущем шаге лежат матрицы Q_i , K_j и V_j . Тогда мы сможем вычислить лишь блок

$$S_{i,j} = \frac{Q_i K_j^T}{\sqrt{d}}$$

по которому даже не посчитать статистики Softmax, ведь мы сможем это сделать лишь после того, как посчитаем все блоки $\{S_{i,j}\}_{j=1}^N$ для каждой пары (K_j, V_j) . Получается, связать вместе FlashAttention и контекстный параллелизм невозможно?

На каждом шаге кольцевого внимания мы не вычисляем конечный результат FlashAttention. Мы каждый раз обновляем статистики глобальные m_t , l_t и выход O_t . Мы получим корректное значение O_t лишь на последнем шаге.

20.2 Зиг-заг кольцевое внимание

У кольцевого внимания есть еще одна проблема. Обычно, в LLM, мы считаем не просто внимание, а маскированное внимание. В таком внимании мы применяем треугольную маску.

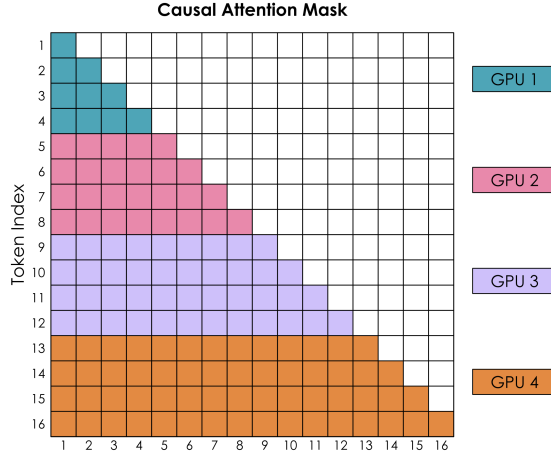


Рис. 35: Разделение ‘зон ответственности’ вычисления матрицы внимания между рангами при контекстном параллелизме. Взято из Ultra-Scale Playbook.

Из-за этого на некоторых рангах лежат блоки Q_i , для которых мы пропускаем вычисление большей части блоков $S_{i,j}$, поскольку они полностью замаскированы.

Из-за этого вычисления становятся несбалансированными между рангами, что приводит к простоям некоторых GPU.

Чтобы это исправить мы можем изменить порядок распределения токенов между рангами так, чтобы каждая нагрузка была равномерно распределена между всеми рангами.

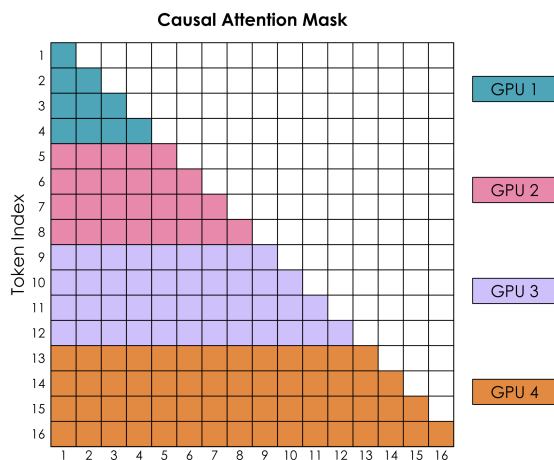


Рис. 36: Распределяем токены так, чтобы выровнять нагрузку на ранги. Взято из Ultra-Scale Playbook.

Такой подход называется **зиг-заг кольцевым вниманием (Zig-Zag Ring Attention)**

21 Конвейерный параллелизм (PP)

21.1 Зачем нужен PP

При использовании тензорного параллелизма мы ограничены в масштабировании размерами одной ноды.

Мы также не можем сильно масштабироваться с использованием CP, поскольку при росте CP увеличивается время вычисления механизма внимания из-за увеличения количества этапов.

Допустим, мы используем $TP = 8$ и $CP = 8$ степень параллелизма. Это позволит распараллелить наши вычисления на 64 GPU.

Мы можем существенно нарастить параллелизм, используя FSDP. При $DP = 512$, $TP = 8$ и $CP = 8$ мы могли бы параллелизовать вычисления на 32768 картах.

Казалось бы, это успех. Если потребуется больше видеокарт, то мы просто будем наращивать степень DP. Но не всё так просто.

TP уже занимает каждую ноду целиком. CP и DP приходится полагаться на более медленную междошовую связь. Для CP требуется выполнять

коммуникации внутри механизма внимания, поэтому желательно, чтобы ноды рангов CP находились рядом.

Получается, что для коммуникаций DP-рангов остаются только самые длинные маршруты. А чем длиннее маршрут коммуникации, тем ниже пропускная способность.

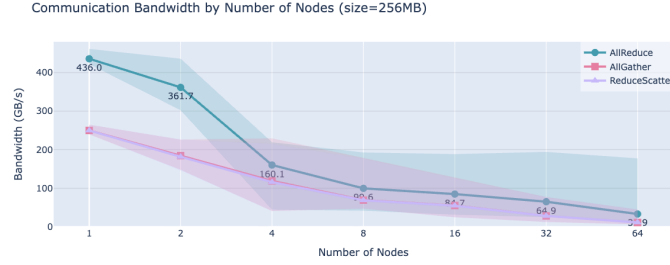


Рис. 37: Пропускная способность межндовой коммуникации в зависимости от количества нод. Взято из Ultra-Scale Playbook.

Это особенно сильно сказывается на ZeRO-3 (FSDP), где для вычисления прямого прохода каждого слоя необходимо выполнить **AllGather**, а для обратного распространения — **AllGather** и **ReduceScatter**.

Использовать ZeRO-2 и ZeRO-3 степени шардирования при таком масштабировании становится слишком дорого по коммуникациям. А при использовании ZeRO-1 из-за реплицирования параметров и градиентов мы довольно скоро упрямся в ограничения по объему памяти.

Решением будет добавить новую степень параллелизма — **конвейерный параллелизм**.

21.2 Стадии конвейерного параллелизма

Конвейерный параллелизм делит модель не в ‘ширину’, как FSDP, а в ‘глубину’.

Пусть модель состоит из последовательности слоёв:

$$f(x) = f_L(f_{L-1}(\dots f_2(f_1(x))))$$

Конвейерный параллелизм разбивает эти слои между несколькими рангами. Например, если есть P **стадий конвейера (pipeline stages)**, то модель можно представить так:

$$f(x) = s_P(s_{P-1}(\dots s_2(s_1(x))))$$

где s_i — блок слоёв, который находится на i -й стадии конвейера. Каждая стадия хранит параметры и градиенты только своих слоёв, но целыми, не разнесенными между рангами, тензорами.

21.3 AFAB

При прямом распространении, первая стадия получает входной батч и считает свою часть модели. После этого она передаёт активации следующей стадии.

Коммуникации между стадиями конвейерного параллелизма обычно — point-to-point операции, то есть **send** и **recv**.

Во время обратного распространения направление коммуникаций меняется. Градиент по входу стадии передаётся назад на предыдущую стадию.

Механизм напоминает конвейер, отсюда и название. Однако, для эффективной работы конвейера необходимо настроить расписание обработки данных различными стадиями.

Самое простое расписание — **AFAB** (All Forward, All Backward). Идея проста: сначала выполняем прямое распространение для всех стадий, а потом — обратное распространение в обратном порядке.

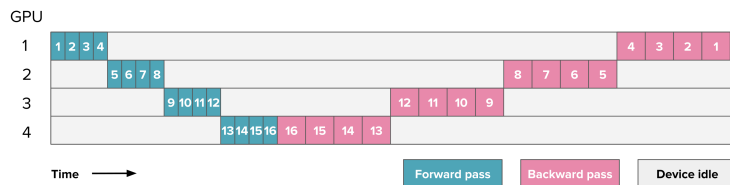


Рис. 38: AFAB. Взято из Ultra-Scale Playbook.

AFAB — крайне неэффективное расписание. Чем больше стадий, тем больше простой каждого ранга. Другая важная проблема: необходимо хранить полные активации всех батчей вплоть до старта обратного распространения.

Отправлять весь батч на пайплайн — достаточно наивное решение. Разделим батч на части — **микробатчи**.

В AFAB мы выполняем стадии последовательно для всего батча. Попробуем обрабатывать стадиями не батч целиком, а микробатч. Как только первый ранг закончит обработку первого микробатча, он одновременно передает полученные активации следующей стадии и начинает обрабатывать второй микробатч.

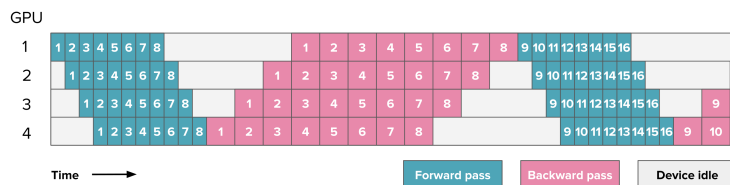


Рис. 39: AFAB по микробатчам. Взято из Ultra-Scale Playbook.

Чем больше микробатчей — тем меньше простой. Поэтому для конвейерного параллелизма выгодно делать микробатч размером в один объект.

21.4 1F1B

Более эффективное расписание, позволяющее эффективнее загрузить ранг — **1F1B (1 Forward 1 Backward)**. Его идея заключается в том, что мы можем начать вычислять обратное распространения не дожидаясь окончания прямого распространения на всех рангах.

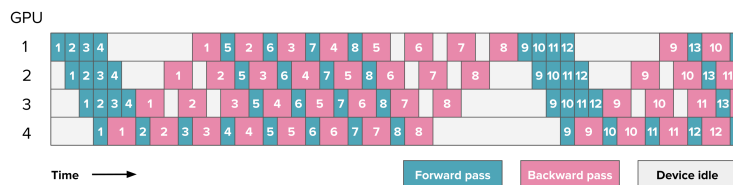


Рис. 40: 1F1В. Взято из Ultra-Scale Playbook.

1F1B позволяет начать вычисления обратного распространения раньше, что позволяет хранить меньше активаций.

Для дальнейшего уменьшения простоя рангов и уменьшения хранимых активаций, можно разбить каждую физическую стадию на несколько виртуальных, строить более эффективные расписания прямого и обратного распространения и прятать коммуникации под вычисления.

Эти идеи реализуют **interleaved pipeline**, **DualPipe pipeline** и **ZeroBubble pipeline**. Но мы не будем их рассматривать.

22 Иерархия параллелизмов

Как мы уже обсуждали, разные параллелизмы имеют разные преимущества и недостатки. Попробуем расположить все параллелизмы в одной иерархии.

Тензорный параллелизм должен располагаться строго внутри одной ноды, поэтому возьмем $TP = 8$.

В контекстном параллелизме каждый этап кольцевого внимания может вычисляться с использованием тензорного параллелизма. Пусть $\text{CP} = 8$. Тогда мы уже можем увеличить общую степень параллелизма до $\text{TP} \cdot \text{CP} = 64$.

Чтобы не было простоя GPU при вычислении нормализации и слёёв Dropout, необходимо использовать параллелизм последовательностей, причем $SP = TP \cdot CP = 64$.

Далее — вычисление FFN-блока. Мы рассматривали как его вычислять с помощью тензорного параллелизма, но его степень ограничена 8. Чтобы решить эту проблему, используется экспертный параллелизм, который, однако, мы можем применять только для параллельного вычисления специальных MoE-блоков, которые мы рассмотрим позднее.

На данный момент в распределенном обучении участвует 64 GPU. Применим конвейерный параллелизм. Объединим несколько слоёв модели в одну стадию конвейера так, чтобы всего было 16 стадий. Тогда $PP = 16$.

В вычислении каждой стадии участвует 64 GPU, а это значит, что общее число GPU в нашем распределенном обучении достигло $TP \cdot CP \cdot PP = 1024$.

Добить нужную степень параллелизма мы можем с помощью DP с ZeRO-1 шардированием. При $DP = 32$ мы достигнем $TP \cdot CP \cdot PP \cdot DP = 32768$ GPU.

Такое количество GPU открывает дорогу к обучению по настоящему больших сетей с триллионами параметров. Но перед этим необходимо подготовить архитектуру модели.